NGUYEN DEVELOPMENT · PHPDBADMIN

Evolutionary Development — phpdbadmin

Timeboxes und Architekturentscheide ADR-001 bis ADR-011

Phase	04 Evolutionary Development
Träger	Nguyen Development (Einzelfirma), Hoang Nguyen · Lizenz MIT
Vorgehen	DSDM/AgilePM — Dokumentation in dokumentationen/, ein PDF je Phase
Stand	29.10.2026, 02:21 · baut auf Commit cab3ec5 + Änderungen dieses Standes auf
Quelle	dokumentationen/ im Repo — dieses PDF wird bei jeder Änderung neu gebaut

Dieses Dossier enthält

1. 04 — Evolutionary Development
2. Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen
3. Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber
4. Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber
5. Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf
6. Anhang E — ADR-005 — SQL-Editor: CodeMirror 6
7. Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel
8. Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund
9. Anhang H — ADR-008 — Anmeldung ohne Passwort
10. Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe
11. Anhang J — ADR-010 — Lizenz MIT
12. Anhang K — ADR-011 — PHP 8.5 als Mindestversion
13. Anhang L — Sicherheitsprüfung N-02 — White-Box nach OWASP
14. Anhang M — Timebox 00 — Fundament
15. Anhang N — Timebox 01 — Katalog und Navigator
16. Anhang O — Timebox 02 — Datenraster und Filter
17. Anhang P — Timebox 03 — SQL-Editor und Befehlspalette → Alpha
18. Anhang Q — Timebox 04 — Bearbeiten mit Vorschau und Rückgängig
19. Anhang R — Timebox 05 — Struktur ändern
20. Anhang S — Timebox 06 — Export und Import im Hintergrund, Statistik
21. Anhang T — TB 07 — Konten, Rechte je Verbindung, Teilen, Protokoll
22. Anhang U — Timebox 08 — Sicherheit, Barrierefreiheit, Mobil, Hilfe, Installation → v1.0
23. Anhang V — Timebox 09 — Server-Verwaltung
24. Anhang W — Timebox 12 — Sitzungen verwalten, SQL-Hilfe mit KI → v1.2

Inhalt

04 — Evolutionary Development
Timeboxes
Architekturentscheide

Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen
Kontext
Entscheid
Optionen
Konsequenzen

Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber
Kontext
Entscheid
Optionen
Konsequenzen

Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber
Kontext
Entscheid
Optionen
Konsequenzen

Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf
Kontext
Entscheid
Optionen
Konsequenzen

Anhang E — ADR-005 — SQL-Editor: CodeMirror 6
Kontext
Entscheid
Optionen
Konsequenzen

Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel
Kontext
Entscheid
Optionen
Konsequenzen

Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund
Kontext
Entscheid
Optionen
Konsequenzen

Anhang H — ADR-008 — Anmeldung ohne Passwort
Kontext
Entscheid
Optionen
Konsequenzen
Nachtrag TB 00 (28.10.2026)

Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe
Kontext
Entscheid
Optionen
Konsequenzen

Anhang J — ADR-010 — Lizenz MIT
Kontext
Entscheid
Optionen
Konsequenzen

Anhang K — ADR-011 — PHP 8.5 als Mindestversion
Kontext
Entscheid
Optionen
Konsequenzen

Anhang L — Sicherheitsprüfung N-02 — White-Box nach OWASP
Behebung
Anhang: Bericht der Prüfung (Stand `58f2f72`, vor der Behebung)

Anhang M — Timebox 00 — Fundament

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Abweichungen von den Wireframes
- Browser-Durchlauf
- Entscheide in der Timebox
- Commits
- Offen
- Gelernt

Anhang N — Timebox 01 — Katalog und Navigator

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Abweichungen von den Wireframes
- Tests
- Commits
- Gelernt

Anhang O — Timebox 02 — Datenraster und Filter

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Abweichungen von den Wireframes
- Tests
- Commits
- Gelernt

Anhang P — Timebox 03 — SQL-Editor und Befehlspalette → Alpha

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Abweichungen von den Wireframes
- Tests
- Commits
- Gelernt

Anhang Q — Timebox 04 — Bearbeiten mit Vorschau und Rückgängig

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Abweichungen von den Wireframes
- Tests
- Commits
- Gelernt

Anhang R — Timebox 05 — Struktur ändern

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Abweichungen von den Wireframes
- Entscheide in der Timebox
- Tests
- Commits
- Offen
- Gelernt

Anhang S — Timebox 06 — Export und Import im Hintergrund, Statistik

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Messwerte
- Abweichungen von den Wireframes
- Tests
- Commits
- Offen
- Nach dem Zusammenführen
- Gelernt

Anhang T — TB 07 — Konten, Rechte je Verbindung, Teilen, Protokoll

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Abweichungen von den Wireframes
- Tests
- Entscheide in der Timebox
- Commits
- Offen
- Gelernt

Anhang U — Timebox 08 — Sicherheit, Barrierefreiheit, Mobil, Hilfe, Installation → v1.0

- Ziel
- Umfang
- Was entstanden ist
- Abnahme v1.0 (Timebox-Plan § 3)
- Tests
- Commits
- Gelernt

Anhang V — Timebox 09 — Server-Verwaltung

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Gestaltungsentscheide (statt Wireframe-Abweichungen)
- Entscheide in der Timebox
- Tests
- Commits
- Offen
- Gelernt

Anhang W — Timebox 12 — Sitzungen verwalten, SQL-Hilfe mit KI → v1.2

- Ziel
- Umfang
- Was entstanden ist
- Abnahmekriterien
- Tests
- Commits
- Gelernt

04 — Evolutionary Development

> **PDF der Phase:** evolutionary-development.pdf — bei jeder Änderung neu gebaut (`python3 dokumentationen/werkzeuge/pdf-bauen.py --nur 04`).

Timebox-Protokolle (`timebox-nn-titel.md`) und Architekturentscheide (`adr/ADR-nnn-titel.md`: Kontext → Optionen → Entscheid → Konsequenzen).

Timeboxes

TB	Inhalt	Anforderungen	Stand
00 Fundament	Starter-Kit bereinigt (kein Passwort, kein Dashboard, kein Profil); Anmeldung mit Code per Mail und Passkey; Einladung; Ersteinrichtung per Konsole mit Link und Passkey-Onboarding; App-Datenbank; Verbindungen mit Test, Verschlüsselung und erlaubten Hosts; Dialekt-Schicht mit Vertragstest auf vier Datenbanken; lokales Prüfskript, Workflow sparsam	A-01 bis A-06, A-09, V-01 bis V-03, V-05, B-01 bis B-03, B-06, B-08	✅ 28.10.2026 — offen: Touch ID von Hand (A-03), erster Lauf des Workflows auf GitHub
01 Katalog und Navigator	Arbeitsbildschirm mit Navigator für vier Datenbanken, «alle Datenbanken», Strukturansicht mit Art je Spalte, Schreibschutz und Umgebung, Kontowarnung, Deutsch und Englisch	O-01, O-04, S-01, V-04, N-06, V-10, O-06, N-04	✅ 29.10.2026 — V-06 mit TB 07; N-04-Matrix noch nie gelaufen
02 Datenraster und Filter	Raster mit Blättern über den Schlüssel, NULL zuletzt, Filtern, Suche, Zählen, Fremdschlüssel in beide Richtungen, Zeilen-Detail, Reiter, Kürzel; erste Seite bei 1 Mio. Zeilen unter 20 ms	D-01, D-02, D-03, D-06, N-01, O-03, O-05, O-09	✅ 29.10.2026 — «?»-Taste von Hand offen
03 SQL-Editor und Befehlspalette	Zerleger je Dialekt, Ausführen mit Zeit- und Zeilengrenze, Abbrechen auf dem Server, SQLite im Arbeitsprozess, Editor mit Vervollständigung, Verlauf, gespeicherte Abfragen, gefährliche Anweisungen nur mit Bestätigung, Befehlspalette ⌘K	Q-01–Q-04, Q-07, Q-09, O-02	✅ 29.10.2026 → **Alpha** — Woche im Alltag offen (Hoang)
04 Bearbeiten mit Vorschau und Rückgängig	Änderungen sammeln, Vorschau aus `pretend()` mit Kaskaden, eine Transaktion auf allen vier, Rückgängig mit Konfliktprüfung, Feld-Editor je Art, Kopieren als TSV/Markdown/INSERT, Schreibschutz auf Zeit	D-04, D-05, D-07, D-09, E-05, N-03, V-11	✅ 29.10.2026
05 Struktur ändern	Tabelle anlegen und ändern mit Vorschau aus `toSql()`, angewendet je Treiber (MySQL/MariaDB Anweisung für Anweisung, PostgreSQL in einer Transaktion, SQLite nach dem Umbauverfahren mit Checks, Triggern und Sichten), umbenennen, kopieren, leeren und löschen nur mit Namen, Datenbank bzw. Schema anlegen und löschen, neue SQLite-Datei; Tests je Treiber mit Elterntabelle und Kindern und mit Fehler mitten in der Folge	S-02–S-05, N-03	✅ 29.10.2026
06 Export und Import	Export als CSV, JSON oder SQL in Paketen über den Schlüssel; Import von CSV mit Zuordnung und Probelauf, SQL-Datei mit eigenem Zerleger; Statistik nach Importen und auf Knopfdruck; Aufträge im Hintergrund, ohne Worker in der Anfrage mit Grenze	E-01, E-02, E-06, B-07	✅ 29.10.2026 — offen: Export einer Abfrage und gefilterter Zeilen
07 Konten, Rechte, Protokoll	Policy je Verbindung (`view`, `write`, `manage`, `share`); Teilen; Verwaltung der Konten und Einladungen; Protokoll mit Filtern, Maskierung und Frist; dazu Favoriten und «zuletzt geöffnet» (V-06)	B-04, A-07, V-09, A-05, A-06, V-06	✅ 29.10.2026 — Verwaltung und Protokoll im Browser geprüft

TB	Inhalt	Anforderungen	Stand
08 Sicherheit, Barrierefreiheit, Mobil, Hilfe, Installation	Sicherheitsprüfung nach OWASP mit Bericht (DSN-Injektion behoben, CSP, erneute Prüfung der Aufträge), Durchlauf 375/768/1440 px, Sprunglink, Hilfe in zwei Sprachen, `composer setup` mit Systemprüfung	N-02, O-07, O-08, N-05, B-05	✅ 29.10.2026 → **v1.0** — offen: axe-Lauf von Hand
09 Server-Verwaltung	Übersicht, Prozesse beenden, Variablen, Konten und Rechte mit Vorschau, Wartung — vier Treiber, Dialekt-Vertrag erweitert, jede Änderung genau ein Eintrag im Protokoll	W-01 bis W-05	✅ 29.10.2026, vorgezogen — Übersicht und Prozesse im Browser geprüft
12 Sitzungen und SQL-Hilfe mit KI	Aktive Sitzungen sehen und beenden; im SQL-Editor «KI fragen» mit eigenem Schlüssel, nur Frage und Schema-Umriss gehen hinaus	A-08, Q-08	✅ 29.10.2026, vorgezogen

Architekturentscheide

Ausformuliert in der Foundation am 28.10.2026 (Kontext → Optionen → Entscheid → Konsequenzen), Grundlage Feasibility und Solution-Architektur:

ADR	Entscheid	Status
ADR-001	Dialekt-Schicht: ein Vertrag, vier Umsetzungen	angenommen
ADR-002	Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber	angenommen
ADR-003	Vorschau aus `toSql()`, Anwenden je Treiber	angenommen
ADR-004	Schreibschutz in Schichten, eine Anweisung je Aufruf	angenommen
ADR-005	SQL-Editor CodeMirror 6	angenommen
ADR-006	Raster TanStack Virtual, Blättern über den Schlüssel	angenommen
ADR-007	Export und Import in Paketen im Hintergrund	angenommen
ADR-008	Anmeldung ohne Passwort	angenommen
ADR-009	App-Datenbank SQLite als Vorgabe	angenommen
ADR-010	Lizenz MIT	angenommen
ADR-011	PHP 8.5 als Mindestversion	angenommen (Hoang 28.10.2026)

Übernahme aus der Feasibility (erledigt in TB 00): Die Proben F-1, F-7 und Teile von F-2 sind Vertragstests der Dialekt-Schicht (`tests/Feature/Dialects`); die Probe-Seiten `routes/probe.php`, `resources/js/pages/probe/*` bleiben lokal hinter der Anmeldung, bis Raster und SQL-Editor sie ersetzen — die Raster-Probe ist seit TB 02 entfernt, die Editor-Probe folgt mit TB 03.

Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-1, F-3)
Betrifft	Katalog, Struktur, SQL-Editor, Server-Verwaltung

Kontext

phpdbadmin soll vier Datenbanken mit derselben Bedienung abdecken (P-01). Laravel 13 liefert Verbindungen, Query Builder, Schema-Grammatiken und Introspektion für alle vier; die Feasibility zeigt, wo es nicht reicht: Check-Constraints, Trigger, Routinen, eigene Typen (`getTypes()` nur PostgreSQL), Prozesse, Rechte, Zeitlimits, Kontorechte. Ausserdem liefert MariaDB ohne Schemaangabe die Tabellen aller Datenbanken des Servers (3'603 statt 1'000).

Optionen

Option	Vorteile	Nachteile
A — eigene Abstraktion für alles	volle Kontrolle	baut nach, was Laravel schon kann und testet; viermal so viel Code
B — Laravel zuerst, Dialekt nur für Unterschiede	wenig eigener Code, Laravel-Updates kommen mit	Grenze zwischen Laravel und Dialekt muss klar sein
C — Paket eines Dritten (z. B. Adminer-Treiber)	viel fertig	fremde Struktur, Lizenzfrage, Bedienkonzept passt nicht

Entscheid

Option B. Ein Vertrag `App\Dialects\Dialect` mit vier Umsetzungen (`MySqlDialect`, `MariaDbDialect` als Ableitung, `PostgresDialect`, `SqliteDialect`). Was Laravel kann, ruft der Dienst direkt; der Dialekt liefert nur, was sich unterscheidet. Der Katalog übergibt **immer** das Schema.

Konsequenzen

- Jede Dialekt-Methode hat einen Vertragstest, der gegen alle vier Herd-Dienste läuft; die Proben aus `tests/Feature/Feasibility` werden dazu umgebaut.
- Neue Datenbanken (SQL Server, R-14) bekommen eine fünfte Umsetzung, die Oberfläche bleibt.
- Typnamen erscheinen im Original und mit einem gemeinsamen Oberbegriff.

Siehe Solution-Architektur · Feasibility

Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-2, F-8)
Betrifft	ConnectionFactory, QueryRunner

Kontext

Zugangsdaten liegen verschlüsselt in der App-Datenbank. Jede Anfrage braucht eine Verbindung mit Zeitzone UTC, Zeitlimit und gegebenenfalls Schreibschutz. Die Feasibility zeigt grosse Unterschiede: MySQL begrenzt nur SELECT und bricht still ab (Ergebnis `1`), MariaDB und PostgreSQL melden 70100 bzw. 57014, SQLite kennt kein Zeitlimit; libpq hält ganze Ergebnisse ausserhalb des PHP-Speichers (+105 MB bei einer Million Zeilen).

Optionen

Option	Vorteile	Nachteile
A — Verbindungen in `config/database.php` eintragen	einfach	Neustart je Verbindung, Geheimnisse in Dateien
B — `DB::build()` je Anfrage aus der App-Datenbank	keine Dateien, Name je Verbindung und Konto, `DB::purge` danach	Verbindungsaufbau je Anfrage (lokal ~1 ms)
C — dauerhafte Verbindungen (Pool)	schneller	PHP-FPM hat keinen gemeinsamen Pool; Zustand zwischen Konten

Entscheid

Option B. `ConnectionFactory` baut je Anfrage mit `timezone`, Zeitlimit (`max_execution_time`, `max_statement_time`, `statement_timeout`) und Optionen. Zeilengrenze je Treiber: MySQL/MariaDB ungepuffert + `KILL QUERY`, PostgreSQL Server-Cursor (`DECLARE … CURSOR`, `FETCH`), SQLite streamt. SQLite-Abfragen aus dem Editor laufen im Arbeitsprozess mit Zeitgrenze. Abbrechen ist eine zweite Anfrage; ein Merker im Cache entscheidet über «abgebrochen», nicht der Fehlercode.

Konsequenzen

- Verbindungsfehler werden je Treiber zugeordnet (MySQL nach Code, PostgreSQL nach Meldungstext).
- Der Arbeitsprozess für SQLite braucht die Warteschlange oder einen Kindprozess; ohne beides gilt die PHP-Laufzeitgrenze.
- Erlaubte Hosts werden vor dem Verbinden geprüft (ADR-004, V-05).

Siehe Solution-Architektur · Feasibility

Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-5)
Betrifft	SchemaEditor, ChangeSet

Kontext

Nichts soll ungesehen passieren (P-02). Die Feasibility belegt: `toSql()` ist auf allen vier genau das, was läuft; `pretend()` liefert für DML eine gute Vorschau, für SQLite-DDL aber eine kaputte. Naiv in einer Transaktion angewendet, wirft Laravel auf MySQL/MariaDB «There is no active transaction» (DDL beendet Transaktionen implizit), und **SQLite löscht beim Umbau einer Elterntabelle alle abhängigen Zeilen** (0 von 2), weil `PRAGMA foreign_keys` in einer Transaktion nicht wirkt.

Optionen

Option	Vorteile	Nachteile
A — alles in einer Transaktion	einfach, scheinbar sicher	auf drei von vier Treibern falsch, auf SQLite Datenverlust
B — Verfahren je Treiber	auf allen vier belegt korrekt (2 von 2 Kindern bleiben)	drei Wege, die Vorschau muss sie erklären
C — Strukturänderungen nur über eigenes SQL	keine Abhängigkeit von Laravel	baut die Grammatiken nach, die SQLite-Umbau schon können

Entscheid

Option B. Vorschau von Strukturänderungen aus `Blueprint::toSql()`, von Datenänderungen aus `pretend()`. Anwenden: MySQL/MariaDB Anweisung für Anweisung ohne Transaktion (Vorschau: «nicht rückrollbar»); PostgreSQL in einer Transaktion; SQLite nach sqlite.org: Fremdschlüssel ausserhalb aus, Umbau in der Transaktion, `foreign_key_check`, Commit, ein. Datenänderungen auf allen vier in einer Transaktion, vorher Sicherung für Rückgängig.

Konsequenzen

- Tests für Strukturänderungen immer mit Elterntabelle und abhängigen Zeilen.
- Bei MySQL/MariaDB meldet die Oberfläche nach einem Fehler, welche Schritte schon gelaufen sind.
- Lektion «SQLite-Tabellenumbau nie in eigener Transaktion» im Vault.

Siehe Solution-Architektur · Feasibility

Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-7)
Betrifft	ReadOnlyGuard, QueryRunner, SQLite-Verbindungen

Kontext

Produktion ist standardmässig schreibgeschützt (V-04), und der SQL-Editor nimmt freies SQL. Ein Schreibschutz über eine Sitzungseinstellung allein lässt sich im Editor abschalten (`SET … READ WRITE`, `PRAGMA query_only = OFF`). Die Feasibility hat einen Wächter mit 5–12 Ausbruchsversuchen je Treiber geprüft: Bestand unverändert.

Optionen

Option	Vorteile	Nachteile
A — nur Lesekonto verlangen	das einzige echte Schloss	viele haben nur ein Konto; lokal unpraktisch
B — Schichten: Lesekonto empfohlen, Nur-lesen-Transaktion je Anweisung, vorbereitet, SQLite nur lesend geöffnet + Authorizer	belegt auf allen vier	jede Anweisung einzeln; Zerleger nötig
C — SQL vor dem Ausführen klassifizieren und Schreibendes ablehnen	ohne Transaktion	Parser-Rennen gegen jede Syntax; Funktionen mit Nebenwirkung

Entscheid

Option B. MySQL/MariaDB: `SET SESSION TRANSACTION READ ONLY` + Transaktion, immer zurückrollen; PostgreSQL: Transaktion + `SET TRANSACTION READ ONLY`, zurückrollen; SQLite: `Pdo\Sqlite::OPEN_READONLY` und `setAuthorizer()` (`ATTACH` nur im erlaubten Ordner). Anweisungen werden auf dem Server zerlegt und einzeln vorbereitet; `unprepared()` ist für Eingaben verboten. Konten mit Superuser- oder Dateirechten werden sichtbar gewarnt (V-10).

Konsequenzen

- Der Zerleger je Dialekt ist Voraussetzung (TB 03, Probe zu Beginn).
- Die Ausbruchsversuche aus `SecurityProbeTest` werden Abnahmetest von V-04.
- PHP 8.5 wird Mindestversion (ADR-011).

Siehe Solution-Architektur · Feasibility

Anhang E — ADR-005 — SQL-Editor: CodeMirror 6

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-4, Freigabe der Pakete durch Hoang)
Betrifft	Oberfläche SQL-Reiter

Kontext

Der Editor braucht Hervorhebung und Vervollständigung je Dialekt aus dem Schema (Q-01). Probeweise installiert: `codemirror` 6.0.2, `@codemirror/lang-sql` 6.10.0 (MIT, 17 Pakete). Belegt: Tabellen aus 1'002 Tabellen in höchstens 2 ms, Aliasse aufgelöst, dialekteigene Schlüsselwörter (`ILIKE` nur PostgreSQL); Chunk 425 KB (138 KB gzip).

Optionen

Option	Vorteile	Nachteile
A — CodeMirror 6	modular, MIT, Dialekte für alle vier, klein genug	eigene Vue-Hülle
B — Monaco	VS-Code-Gefühl	mehrere MB, SQL-Dialekte schwächer, Web-Worker
C — Eigenbau mit Textarea	keine Abhängigkeit	keine Vervollständigung, viel Arbeit

Entscheid

Option A, als eigener Chunk nur auf dem SQL-Reiter geladen.

Konsequenzen

- Sichtprüfung im Browser durch Hoang steht aus (Probe-Seite `/probe/editor`).
- Bei Updates der Pakete Lizenzliste neu erzeugen (ADR-010).

Siehe Solution-Architektur · Feasibility

Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel

Feld	Wert
Status	Angenommen — bestätigt von Hoang am 28.10.2026 (Endlos scrollen statt Seiten zu 50)
Datum	28.10.2026 (Foundation; F-3)
Betrifft	Datenraster

Kontext

Tabellen mit Millionen Zeilen müssen sofort erscheinen (N-01). Gemessen: erste Seite über den Schlüssel 0,1–0,6 ms in der Datenbank, `OFFSET 900000` 18–232 ms; im Browser 281 ms für die erste Seite, davon fast alles Laravel und Herd; im DOM bleiben rund 25 Zeilen.

Optionen

Option	Vorteile	Nachteile
A — eigenes Raster mit `@tanstack/vue-virtual` (MIT, 34 KB Chunk)	volle Kontrolle über Bearbeiten, Zustände, Gestaltung	Raster selbst bauen
B — fertiges Datenraster (AG Grid Community u. a.)	viel fertig	schwer, eigene Bedienlogik, Gestaltung schwer anzugleichen
C — ohne Virtualisierung, Seiten zu 100	einfach	kein flüssiges Scrollen, Seitenwechsel

Entscheid

Option A. Seiten zu 200 Zeilen über den Schlüssel, die nächste vorgeladen; `OFFSET` nur ohne Primärschlüssel; erste Spalte fixiert, waagrecht scrollen nur hier.

Konsequenzen

- Geänderte, neue und zu löschende Zeilen sind Zustand im Browser bis «Anwenden» (D-04).
- Kartenliste unter 768 px (G-07).

Siehe Solution-Architektur · Feasibility

Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-8, F-3)
Betrifft	Transfer

Kontext

Gemessen bei einer Million Zeilen: `lazyById(5000)` 3–3,3 MB auf allen vier; `cursor()` gepuffert 62 MB (MySQL/MariaDB), PostgreSQL +105 MB im Prozess. Import von 100'000 Zeilen in Paketen zu 1'000: 143–502 ms. Nach dem Füllen fehlt MySQL/MariaDB die Statistik — Sortieren über den Index 300 statt 2 ms.

Optionen

Option	Vorteile	Nachteile
A — Pakete über den Schlüssel, im Hintergrund, Datei mit Frist	gleich auf allen vier, wenig Speicher	Warteschlange nötig (ohne Worker synchron mit Grenze)
B — Server-Werkzeuge (`mysqldump`, `pg_dump`)	schnell, vollständig	nicht überall installiert, Versionen müssen passen — nur für Sicherung (E-03)

Entscheid

Option A für Export und Import; Probelauf beim Import in einer zurückgerollten Transaktion; nach eigenen Importen `ANALYZE` (E-06). Server-Werkzeuge nur für Sicherung und Wiederherstellung (E-03, v1.1).

Konsequenzen

- Exportdateien unter `storage/app/private/transfers`, Frist 24 Stunden.
- Tabellen ohne Primärschlüssel exportieren über `cursor()` ungepuffert bzw. Server-Cursor.

Siehe Solution-Architektur · Feasibility

Anhang H — ADR-008 — Anmeldung ohne Passwort

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Vorgabe Hoang; F-6)
Betrifft	Anmeldung, Konten

Kontext

Vorgabe: kein Passwort, nur Passkey und Code per Mail; kein Dashboard, kein Profil. Fortify 1.41 registriert `POST /login` und die Passwortbestätigung unabhängig von den Features; das Feature `passkeys` steht im Starter-Kit mit `confirmPassword: true`. Basalto hat dasselbe mit denselben Versionen gebaut (Code-Login, Abweisung, Software-Authenticator für Tests).

Optionen

Option	Vorteile	Nachteile
A — Fortify nur für Passkeys, eigener Code-Login, Passwort-Endpunkte abweisen	Muster erprobt, Passkeys von Laravel	Fortify-Updates können Routen verschieben (Lektion)
B — Fortify ganz entfernen, Passkeys selbst	keine Überraschungen aus Fortify	WebAuthn selbst prüfen — mehr Risiko

Entscheid

Option A. `Fortify::authenticateUsing()` liefert `null`; Passwortbestätigung durch Bestätigung per Passkey bzw. Code ersetzt; Passwort-Spalten und -Tabellen entfernt; erstes Konto per `phpdbadmin:install`, weitere per Einladung.

Konsequenzen

- Test ruft jede alte Adresse des Starter-Kits auf (404/405).
- Vor jedem `composer update` Routenliste sichern und vergleichen.
- Parameter nie `{passkey}` nennen (globale Bindung von `laravel/passkeys`).

Nachtrag TB 00 (28.10.2026)

Umgesetzt mit einer strengeren Form von Option A: `Fortify::ignoreRoutes()` statt `authenticateUsing()`. Fortify registriert damit **keine** eigenen Routen; `routes/auth.php` trägt die Passkey-Wege der Bibliothek (`Laravel\Passkeys\Http\Controllers`) unter den Namen, die Fortify verwendet hätte. Ein Passwort-Endpunkt existiert gar nicht (`POST /login` → 405). Die Middleware `password.confirm` leitet

auf eine eigene Seite (Bestätigen per Passkey oder Code an die eigene Adresse); jede Anmeldung setzt die Bestätigung. Fortify bleibt nur, weil es die Passkey-Konfiguration (`passkeys.*`) aus `config/fortify.php` ableitet.

Ersteinrichtung (28.10.2026, Wunsch Hoang): Das erste Konto entsteht im Browser oder per Konsole. Im Browser schützt ein Einrichtungsschlüssel, den die App beim ersten Aufruf von `/setup` als Datei auf dem Server ablegt (`storage/app/private/einrichtungsschluessel.txt`, 0600) und nach der Einrichtung löscht — so wird nicht der erste Besucher einer frisch installierten Seite Administrator (T-19). Bewusst nicht gewählt: offene Einrichtung für den ersten Besucher (Übernahme), Freigabe nach Absender-IP (hinter einem Reverse-Proxy kommt jede Anfrage von `127.0.0.1`), Bestätigung per Mail (auf einem frischen Server oft noch nicht eingerichtet). Nach der Einrichtung folgt das Onboarding mit Passkey.

Siehe Solution-Architektur · Feasibility

Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; R-31)
Betrifft	Installation, Betrieb

Kontext

Die App-Datenbank hält Konten, Verbindungen, Verlauf und Protokoll — wenig Daten, wenige gleichzeitige Schreiber. Eine Installation neben fremden Datenbanken soll ohne zusätzlichen Server laufen.

Optionen

Option	Vorteile	Nachteile
A — SQLite als Vorgabe, die anderen drei möglich	keine Einrichtung, Sicherung = eine Datei	Schreibsperren bei vielen Parallelzugriffen (`busy_timeout` , WAL)
B — MariaDB/PostgreSQL vorgeschrieben	robuster bei vielen Konten	zusätzlicher Server nur für phpdbadmin

Entscheid

Option A, mit WAL und `busy_timeout`; die Testsuite läuft mit jeder der vier als App-Datenbank (B-06).

Konsequenzen

- Regeln für Spaltentypen gelten trotzdem für alle vier (Datenmodell § 1).
- Die App-Datenbank erscheint nie als Zielverbindung (R-17).

Siehe Solution-Architektur · Feasibility

Anhang J — ADR-010 — Lizenz MIT

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Entscheid Hoang, R-40; F-9)
Betrifft	Repository, Abhängigkeiten

Kontext

Hoang hat MIT gewählt (Markt überwiegend kostenlos, Schaufenster für Nguyen Development). Geprüft: Composer 101 Pakete MIT/BSD/Apache, `nette/*` wahlweise BSD-3; npm-Laufzeit MIT/ISC/BSD/Apache; neue Pakete MIT.

Optionen

Option	Vorteile	Nachteile
A — MIT	einfach, verbreitet	kein Schutz gegen geschlossene Ableitungen
B — GPL-3	Ableitungen bleiben offen	schreckt Einsatz in Firmen ab

Entscheid

Option A. `LICENSE` mit MIT vor dem ersten öffentlichen Push; Code aus GPL-Projekten (phpMyAdmin, phpPgAdmin, Adminer) wird nicht übernommen.

Konsequenzen

- Lizenzliste vor jedem Release erneut erzeugen.
- Hinweis «nicht verbunden mit phpMyAdmin» in README und Website (F-10).

Siehe Solution-Architektur · Feasibility

Anhang K — ADR-011 — PHP 8.5 als Mindestversion

Feld	Wert
Status	Angenommen (Hoang, 28.10.2026)
Datum	28.10.2026 (Foundation)
Betrifft	Installation, Hosting, `composer.json`

Kontext

Der Schutz von SQLite gegen `ATTACH` fremder Dateien braucht `Pdo\Sqlite::setAuthorizer()`. Geprüft in Herd: PHP 8.4.27 hat `Pdo\Sqlite`, aber keinen Authorizer; PHP 8.5.12 hat ihn. Laravel 13 verlangt nur PHP 8.3, `composer.json` steht auf `^8.3`.

Optionen

Option	Vorteile	Nachteile
A — PHP 8.5 vorschreiben	Schutz vollständig, ein Weg	Hosting mit PHP 8.4 fällt weg
B — PHP 8.4 erlauben, SQLite-`ATTACH` dort per Textprüfung sperren	breiter einsetzbar	Textprüfung ist schwächer; zwei Wege testen
C — PHP 8.4 erlauben, SQLite-Verbindungen dort nur lesend und ohne Editor	einfach	halbe Funktion

Entscheid

Option A (entschieden von Hoang am 28.10.2026). `composer.json` auf `^8.5` in TB 00; Herd läuft bereits mit 8.5.

Konsequenzen

- Die Installationsanleitung nennt PHP 8.5 und `pdo_sqlite`, `pdo_mysql`, `pdo_pgsql`.

Siehe Solution-Architektur · Feasibility

Anhang L — Sicherheitsprüfung N-02 — White-Box nach OWASP

Stand: 29.10.2026 · Abnahme v1.0: «Prüfbericht ohne hohen Befund» — **erreicht nach Behebung** (`cab3ec5`).

Die Prüfung lief als eigener, nur lesender Durchgang über Quellcode, Tests und Proben gegen die lokalen Server (MariaDB, MySQL, PostgreSQL); Grundlage war das Bedrohungsmodell T-01 bis T-19 (Feasibility). Sie fand **einen hohen**, drei mittlere und sieben niedrige Befunde. Der Bericht steht unten unverändert; hier, was daraus wurde.

Behebung

Befund	Schwere	Behebung	Nachweis
H-01 DSN-Injektion über den Datenbanknamen umgeht die erlaubten Hosts	hoch	Namen mit `;` `'` `"` `=` `\` oder Steuerzeichen abgewiesen — am Feld (`ValidatesConnectionSettings`) **und** bei jedem Verbinden (`HostPolicy::guardDatabase` in `ConnectionFactory::address`), also auch für gespeicherte Verbindungen und Datenbanken aus «alle Datenbanken»; Prüfung fail-closed	`HostPolicyTest::test_database_names_that_would_change_the_dsn_are_refused` (7 Fälle), `TestConnectionControllerTest::test_database_name_cannot_smuggle_a_second_host_into_the_dsn`, `…::test_stored_connection_with_such_a_name_never_connects`
M-01 App-Datenbank als Ziel, wenn sie nicht SQLite ist	mittel	gleiche Adresse (Loopback gleich behandelt), gleicher Port, gleicher Name, gleiche Treiberfamilie (MySQL = MariaDB) → `DatabaseNotAllowed`; `phpdbadmin:check` warnt, wenn der Server der App-Datenbank erlaubt ist (Restrisiko MySQL: Lesen über `datenbank.tabelle` mit einem Konto, das Rechte darauf hat)	`HostPolicyTest::test_the_app_database_on_a_server_is_never_a_target`
M-02 Aufträge prüfen Recht und Schreibschutz beim Ausführen nicht erneut; Importer öffnen die Sitzung selbst	mittel	`TransferRunner::authorize()` vor jedem Lauf (Konto aktiv, Recht `write` bzw. `view`, Schreibschutz); Importer und Statistik rufen `TargetConnections::assertWritable()` vor `set session transaction read write`	`TransferRunnerTest::test_import_queued_before_the_connection_became_read_only_writes_nothing` (MariaDB), `…::test_export_of_an_account_that_lost_access_does_not_run`
M-03 Drosselung der Code-Prüfung nur je Sitzung	mittel	zusätzlich je Mail-Adresse (Fingerabdruck): 10 je Minute, 30 je Stunde	`LoginControllerTest::test_code_guesses_are_throttled_per_address_even_with_a_new_session_each_time` (jeder Versuch von anderer IP)

Befund	Schwere	Behebung	Nachweis
N-01 keine Sicherheits-Header, kein CSP	niedrig	`SecurityHeaders`: CSP mit Nonce je Antwort (`script-src 'self' 'nonce-…'`, `frame-ancestors 'none'`, `object-src 'none'`, `base-uri`, `form-action`), `X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy: same-origin`, COOP, Permissions-Policy, HSTS in Produktion über HTTPS. In der Umgebung `local` nur `Report-Only`, weil Laravel Herd ein eigenes Inline-Skript einfügt; Browser-Durchlauf (Raster, SQL-Editor, Export, Konto): keine Verletzung der App selbst	`SecurityHeadersTest`
N-02 Route `storage/{path}` über dem privaten Speicher	niedrig	`'serve' => false` auf der Disk `local`	`route:list` ohne `storage/*`
N-03 keine CSRF-Probe mit echter Middleware	niedrig	Test mit eingeschalteter `PreventRequestForgery`: fremde Herkunft und fehlendes Token → 419	`RequestForgeryTest`
N-04 Einladung meldet in ein bestehendes Konto an	niedrig	Einladung legt nur noch an; gibt es die Adresse schon, Meldung «mit einem Code anmelden»	`InvitationControllerTest::test_an_invitation_never_signs_into_an_existing_account`
N-05 Passwörter im Verlauf im Klartext	niedrig	`Masking::secrets()` im Verlauf (`IDENTIFIED BY`, `PASSWORD`, `SET PASSWORD FOR … =`); gespeicherte Abfragen bleiben bewusst wörtlich (ausdrückliche Handlung)	`MaskingTest::test_secrets_are_masked_and_the_rest_stays_readable`, `SqlControllerTest::test_history_keeps_no_passwords`
N-06 Leser bekommen eine beschreibbare Sitzung (SQLite ganz ohne zweite Schicht)	niedrig	`TargetConnections::open()` öffnet nur lesend, wenn das angemeldete Konto nicht `write` darf	ganze Suite grün
N-07 Abbruch-Merker vor der Besitzprüfung	niedrig	erst prüfen, dann merken	`QueryRunnerTest::test_a_run_of_another_account_cannot_be_cancelled`
I-01 Servermeldungen unmaskiert	Hinweis	Aufträge maskieren jetzt (`Masking::exception`); Raster und Editor zeigen die Meldung des Servers bewusst — sie gehört dem Konto, das die Abfrage stellt	—
I-05 Vorschau ohne Recht «schreiben»	Hinweis	Vorschau verlangt `write`	—
I-07 Systemprüfung ohne Cookie- und Host-Hinweise	Hinweis	`phpdbadmin:check` warnt bei fehlendem `SESSION_SECURE_COOKIE` in Produktion und bei erlaubtem Server der App-Datenbank	—
I-02, I-03, I-04, I-06	Hinweis	offen, bewusst: Entwicklungswerkzeuge nur in `local`; CSV-Export ohne Formelschutz (Daten bleiben unverändert, in der Hilfe zu dokumentieren); erlaubte Hosts ohne Ports und Verbindungsaufbau nur mit Zeitlimit des Treibers; Hilfeseite mit `v-html` nur für mitgelieferten Text	—

Nicht geprüft im ersten Durchgang war die KI-Hilfe (Q-08) und die Sitzungsliste (A-08), die während der Prüfung entstanden: Schlüssel verschlüsselt und `$hidden`, nur Struktur im Umriss (Test mit Attrappe: keine Datenzeile im Auftrag), `Gate::authorize('view')`, Schema gegen die Liste geprüft, Protokolleintrag je Anfrage; Sitzungskennung nur als HMAC im Browser. Ebenso die nach der Prüfung zusammengeführten Timeboxen 05 und 10–11 — sie folgen denselben Mustern (Policy, `withWritable`, Bezeichner-Quoting, Protokoll) und werden vor der Freigabe von v1.1/v1.2 in einem zweiten Durchgang geprüft.

Anhang: Bericht der Prüfung (Stand `58f2f72`, vor der Behebung)

Datum: 29.10.2026 · **Prüfer:** Sicherheitsdurchgang (White-Box, Quellcode + lokale Proben) **Geprüfter Stand:** Arbeitskopie `das Repository`, HEAD **58f2f72** («TB 08: Sprunglink zum Inhalt …»). Der Auftrag nannte b43bf31; der Hauptstrang hat während der Prüfung vier TB-08-Commits nachgelegt (Installationsbefehl, Systemprüfung, Hilfeseite, Sprunglink). Geprüft wurde die Arbeitskopie, also der neuere Stand — die neuen Dateien (`HelpController`, `SystemCheck`, `InstallCommand`) sind mit drin. **Umfang:** `routes/*`, `app/**`, `config/*`, `bootstrap/app.php`, `resources/js/**`, `resources/views/app.blade.php`, `.env.example`, Tests unter `tests/`. Dazu Wegwerf-Proben unter `(Wegwerf-Probe, nicht im Repository)` gegen die Herd-Dienste (MariaDB 3306, MySQL 3307, PostgreSQL 5432). **Keine Änderung im Repository**, keine Migration gegen die App-Datenbank, keine Browser-Werkzeuge. `.env` wurde nicht ausgegeben.

---

1. Kurzfazit

Es gibt einen hohen Befund. Die Abnahme «Prüfbericht nach OWASP ohne hohen Befund» ist damit **noch nicht erreicht**.

- **1 × hoch:** Über den **Datenbanknamen** lassen sich dem Verbindungsaufbau zusätzliche DSN-Parameter unterschieben (`host=`, `port=`). Damit baut phpdbadmin Verbindungen zu **beliebigen Hosts** auf — die erlaubten Hosts (V-05, T-05) sind wirkungslos. Belegt mit einer Probe auf allen drei Serverdialekten; bei PostgreSQL entsteht eine voll nutzbare Sitzung, bei MySQL/MariaDB eine authentifizierte Sitzung mit frei wählbarem Datenbanknamen.
- **3 × mittel:** App-Datenbank nur bei SQLite geschützt (T-17); Schreibschutz und Recht werden beim Ausführen eines Hintergrund-Auftrags nicht erneut geprüft, und die Importer heben die Sitzungs-Schicht selbst auf (T-06); die Drosselung der Code-Prüfung hängt an der Sitzungskennung und ist damit umgehbar (T-01).
- **7 × niedrig**, **7 × Hinweis** — darunter die fehlenden Sicherheits-Header/CSP (zweite Hälfte von T-14) und die fehlende CSRF-Probe mit echter Middleware (T-15).

Alles andere hält: Zugriffskontrolle je Verbindung, Bezeichner-Quoting, Verschlüsselung der Zugangsdaten, Maskierung im Protokoll, Grenzen gegen DoS, keine XSS-Senke für Datenbankinhalte. `composer audit` und `npm audit --omit=dev`: **keine bekannten Lücken**.

---

2. Bedrohungen T-01 bis T-19

Nr.	Status	Beleg (Datei:Zeile · Test)	Lücke
T-01 Konto übernommen	**teilweise**	`routes/auth.php:21–25` (`throttle:login-code`, `throttle:login-verify`), `app/Providers/FortifyServiceProvider.php:39–56`, `app/Services/Auth/LoginCodes.php:17–70` (6 Ziffern, HMAC, einmalig, 5 Fehlversuche), `LoginController.php:88–92` (`Auth::login` + `session()->regenerate()`) · `LoginControllerTest::test_code_signs_in_with_new_session_and_records_login`, `…::test_throttles_code_requests_per_address_after_five_per_minute`, `LoginCodesTest`, `PasskeyLoginControllerTest`	Limiter `login-verify` schlüsselt auf `session()->getId()` — vom Angreifer frei erneuerbar (Befund M-03). Kein Lockout je Adresse.
T-02 Passwort-Endpunkte	**umgesetzt**	`FortifyServiceProvider.php:24` (`Fortify::ignoreRoutes()`), `config/fortify.php:165–169` (nur `Features::passkeys`) · `StarterKitRemovalTest::test_removed_address_is_not_found` (17 Adressen), `…::test_password_login_is_not_accepted`	—
T-03 Zugangsdaten im Browser	**umgesetzt**	`app/Models/DatabaseConnection.php:50` `#[Hidden(['password'])]`, `ConnectionController.php:85–100` (Formular zeigt nur `hasPassword`), `HandleInertiaRequests.php:56–66` · `ConnectionControllerTest::test_overview_never_contains_passwords`, `…::test_edit_page_shows_whether_password_is_stored_but_never_the_password`	—
T-04 Klartext auf der Platte	**umgesetzt**	`DatabaseConnection.php:72` (`'password' => 'encrypted'`), `TargetConnections.php:110` (`Crypt::encryptString` in der Sitzung) · `ConnectionControllerTest::test_stores_server_connection_with_encrypted_password`, `TargetConnectionsTest::test_without_stored_password_opens_only_after_it_was_given_for_the_session`	—

Nr.	Status	Beleg (Datei:Zeile · Test)	Lücke
T-05 SSRF / erlaubte Hosts	**fehlt (umgangen)**	`HostPolicy.php:47–76` (einmal auflösen, geprüfte Adresse verbinden), `ValidatesConnectionSettings.php:63–73` · `HostPolicyTest` (7 Fälle), `ConnectionControllerTest::test_rejects_host_outside_allowed_list`	**H-01:** Der Datenbankname geht ungefiltert in den DSN; `host=` / `port=` darin überschreiben die geprüfte Adresse. Die Prüfung greift nur für das Feld «Host».
T-06 Schreiben trotz Schreibschutz	**teilweise**	`ConnectionFactory.php:33–75` (Init-Befehl, `OPEN_READONLY`, Authorizer), `MySqlDialect.php:212–223`, `PostgresDialect.php:301–319`, `SqliteDialect.php:178–182`, `WriteUnlock.php` (Freigabe an die Sitzung gebunden) · `DialectContractTest::test_read_only_guard_withstands_escape_attempts` (5–12 Versuche je Treiber), `TargetConnectionsTest::test_read_only_connection_rejects_every_write`, `…::test_session_protections_survive_a_reconnect`, `…::test_writable_connection_is_refused_for_read_only_connection`	**M-02:** `TransferRunner::execute()` prüft `read_only` und das Recht nicht erneut; `CsvImporter.php:157`, `SqlImporter.php:117`, `Statistics.php:23` setzen auf der Verbindung aus `open()` selbst `set session transaction read write` — damit fällt die Sitzungs-Schicht (Probe belegt). Für SQLite gibt es ohne `read_only`-Flag gar keine Schicht (N-06).
T-07 Mehrfachanweisungen	**umgesetzt**	`MySqlDialect.php:40` (`ATTR_MULTI_STATEMENTS => false`), `ConnectionFactory` ohne `unprepared()` für Eingaben · `DialectContractTest::test_multiple_statements_in_one_call_are_refused`	SQL-**Import** führt bewusst ganze Skripte aus (`SqlImporter.php:142`, `exec()` je zerlegter Anweisung) — nur mit Recht «schreiben», so gewollt.
T-08 SQLite `ATTACH`	**umgesetzt**	`ConnectionFactory.php:150–175` (`setAuthorizer`, nur erlaubte Ordner, `''` / `:memory:` erlaubt), `HostPolicy.php:78–110` (`realpath`, App-Datenbank nie) · `DialectContractTest::test_sqlite_attach_only_inside_allowed_folder`, `…::test_sqlite_vacuum_and_attach_without_file_are_allowed`, `TargetConnectionsTest::test_session_protections_survive_a_reconnect`, `HostPolicyTest::test_refuses_sqlite_path_outside_allowed_folders`	—

Nr.	Status	Beleg (Datei:Zeile · Test)	Lücke
T-09 Konto mit Dateirechten	**umgesetzt**	`MySqlDialect.php:232–250`, `PostgresDialect.php:328–350` (`rolsuper`, `FILE`, `secure_file_priv`), Anzeige über `account_warnings` (`WorkspaceController.php:133`) · `DialectContractTest::test_warns_about_root_account_but_not_about_read_account`, `ConnectionControllerTest::test_overview_shows_stored_account_warnings_in_the_language_of_the_account`	Warnung, keine Sperre — so im Bedrohungsmodell vorgesehen (Restrisiko «Der Server ist die Grenze»).
T-10 `LOAD DATA LOCAL INFILE`	**umgesetzt**	`MySqlDialect.php:41` (`ATTR_LOCAL_INFILE => false`) · `DialectContractTest::test_mysql_never_uploads_files_of_the_app_server`	—
T-11 Daten in Meldungen/Protokoll	**teilweise**	`bootstrap/app.php:41–52` (QueryException maskiert ins Log, ersetzt den Standardeintrag), `Masking.php` (SQL, Meldungen, Bindungen), `AuditLog.php:28–60`, `ServerPlan.php:50–56` (Passwörter) · `MaskingTest` (Unit), `AuditLogTest`, `DialectServerContractTest::test_preview_masks_the_password_and_quotes_names_that_look_like_sql`	Meldungen des Servers gehen unmaskiert an die Oberfläche (`QueryRunner.php:172`, `TableData.php:461`) — anders als `RowChangeController` (I-01). `query_history.sql` speichert die Anweisung im Klartext (N-05).
T-12 lange Abfragen	**umgesetzt**	`config/phpdbadmin.php` (`statement_timeout_ms`, `editor.timeout_seconds`), `Dialect::setStatementTimeout`, `QueryRunner.php:238–300` (SQLite im Arbeitsprozess, SIGKILL), Abbrechen über eigene Verbindung (`QueryRunner.php:104–135`) · `DialectContractTest::test_statement_timeout_stops_long_query_within_two_seconds`, `…::test_sqlite_says_it_has_no_statement_timeout`, `QueryRunnerTest`	Beim Verbinden mit falschem Protokoll greift `PDO::ATTR_TIMEOUT` nicht (Probe: 120 s, I-04).
T-13 riesige Ergebnisse	**umgesetzt**	`editor.row_limit` / `max_row_limit`, `ResultReader`, `GridQuery.php:75` (`limit` max 500), `RowEditor::MAX_CHANGES = 1000`, Export in Paketen (`TableReader`, Server-Cursor), `transfers.sync_max_rows/_bytes` · `QueryRunnerTest`, `LargeTransferTest`, `GridPerformanceTest`	—

Nr.	Status	Beleg (Datei:Zeile · Test)	Lücke
T-14 XSS über Datenbankinhalte	**teilweise**	Kein `v-html` / `innerHTML` für Datenbankinhalte (einzige Stelle: `resources/js/pages/help/Show.vue:49`, mitgelieferter Markdown-Text, `html_input => strip`), Blade escapt (`app.blade.php:10`)	**Kein CSP** und keine weiteren Sicherheits-Header (N-01) — die zweite Hälfte des Gegenmittels fehlt.
T-15 CSRF	**teilweise**	`Illuminate\Foundation\Http\Middleware\PreventRequestForgery` in der `web`-Gruppe (geprüft über die Middleware-Konfiguration), Cookies verschlüsselt ausser `appearance`, `samesite=lax`	**Kein Test mit echter Middleware** (N-03) — das Bedrohungsmodell verlangt genau das.
T-16 fremde Verbindung	**umgesetzt**	`AppServiceProvider.php:75–90` (`{connection}` → `visibleTo()->findOrFail()` ⇒ 404), `DatabaseConnection.php:120–147`, Policy `view/write/manage/share`, `TransferWorkspaceController::ownTransfer` (404), `SqlController.php:194` (gespeicherte Abfrage), `RowEditor::undo` (nach `user_id` + `connection_id`), `AuditController` nur `can:administrate` · `ConnectionControllerTest::test_returns_404_for_connection_of_other_account`, `…::test_admin_sees_every_shared_connection_but_no_personal_connection_of_others`, `DatabaseConnectionPolicyTest`, `TransferControllerTest`, `SqlControllerTest`, `ImportControllerTest`	—
T-17 phpdbadmin verwaltet sich selbst	**teilweise**	`HostPolicy.php:29–38`: der Schutz greift **nur**, wenn die App-Datenbank SQLite ist	**M-01:** Mit MySQL/MariaDB/PostgreSQL als App-Datenbank ist sie als Zielverbindung erreichbar.
T-18 Probe-Seiten	**umgesetzt**	`routes/` enthält nur `web.php`, `auth.php`, `console.php`; keine `probe.php`, keine `/probe/*`-Route (`php artisan route:list`)	Dev-Werkzeuge der Pakete sind in `local` offen: `_boost/browser-logs` ohne Middleware, Inertia-DevTools ohne Anmeldung (I-02).

Nr.	Status	Beleg (Datei:Zeile · Test)	Lücke
T-19 Ersteinrichtung	**umgesetzt**	`SetupKey.php` (Datei 0600 in `storage/app/private`, Ordner 0700, nie überschrieben, `hash_equals`, nach Gebrauch gelöscht), `FirstSetupController.php:38–75` (nur ohne Konto, `Cache::lock`, `setup.failed` ins Protokoll), `routes/auth.php:33` (`throttle:login-verify`), Konsolenweg mit signiertem Link (`InstallCommand`, `SetupController::isOpen`) · `SetupKeyTest`, `FirstSetupControllerTest`, `SetupControllerTest`, `InstallCommandTest`	—

3. Befunde

HOCH

H-01 · DSN-Injektion über den Datenbanknamen umgeht die erlaubten Hosts (SSRF)

Ort: `app/Http/Requests/Connections/ValidatesConnectionSettings.php:34` (Regel für `database`), `app/Dialects/PostgresDialect.php:34`, `app/Dialects/MySqlDialect.php:58`, `app/Connections/ConnectionFactory.php:33–48`; Senke: `vendor/laravel/framework/src/Illuminate/Database/Connectors/PostgresConnector.php:74` (`dbname='…'`) bzw. `MySqlConnector.php:80–84` (`;dbname=…`).

Szenario (Eingabe → Wirkung). Jedes angemeldete Konto darf eine Verbindung anlegen (`connections.store` ohne weitere Rechte) und «Verbindung testen» aufrufen. Der Host wird gegen `PHPDBADMIN_ALLOWED_HOSTS` geprüft, der **Datenbankname** nur auf `string|max:255`. Er landet unverändert im DSN:

- PostgreSQL: `database = postgres' host='10.1.2.3' port='5432` → `pgsql:host=<geprüft>;dbname='postgres' host='10.1.2.3' port='5432'…` — libpq nimmt den zuletzt gesetzten Parameter.
- MySQL/MariaDB: `database = ;host=10.1.2.3;port=3306;dbname=kundendaten` → `mysql:host=<geprüft>;port=…;dbname=;host=10.1.2.3;port=3306;dbname=kundendaten`.

Belegt (`(Wegwerf-Probe, nicht im Repository)`, `dsn-mysql.php`, `dsn-mysql2.php`, `validierung.php`), erlaubt war jeweils nur `192.0.2.10` (nicht erreichbar):

```
PostgreSQL   VERBUNDEN (0.01s) -> inet_server_addr 127.0.0.1 / current_database postgres
MariaDB      ;host=127.0.0.1;port=9;dbname=x    → (0.00s) [2002] Connection refused
(Kontrolle 192.0.2.10: 2.00s Timeout)
MariaDB      ;host=127.0.0.1;port=3307;dbname=x → (0.00s) [1049] Unknown database 'x'    (echter
MySQL-Server, Auth durchlaufen)
Validierung  beide Nutzlasten gehen unverändert durch TestConnectionRequest::rules()
```

Damit ist T-05/V-05 vollständig aushebelbar: Verbindungen in das interne Netz (auch auf Ports, die der erlaubte Host nie hatte), Port- und Dienst-Erkennung über die unterschiedlichen Fehlermeldungen, und bei PostgreSQL eine voll nutzbare Sitzung inklusive Datenraster, SQL-Editor und Export. Zusätzlich lassen sich

bei PostgreSQL weitere libpq-Parameter setzen (z. B. `sslmode`, `connect_timeout`) — die SSL-Wahl der Oberfläche wird so übergangen. Die eingetragenen Zugangsdaten gehen an den selbst gewählten Host.

Behebung. Den Datenbanknamen (und jedes andere Feld, das in einen DSN wandert) als Bezeichner prüfen, nicht als freien Text — z. B. Regel `regex:/^[^\x00-\x1F;\'"\\\\]+$/u` plus Verbot von `=`; sauberer: den DSN nicht aus Text zusammensetzen, sondern bei PostgreSQL `dbname` über die Verbindungs-URI korrekt quoten bzw. nach dem Aufbau prüfen, dass die Gegenstelle die geprüfte Adresse ist. Empfehlung zusätzlich als zweite Schicht: nach dem Verbinden `inet_server_addr()` bzw. `@@hostname` / `SHOW VARIABLES LIKE 'port'` gegen die geprüfte Adresse halten und sonst trennen. Gleiches Muster für `sqlite_path` bleibt unverändert (dort greift `realpath`).

Test. `ConnectionControllerTest` / `TestConnectionControllerTest`: Datenbankname mit `;host=`, `' host='`, `=`, `;`, NUL und Zeilenumbruch wird mit Fehler am Feld `database` abgewiesen (je Treiber); `TargetConnectionsTest`: `ConnectionFactory::make()` mit solcher Eingabe wirft `ConnectionNotAllowed` statt zu verbinden — als Vertragstest gegen die drei Server (mit erlaubtem Host `192.0.2.10` und injiziertem `127.0.0.1`, wie in der Probe).

MITTEL

M-01 · Die eigene App-Datenbank ist als Zielverbindung erreichbar, sobald sie nicht SQLite ist (T-17)

Ort: `app/Connections/HostPolicy.php:29–38`.

Szenario. `fromConfig()` setzt `appDatabase` nur bei `database.default === 'sqlite'`; geprüft wird dieser Wert ausserdem nur in `resolveSqlitePath()`. Läuft die Installation mit MariaDB/MySQL/PostgreSQL als App-Datenbank (in `.env.example` nur kommentiert, im Betrieb aber eine naheliegende Wahl, und `SystemCheck` unterstützt sie ausdrücklich), legt **jedes** Konto — auch ein Mitglied mit nur einer eigenen Verbindung — eine Verbindung auf `127.0.0.1` zur Datenbank von phpdbadmin an (Host ist in der Vorgabe erlaubt, Zugang in Herd `root` ohne Passwort). Danach: `update users set role = 'admin' where …` über das Datenraster oder den SQL-Editor, Sitzungen stehlen (`sessions`), Protokolleinträge löschen (`audit_log`), verschlüsselte Passwörter aller Verbindungen abziehen. Rechteausweitung bis Administrator und Umgehung des Protokolls.

Behebung. Den Vergleich treiberübergreifend ziehen: beim Speichern **und** bei jedem Verbinden Host (aufgelöste Adresse), Port und Datenbankname gegen `config('database.connections.'.config('database.default'))` halten und ablehnen («Die eigene Datenbank von phpdbadmin ist hier nicht erreichbar.»). Für «alle Datenbanken» zusätzlich beim Schema-/Datenbankwechsel prüfen. Zu H-01 hin wirkt dieselbe Prüfung nur, wenn der DSN nicht mehr manipulierbar ist — beide Befunde gemeinsam beheben.

Test. `TargetConnectionsTest`: mit `DB_CONNECTION=mysql` (bzw. `pgsql`) auf die App-Datenbank zeigende Einstellungen → `ConnectionNotAllowed` mit `ConnectionProblem::APP_DATABASE`; `ConnectionControllerTest`: `store` / `update` antwortet mit Fehler am Feld `database`.

M-02 · Schreibschutz und Recht werden beim Ausführen eines Auftrags nicht erneut geprüft — und der Importer hebt die Sitzungs-Schicht selbst auf

Ort: `app/Services/Transfer/TransferRunner.php:133–175` (`execute()` ohne Rechte-/`read_only`-Prüfung), `app/Services/Transfer/CsvImporter.php:157`, `app/Services/Transfer/SqlImporter.php:117`, `app/Services/Transfer/Statistics.php:23` (`set session transaction read write` auf der Verbindung aus `TargetConnections::open()`), `app/Connections/TargetConnections.php:41`.

Szenario. Geprüft wird nur in der Anfrage (`TransferWorkspaceController::authorizeWrite()`). Der Auftrag läuft später: `RunImport` → `TransferRunner::execute()` → `targets->open()`. Wird die Verbindung zwischen Einreihen und Ausführen auf «schreibgeschützt» gestellt (oder dem Konto das Recht «schreiben» entzogen, oder die Freigabe geteilt/entzogen), läuft der Import trotzdem — `open()` liefert zwar eine schreibgeschützte Sitzung, aber der Importer setzt sie mit `set session transaction read write` selbst zurück. Dass diese Anweisung die Schicht tatsächlich öffnet, ist belegt (`(Wegwerf-Probe, nicht im Repository)`):

```
MariaDB: INSERT ohne Trick abgewiesen — SQLSTATE[25006] 1792
MariaDB: INSERT nach «set session transaction read write» GELUNGEN -> Schutzschicht umgangen
MySQL:   dito
```

Bei PostgreSQL und SQLite hält die Schicht in diesem Fall (kein `read write` für diese Treiber bzw. `OPEN_READONLY`), dort scheitert der Auftrag mit 25006. Das Zeitfenster und die fehlende Nachprüfung sind aus dem Code belegt; ein Ende-zu-Ende-Lauf über die Warteschlange wurde **nicht** gefahren (als «Zeitfenster» zu lesen, nicht als trivial auslösbar).

Behebung. (1) In `TransferRunner::execute()` vor dem Start erneut prüfen: Auftrag gehört zu einem aktiven Konto, Konto hat `write` an der Verbindung, Verbindung ist nicht `read_only` — sonst Auftrag als «abgebrochen» beenden. (2) Schreibende Dienste die Verbindung über `TargetConnections::withWritable()` holen lassen (wirft `ReadOnlyConnection`) statt über `open()` mit anschliessendem `set session transaction read write`; dann braucht es die Anweisung gar nicht. (3) `Statistics::refresh()` genauso.

Test. `TransferRunnerTest`: Auftrag in `Queued`, danach `read_only = true` setzen bzw. Recht entziehen → `execute()` schreibt nichts, Auftrag endet als `cancelled` / `failed`; Vertragstest in `DialectContractTest`: `set session transaction read write` auf einer `readOnly`-Verbindung **ausserhalb** von `runReadOnly` darf keinen Schreibzugriff erlauben (ergänzt die bestehende Probe, die jeden Versuch in `runReadOnly` kapselt und die Lücke deshalb nicht sieht).

M-03 · Drosselung der Code-Prüfung hängt an der Sitzungskennung (T-01)

Ort: `app/Providers/FortifyServiceProvider.php:52–55` (`Limit::perMinute(10)->by('login-verify:'.$request->session()->getId().'|'.$request->ip())`), `app/Models/LoginCode.php:52–58`, `app/Services/Auth/LoginCodes.php:38–70`.

Szenario. Die Anmeldung kennt nur einen Faktor: den sechsstelligen Code. Die Sitzungskennung ist clientseitig beliebig erneuerbar (Cookie löschen) — der Limiter `login-verify` greift damit nicht mehr. Begrenzend bleibt nur `login-code` (5/min je Adresse+IP, 20/min je IP) sowie die 5 Fehlversuche je Code. Rechnung: 5 neue Codes/min × 5 Versuche = 25 Rateversuche/min je Adresse ⇒ ~1'500/h ⇒ ~36'000/Tag gegen 10^6 Möglichkeiten ≈ 3,5 % Trefferwahrscheinlichkeit pro Tag und Adresse, mit mehreren IPs mehr. Ein Konto-Lockout je Adresse gibt es nicht; jeder Versuch ist allerdings im Protokoll (`auth.failed`), und das Opfer bekommt die Mails (auffällig). Gerechnet, nicht durchprobiert.

Behebung. Zweiten Limiter ohne Clienteinfluss ergänzen, z. B. `Limit::perMinute(10)->by('login-verify:'.Fingerprint::ofEmail($emailAusDerSitzung))` plus `Limit::perHour(30)` je Adresse; zusätzlich nach n Fehlversuchen je Adresse eine Sperrminute (wie Basalto) oder den Code auf 8 Zeichen aus `SetupKey::ALPHABET` verlängern.

Test. `LoginControllerTest`: nach fünf falschen Codes mit jeweils **neuer Sitzung** (`$this->flushSession()` / neuer Testclient) antwortet `login.code.verify` mit 429 bzw. der Drosselmeldung; Protokolleintrag je Versuch bleibt.

NIEDRIG

N-01 · Keine Sicherheits-Header, kein CSP (zweite Hälfte von T-14)

Ort: `bootstrap/app.php:24–34` (keine Header-Middleware), Antwort von `https://phpdbadmin.test/login` geprüft: nur `cache-control`, `link`, `set-cookie` — **kein** `content-security-policy`, `x-frame-options` / `frame-ancestors`, `referrer-policy`, `x-content-type-options`, `strict-transport-security`.

Szenario. Eine künftige XSS-Senke (z. B. die neue Hilfeseite mit `v-html`, oder eine Bibliothek) wirkt ohne CSP voll; die Oberfläche lässt sich in einen fremden Rahmen setzen (Clickjacking auf «Importieren», «Anwenden», «Prozess beenden»); `Referer` trägt Verbindungs-, Schema- und Tabellennamen nach draussen. Die Cookies sind in Ordnung (`secure; httponly; samesite=lax` — `session.secure = null` erbt bei HTTPS automatisch `secure`).

Behebung. Eine kleine Middleware in der `web`-Gruppe: `Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect-src 'self'; frame-ancestors 'none'; base-uri 'none'; object-src 'none'` (das Inline-Skript in `app.blade.php:12–22` braucht eine Nonce oder wandert in eine Datei), dazu `Referrer-Policy: same-origin`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` und in Produktion HSTS. `SESSION_SECURE_COOKIE=true` in die Produktionsdoku.

Test. Feature-Test «jede Antwort trägt CSP und `frame-ancestors 'none'`» auf `/login` und einer Arbeitsbildschirm-Seite; Sichtprüfung der Konsole auf CSP-Verstösse.

N-02 · `'serve' => true` öffnet signierte Lese- und Schreibrouten auf `storage/app/private`

Ort: `config/filesystems.php:33–39`; daraus registriert Laravel `GET storage/{path}` (`storage.local`) und `PUT storage/{path}` (`storage.local.upload`) — **ohne** `web` / `auth`, nur mit signierter URL (`vendor/.../Filesystem/ServeFile.php:27`, `ReceiveFile.php:27`).

Szenario. phpdbadmin nutzt `temporaryUrl()` nirgends (geprüft); die Routen sind reine Angriffsfläche über dem privaten Speicher — Export-Dateien fremder Konten, hochgeladene Importdateien und `einrichtungsschluessel.txt`. Wer je eine signierte URL in die Hand bekommt (Protokoll, Proxy-Log, geleakter `APP_KEY`), liest und **schreibt** ohne Anmeldung in diesen Ordner.

Behebung. `'serve' => false` für die Disk `local` setzen (Downloads laufen über `TransferController::download` mit Rechteprüfung).

Test. Feature-Test: `GET /storage/transfers/…` ohne Signatur → 404, und die Route existiert nach der Änderung nicht mehr (`Route::has('storage.local')` ist falsch).

N-03 · Keine CSRF-Probe mit echter Middleware (T-15)

Ort: `tests/` — kein Treffer für `PreventRequestForgery` / `csrf`; Laravels Testkern schaltet die Prüfung in Tests ab, die bestehenden Tests belegen T-15 also nicht.

Szenario. Rein eine Nachweislücke: ein künftiges `withoutMiddleware`, eine `except`-Liste oder eine API-Route fiele niemandem auf. Die Middleware ist derzeit vorhanden und aktiv.

Behebung/Test. Ein Test, der die Middleware ausdrücklich einschaltet (`$this->withMiddleware(PreventRequestForgery::class)` bzw. `Route::middleware('web')`-Test) und für eine schreibende Route (`POST /c/{id}/unlock`, `POST /c/{id}/transfers/import`) ohne Token 419 erwartet, mit Token 2xx/302.

N-04 · Einladung meldet in ein bestehendes Konto an

Ort: `app/Http/Controllers/Auth/InvitationController.php:44–57` (`Invitation::findByToken` → `User::firstOrCreate` → `Auth::login`).

Szenario. Beim Einladen verhindert `Rule::unique('users','email')` doppelte Konten, und `resend` prüft zusätzlich. `accept` prüft es nicht: Eine Einladung, die vor dem Entstehen des Kontos verschickt wurde (z. B. zwei Einladungen hintereinander, oder Konto über `phpdbadmin:install`/zweite Einladung entstanden), bleibt bis zu 7 Tage ein **zweiter Anmeldeweg ohne Code und ohne Passkey** — wer die Mail liest (altes Postfach, weitergeleitete Mail, Archiv), landet im bestehenden Konto, inklusive Administratorrechten, wenn die Einladung so lautete.

Behebung. In `accept()` ablehnen, wenn zur Adresse schon ein Konto existiert, dessen `created_at` vor `invitation.created_at` liegt (bzw. generell, wenn `User` existiert und `invitation.accepted_at === null` nicht aus demselben Vorgang stammt) — und offene Einladungen beim Anlegen eines Kontos derselben Adresse automatisch verfallen lassen.

Test. `InvitationControllerTest`: Einladung ausstellen, Konto derselben Adresse anderweitig anlegen, dann Einladung annehmen → Fehlermeldung, `assertGuest()`.

N-05 · Verlauf und gespeicherte Abfragen halten die Anweisung im Klartext

Ort: `app/Http/Controllers/Workspace/SqlController.php:142–150` (`QueryHistoryEntry` mit `$result['sql']`), `…:178–185` (`SavedQuery`), `app/Models/QueryHistoryEntry.php:35–38` (90 Tage).

Szenario. Wer im SQL-Editor `CREATE USER 'app'@'%' IDENTIFIED BY 'geheim'` oder ein `INSERT` mit Personendaten ausführt, hat den Wert 90 Tage im Klartext in der App-Datenbank — im Protokoll wird genau das maskiert (`AuditLog` → `Masking::sql`). Ein Zugriff auf die App-Datenbank (siehe M-01) oder ein Backup davon gibt diese Werte her. Nur das eigene Konto sieht den Verlauf (geprüft), Administratoren nicht.

Behebung. Entweder bewusst dokumentieren (der Verlauf soll wieder ausführbar sein) **und** die Aufbewahrung kurz halten, oder den Verlauf maskiert speichern und den Originaltext nur in der Sitzung halten. Mindestens: im Datenmodell/Doku als «enthält Werte» kennzeichnen, damit Backups entsprechend behandelt werden.

Test. `SqlControllerTest`: Nach `CREATE USER … IDENTIFIED BY …` enthält `query_history.sql` kein Passwort (bei Variante «maskiert speichern»).

N-06 · SQLite: Schreibschutz-Schicht nur am Flag, nicht am Recht

Ort: `app/Connections/TargetConnections.php:41`.

Szenario. `open()` öffnet die Datei nur dann mit `OPEN_READONLY`, wenn die **Verbindung** als schreibgeschützt markiert ist — unabhängig davon, ob das Konto «schreiben» darf. Für ein Konto mit nur «ansehen» hängt der Schutz damit allein an den Controllern (die ihn konsequent prüfen, geprüft); bei SQLite gibt es keine zweite Schicht wie die Nur-Lese-Transaktion der Server. Kein Ausbruchsweg gefunden — ein freies `WHERE` (`TableData.php:213`) kann in SQLite nichts schreiben.

Behebung. `open()` zusätzlich read-only öffnen, wenn das Konto an dieser Verbindung kein `write` hat (Gate abfragen bzw. Recht übergeben).

Test. `TargetConnectionsTest`: Konto mit Recht «ansehen» → `open()` auf einer nicht schreibgeschützten SQLite-Verbindung weist `insert` ab.

N-07 · Abbruch-Merker wird vor der Besitzprüfung gesetzt

Ort: `app/Services/Sql/QueryRunner.php:106–118`.

Szenario. `cancel()` schreibt `sql-abbruch:<runId>` in den Cache, **bevor** geprüft wird, ob der Lauf dem Konto gehört. Wer eine fremde Laufkennung kennt, lässt den fremden Lauf als «abgebrochen» enden (weitere Anweisungen werden übersprungen). Die Kennung ist `crypto.randomUUID()` (`resources/js/pages/workspace/Sql.vue:126–130`), also praktisch nicht zu erraten; im Fallback (kein `crypto`) ist sie aus Zeit und `Math.random()` gebaut.

Behebung. Erst den Lauf aus dem Cache lesen und Konto sowie Verbindung prüfen, dann den Merker setzen; Fallback-Kennung serverseitig vergeben.

Test. `SqlControllerTest`: zweites Konto ruft `sql/cancel` mit der Laufkennung des ersten → kein Merker, Lauf des ersten Kontos bleibt «ok».

HINWEIS

- **I-01 · Servermeldungen unmaskiert in der Oberfläche.** `QueryRunner.php:172` und `TableData.php:461` geben die Meldung des Servers unverändert zurück (z. B. «Duplicate entry 'kunde@example.ch'»), während `RowChangeController.php:140` sie über `Masking::exception()` schickt. Die Meldung sieht nur, wer die Verbindung ohnehin lesen darf — Auswirkung gering, aber uneinheitlich zu T-11. Einheitlich maskieren oder in der Doku festhalten, warum hier nicht.
- **I-02 · Dev-Werkzeuge in `local`.** `POST /_boost/browser-logs` hat **keine** Middleware (kein `web`, kein `auth`) und schreibt beliebigen Text in den Log-Kanal `browser` (Log-Verschmutzung, Platzverbrauch); `GET /_inertia/devtools/entries` ist in `local` ohne Anmeldung lesbar (`vendor/inertiajs/inertia-laravel/src/DevTools/Http/Authorize.php:26`) und enthält aufgezeichnete Props. Beides nur in `local` bzw. nur mit Dev-Abhängigkeiten (Boost ist `require-dev`), aber auf einem geteilten Netz (Herd im WLAN) offen. `inertia.devtools.enabled => false` ausserhalb der eigenen Maschine bzw. ein Gate setzen.
- **I-03 · CSV-Export neutralisiert Formelzeichen nicht.** `Values::csv` / `CsvWriter` schreiben `=`, `+`, `-`, `@` am Feldanfang unverändert (Tabellenprogramme führen das aus); das Protokoll-CSV tut es korrekt (`AuditController.php:168`). Für Datenexporte ist das die richtige Wahl (Round-Trip, `RoundTripTest`) — in der Hilfe/Doku erwähnen.
- **I-04 · Erlaubte Hosts kennen keine Ports; Verbindungsaufbau ohne harte Zeitgrenze.** Mit erlaubtem `127.0.0.1` ist jeder lokale Port erreichbar, die Fehlermeldungen unterscheiden Dienste (Oracle für einen Port-Scan). Zeigt man den MySQL-Treiber auf einen PostgreSQL-Port, hängt der Aufbau 120 s (`PDO::ATTR_TIMEOUT` greift nicht; Probe `dsn-mysql2.php`) — eine Anfrage blockiert so einen FPM-Arbeiter. Erlaubte Hosts optional mit Port notieren (`host:port`) und den Aufbau zusätzlich hart begrenzen.
- **I-05 · `changes/preview` ohne Recht «schreiben».** `routes/web.php:76` → `RowChangeController::preview` prüft nur `view` (die Vorschau führt über `pretend()` nichts aus, liest aber Kaskaden-Zählungen). Konsistenz: `Gate::authorize('write')` auch hier.
- **I-06 · Hilfeseite mit `v-html` + AttributesExtension.** `HelpController.php:23` erzeugt HTML aus `resources/help/*.md` (`html_input => strip`, `allow_unsafe_links => false`) und `resources/js/pages/help/Show.vue:49` rendert es mit `v-html`. Heute mitgelieferter Text, also unkritisch; die `AttributesExtension` erlaubt aber Attribute (`{onclick=…}`) — wer die Datei pflegt, kann unabsichtlich XSS einbauen. Entweder Attribute auf eine Positivliste begrenzen oder den Text zu Komponenten rendern.
- **I-07 · Systemprüfung deckt zwei Konfigurationspunkte nicht ab.** `SystemCheck` prüft APP_KEY, APP_DEBUG in Produktion, HTTPS, Schreibrechte, Mail, Worker, Scheduler — nicht aber `PHPDBADMIN_ALLOWED_HOSTS` (Vorgabe `127.0.0.1,::1,localhost`; leer gesetzt heisst «nichts erlaubt», zu

weit gesetzt heisst «alles erlaubt») und nicht `SESSION_SECURE_COOKIE` / `APP_URL`-Schema in Produktion. Je eine Zeile in `SystemCheck::run()`.

---

4. Was gut ist

- **Zugriffskontrolle.** `{connection}` wird vor der Routenbindung auf das Sichtbare eingeschränkt (404 statt 403), Policy `view ⊂ write ⊂ manage`, `share` nur für Administratoren. Fremde Aufträge, gespeicherte Abfragen, Verlauf, Rückgängig-Schnappschüsse und Passkeys sind je Konto geschlossen — jeder dieser Wege hat einen Test mit zweitem Konto. Admin-Routen hängen an `can:administrate`; die Policy verhindert, dass sich die letzte Administratorin selbst aussperrt.
- **Injection.** Bezeichner laufen durchgehend über die Grammatik bzw. `quoteIdentifier()` (mit dem bewussten Verzicht auf `wrap()` dort, wo dessen Zerlegen gefährlich wäre — gut begründet in `BaseDialect`), Werte immer gebunden. Rechte, Ebenen, Wartungsarten und Privilegien kommen aus festen Listen (`allowedPrivileges()` wirft, statt SQL zu bauen); Konto- und Hostnamen sind regex-geprüft und gequotet. `TrustedSql` macht jede bewusste Rohstelle auffindbar — genau das, was eine Prüfung braucht.
- **Zugangsdaten.** `encrypted`-Cast, `#[Hidden]`, Passwort nie als Prop (Test über die ausgelieferte Antwort), Sitzungspasswort verschlüsselt, Hintergrundaufträge tragen nur die Kennung, Passwort in Vorschau/Protokoll maskiert (`ServerPlan::mask`, `unsetEventDispatcher()` vor Anweisungen mit Passwort).
- **Schreibschutz in Schichten** mit echten Ausbruchsproben je Treiber (inkl. `commit`, `do $$…$$`, `nextval`, `pragma query_only`), Freigabe auf Zeit an die Sitzung gebunden, mit Pflichtgrund und im Protokoll; Produktionsbestätigung serverseitig über `Rule::in([$connection->name])`.
- **Maskierung und Protokoll.** Der Reporter in `bootstrap/app.php` ersetzt den Standard-Logeintrag für `QueryException` vollständig — Bindungen landen nicht im Log. Jede schreibende Aktion hat einen Eintrag, auch die gescheiterte, mit Zeilenzahl und ohne Werte; IP und fremde Adressen nur als HMAC; Aufbewahrung über `model:prune`.
- **Grenzen.** Zeitlimit je Anweisung, SQLite im Arbeitsprozess mit SIGKILL, Zeilengrenzen (ungepuffert + `KILL QUERY`, Server-Cursor), Export/Import in Paketen, Grenzen ohne Worker mit erklärender Meldung statt Timeout, Upload-Grösse und Endungen geprüft, Dateien unter `storage/app/private/transfers/<ULID>/source.<ext>` mit festem Namen (kein Pfaddurchstieg), Download nur über den Besitzer-Pfad.
- **Anmeldung.** Kein Passwort-Endpunkt (strenger als geplant: `Fortify::ignoreRoutes()`), gleiche Antwort für unbekannte Adressen und Mail erst nach der Antwort, Code als HMAC, einmalig per bedingtem Update (Rennen), Sitzung neu nach jeder Anmeldung, gesperrte Konten fliegen mit der nächsten Anfrage raus und auch über Passkey nicht herein, Ersteinrichtung über eine 0600-Datei auf dem Server.
- **Abhängigkeiten.** `composer audit`: keine Advisories; `npm audit --omit=dev`: 0 Lücken.

---

5. Proben (Wegwerf, `(Wegwerf-Probe, nicht im Repository)`)

Datei	Zweck	Ergebnis
`dsn-injektion.php`	`host=`-Injektion über den Datenbanknamen, alle drei Serverdialekte	PostgreSQL voll verbunden zu `127.0.0.1` trotz Freigabe nur für `192.0.2.10`
`dsn-mysql.php`, `dsn-mysql2.php`	Varianten für MySQL/MariaDB, Gegenprobe mit totem Port	Host und Port überschrieben (`Connection refused` in 0,00 s statt 2 s Timeout), Datenbankname frei wählbar

Datei	Zweck	Ergebnis
`validierung.php`	gehen die Nutzlasten durch `TestConnectionRequest::rules()`?	ja, unverändert
`schreibschutz.php`	hält die Sitzungs-Schicht gegen `set session transaction read write`?	nein (MySQL und MariaDB), INSERT gelingt

Ausgeführte Tests (unverändert, nur gelesen): `HostPolicyTest`, `StarterKitRemovalTest`, `DatabaseConnectionPolicyTest`, `MaskingTest` (66 Tests, grün) und `TargetConnectionsTest` (21 Tests, grün), je mit `PHPDBADMIN_TEST_DATABASE=phpdbadmin_test_pruefung`.

Aufräumen: Die Schreibschutz-Probe hat auf MariaDB und MySQL die Datenbank `phpdbadmin_n02_probe` angelegt; sie ist nach der Prüfung wieder entfernt (`aufraeumen.php`). **Im Repository wurde nichts geändert** — die Änderungen, die `git status` am Ende der Prüfung zeigt (KI-Hilfe im SQL-Editor, Sitzungsliste im Konto, `ai_api_key`), stammen aus der laufenden Arbeit des Hauptstrangs in derselben Arbeitskopie.

6. Nicht geprüft

- **Arbeit des Hauptstrangs, die während der Prüfung entstand** (unversioniert): `app/Http/Controllers/Workspace/SqlAssistController.php`, `app/Services/Ai/*`, `app/Http/Controllers/Account/{AiKeyController,SessionController}.php`, `app/Services/Auth/ActiveSessions.php`, Migration `…add_ai_api_key_to_users_table`. Erster Blick ohne Befund: Schlüssel mit `encrypted`-Cast und in `$hidden` (`app/Models/User.php:55,78`), `Gate::authorize('view', …)` und `Rule::in($this->catalog->schemas(…))` im Assistenten, nur Struktur im Umriss (`SchemaOutline`), Eintrag `sql.assist` im Protokoll. **Vor v1.0 eigens zu prüfen:** der ausgehende Aufruf zum KI-Anbieter liegt ausserhalb der erlaubten Hosts (eigene Regel nötig), der Schlüssel liegt in der App-Datenbank (siehe M-01), und es ist zu belegen, dass nie Zeilenwerte mitgehen (Q-08).
- **Laufzeitprüfung in der Oberfläche** (keine Browser-Werkzeuge erlaubt): CSP-Verstösse in der Konsole, Clickjacking-Probe, Verhalten der Drosselung im echten Anmeldefluss.
- **Ende-zu-Ende-Lauf von M-02** über eine echte Warteschlange (Mechanismus belegt, Zeitfenster aus dem Code abgeleitet).
- **Gesamter Testlauf** (`composer ci:local`): nur die fünf oben genannten Suiten wurden gefahren, um die Arbeitskopie des Hauptstrangs nicht zu blockieren.

Anhang M — Timebox 00 — Fundament

Stand: 28.10.2026 · ✅ abgeschlossen — gestartet auf Hoangs «Ja, TB 00 starten», gebaut in fünf Scheiben nach dem Timebox-Plan § 2; während Scheibe 5 auf Hoangs Wunsch ergänzt um **Ersteinrichtung und Passkey-Onboarding**.

Ziel

Ein sauberes Fundament ohne Starter-Kit-Reste: Anmeldung ohne Passwort, Verbindungen zu vier Datenbanken anlegen und testen, eine Dialekt-Schicht mit Vertragstests — geprüft gegen MySQL 8.4, MariaDB 10.11, PostgreSQL 18 und SQLite.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Starter-Kit bereinigen	A-01, A-04, A-05, B-01	✅
2 Anmeldung	A-02, A-03, A-06, A-09	✅ (A-03: Handtest mit Touch ID offen, Hoang)
3 Verbindungen	V-01, V-02, V-03, V-05, B-06	✅
4 Dialekt-Schicht	B-02 (+ Grundlage V-04, V-10)	✅
5 Qualität und Abnahme	B-03, B-08	✅ lokal; Workflow noch nie auf GitHub gelaufen (kein Remote)
Zusatz: Ersteinrichtung	A-06, A-03	✅ (Wunsch Hoang während Scheibe 5)

Wireframe-Bereiche 1 (Anmeldung), 2 (Code), 3 (Verbindungen), 4 (Verbindung anlegen), 13 (Konto).

Was entstanden ist

Anmeldung ohne Passwort (ADR-008, Nachtrag). Fortify registriert keine eigenen Routen mehr (`Fortify::ignoreRoutes()`); `routes/auth.php` trägt nur die Passkey-Wege der Bibliothek. `POST /login` antwortet 405 — einen Passwort-Endpunkt gibt es nicht. Code per Mail: sechs Ziffern, HMAC-Prüfsummen für Code, Adresse und IP, zehn Minuten, einmalig (bedingtes Update gegen zwei gleichzeitige Einlösungen), Sperre nach fünf Fehlversuchen, benannte Limiter je Adresse und IP, gleiche Antwort für unbekannte Adressen, die Mail geht erst nach der Antwort hinaus (`defer`), die Sitzung wird neu erzeugt. Der Code steht bewusst nicht im Betreff (Sperrbildschirm). Die Passwortbestätigung ist ersetzt durch «Bestätigen» per Passkey oder Code an die eigene Adresse; jede Anmeldung gilt als Bestätigung. Passkeys anlegen, umbenennen, entfernen im Konto. Protokoll: Anmeldungen (mit Weg: Code, Passkey, Einladung, Einrichtung), Fehlversuche (Adresse nur als Prüfsumme), Abmelden, Passkey-Änderungen.

Ersteinrichtung und Onboarding. Ohne jedes Konto zeigt `/login` die Ersteinrichtung statt einer Anmeldung ohne Ausweg. **Im Browser** (Hauptweg, Hoangs zweiter Wunsch: «nicht jeder hat ein Terminal»): `/setup` legt einen Einrichtungsschlüssel als Datei auf dem Server ab (`storage/app/private/einrichtungsschluessel.txt`, 0600, nie überschrieben, nach der Einrichtung gelöscht); wer die Installation hochgeladen hat, liest ihn über Dateimanager, FTP oder Editor und gibt ihn mit Name und E-Mail ein — ein zufälliger Besucher kann so nicht Administrator werden (T-19). **Per Konsole** (Alternative): `php artisan phpdbadmin:install` legt genau ein Administratorkonto an und gibt einen signierten Link aus (30 Minuten, gilt nur bis zur ersten Anmeldung; erst ein Knopf meldet an, damit Link-Vorschauen ihn nicht verbrauchen). Die erste Anmeldung gelingt so auch, wenn der Server noch keine Mail

versenden kann; solange die Ersteinrichtung offen ist, stellt der Befehl einen neuen Link aus. Wer sich ohne Passkey anmeldet (Code, Einladung, Einrichtungslink), landet im Onboarding «Passkey jetzt anlegen»; das erste Konto sieht die Variante «Ersteinrichtung» mit den nächsten Schritten. A-06 ist in CR-001 Fassung 5 entsprechend angepasst.

Verbindungen. Tabellen `connections` (ULID, Passwort mit `encrypted`-Cast) und `connection_user` nach dem Datenmodell. Anlegen, bearbeiten, entfernen für vier Treiber; Produktion schreibgeschützt vorgegeben; Passwort nie im Browser (die Seite erfährt nur, ob eines gespeichert ist), leeres Feld behält das gespeicherte, «nicht speichern» möglich. «Verbindung testen» mit Prüfschritten (Host erlaubt → Server erreichbar → Anmeldung → Rechte des Kontos), Fehler in Klartext je Treiber, Meldung des Servers zum Aufklappen. `HostPolicy`: Namen, Platzhalter, Adressen, Netze; einmal auflösen und zur geprüften Adresse verbinden; SQLite nur in erlaubten Ordnern über `realpath()`, die App-Datenbank nie. `{connection}` löst nur sichtbare Verbindungen auf — fremde sind 404 (T-16).

Dialekt-Schicht (B-02). Vertrag `App\Dialects\Dialect` mit Basis über Laravels Introspektion (Schema immer ausdrücklich) und vier Umsetzungen: Konfiguration (UTC, Zeitlimit beim Verbinden, SSL, Schreibschutz beim Öffnen bzw. für die Sitzung), Version, Fehlerzuordnung, Schemas, Tabellen und Sichten mit Zeilenschätzung, Check-Constraints und Trigger, Zeitlimit je Anweisung, Schreibschutz-Wächter (eine Anweisung je Aufruf in einer Nur-lesen-Transaktion, immer zurückgerollt), Kontorechte. `ConnectionFactory`: keine Mehrfachanweisungen, kein `LOCAL INFILE`, SQLite-`ATTACH` nur in erlaubten Ordnern (Authorizer).

Gestaltung. Palette aus `werkzeuge/kontraste.py` im Stylesheet (hell und dunkel), Inter und JetBrains Mono beim Build heruntergeladen und mit ausgeliefert (G-02), Wortmarke (G-06), Umgebungszeichen ○ ◐ ● mit Wort (G-03).

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
A-01	alte Adressen 404/405, Spalten weg	`StarterKitRemovalTest`: 17 Adressen 404, `POST /login` 405, keine Spalten `password`, `two_factor_*`, keine Tabelle `password_reset_tokens`
A-02	Erfolg, Ablauf, Wiederverwendung, falscher Code, Drosselung, unbekannte Adresse, Sitzung neu	`LoginCodesTest` (6), `LoginControllerTest` (14) — Sitzungs-ID vor/nach über das Cookie
A-03	Zeremonie mit virtuellem Authenticator; einmal Touch ID	`PasskeyLoginControllerTest`, `PasskeyManagementTest`, `ConfirmControllerTest` mit dem Software-Authenticator (aus Basalto); **Touch ID von Hand: offen (Hoang)**
A-04	`/dashboard` 404, Anmeldung führt zur Übersicht	`StarterKitRemovalTest`; Anmeldung mit Passkey → Übersicht, ohne Passkey → Onboarding
A-05	`/settings/profile` 404, Kontomenü	Test + Browser-Durchlauf (Menü Sicherheit, Darstellung, Abmelden)
A-06	Einladung annehmen, abgelaufene ablehnen; Befehl legt genau ein Admin-Konto an; falscher Schlüssel legt kein Konto an	`InvitationControllerTest` (7), `InstallCommandTest` (5), `InviteCommandTest` (3), `SetupControllerTest` (5), `FirstSetupControllerTest` (6), `SetupKeyTest` (4); im Browser mit falschem und richtigem Schlüssel durchgespielt
A-09	Eintrag mit Zeit, Konto bzw. Adresse (gehasht), IP-Prüfsumme	Tests prüfen `auth.login`, `auth.failed` (ohne Klartext-Adresse), `auth.logout`
V-01	je Treiber ein Test gegen den echten Dienst; Fehler am Feld	`ConnectionTesterTest` (Herd-Dienste), `ConnectionControllerTest`

Anforderung	Kriterium (CR-001)	Nachweis
V-02	je Fehlerart ein Test mit erzwungenem Fehler	nicht erreichbar, Zugang verweigert (ohne Passwort in der Meldung), Datenbank fehlt — je Server; SSL (PostgreSQL), Host nicht erlaubt, Datei keine Datenbank
V-03	kein Passwort in Props und JSON; Spalte verschlüsselt	Übersicht, Bearbeiten, Test-Endpunkt; Rohwert ≠ Klartext
V-05	nicht erlaubter Host und Pfad (auch `..`, Verknüpfung)	`HostPolicyTest` (21 Fälle aus 5 Tests), Controller am Feld; im Browser `~/Development/../../../etc/hosts` abgewiesen
B-01	Routen und Seiten ohne Starter-Kit-Reste	Routenliste: nur eigene und Passkey-Wege; Seitenordner `auth`, `connections`, `account`, `onboarding`, `probe`
B-02	Vertragstest gegen alle vier Umsetzungen	`DialectContractTest`: 12 Tests, 32 Fälle über die Treiber — alle grün
B-03	lokales Skript fährt alle vier; Workflow sparsam	`composer ci:local`; Workflow mit `paths-ignore`, `concurrency`, `timeout-minutes`, Matrix wöchentlich — **noch nie auf GitHub gelaufen**
B-06	Suite läuft mit jeder App-Datenbank	`composer test:app-db`: grün mit SQLite, MariaDB, MySQL, PostgreSQL als App-Datenbank
B-08	Prüfkette in der Reihenfolge der CI	`vp check` → `vue-tsc` → Pint → Larastan (Stufe 7) → PHPUnit

Stand der Suite: 221 Tests, 209 bestanden, 12 übersprungen (die schweren Raster- und Export-Proben mit 1 Mio. Zeilen, bewusst nur auf Wunsch).

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
01 Anmeldung	Sprachwahl Deutsch · English fehlt	Englisch ist nicht Teil von TB 00
01 Anmeldung	neu: Ersteinrichtung, wenn es kein Konto gibt	Wunsch Hoang; noch nicht im Generator
02 Code	«Code abgelaufen» hat keine eigene Ansicht; eine Meldung für falsch und abgelaufen	verrät nichts über den Zustand eines Codes
02 Code	Hinweis ergänzt: «…sofern es dazu ein Konto gibt»	ehrlich bei gleicher Antwort für unbekannte Adressen
03 Verbindungen	ohne «Zuletzt geöffnet», Favoriten, Befehlspalette, Klick in den Arbeitsbildschirm	kommen mit ihren Timeboxes (V-06, TB 01, TB 11)
04 Verbindung anlegen	SSL ohne «Zertifikat prüfen»	verbunden wird zur geprüften Adresse (V-05); Prüfen des Zertifikats braucht eine eigene Anforderung
13 Konto	ohne Sitzungen und Dichte	A-08 (TB 04), G-04 mit dem Raster (TB 01)
13 Konto	Kontomenü ohne «Verwaltung»	TB 07
neu	Ersteinrichtung (Link) und Onboarding «Passkey jetzt anlegen»	Wunsch Hoang; Wireframes dazu noch nicht im Generator

Browser-Durchlauf

Gegen einen gebauten Stand in einem Herd-Workspace (der eingebaute Browser erreicht den Vite-Dev-Server nicht): Bereiche 1–4 und 13 bei 1440 und 375 px, hell und dunkel, Anmeldung per Code mit echter Mail in Herd, Verbindung zu MariaDB anlegen und testen, abgewiesener Pfad mit `..`, Ersteinrichtung auf einer Installation ohne Konto. Gefunden und behoben: Adressen brachen mitten im Wort, Codefeld zu klein,

Hinweis ohne Mail-Symbol und Ablaufzeit, Kopf «Verbindung testen» brach um, **Warnsymbol fehlte wegen eines vergessenen Imports** (jetzt meldet `vue-tsc` unbekannte Komponenten), «· Konto» ohne Datenbank, SQLite-Platzhalter mit einem hier nicht erlaubten Ordner, leeres Passwort galt als «gespeichert».

Entscheide in der Timebox

Entscheid	Begründung
`Fortify::ignoreRoutes()` statt `authenticateUsing()`	kein Passwort-Endpunkt statt eines abweisenden (ADR-008, Nachtrag)
Route `login.code.show` statt `login.code.form`	Wayfinder hängt verschachtelte Routen an die Funktion; `form` überschrieb die Formular-Variante und machte das Formular zum GET
kein «angemeldet bleiben» (`remember_token` entfernt)	Datenmodell § 1; Sitzungsdauer genügt
Probe-Seiten bleiben lokal hinter der Anmeldung	Sichtprüfung des Editors durch Hoang offen; Raster (TB 01) und SQL-Editor (TB 05) ersetzen sie
Proben F-1, F-7 und Teile von F-2 → Vertragstests	Probe-Dateien entfernt; Abbrechen, DDL-Vorschau, Raster und Export bleiben Proben bis zu ihren Timeboxes
Lesekonto `phpdbadmin_lesen` und Datenbanken `phpdbadmin_test`, `phpdbadmin_app_test` auf den lokalen Herd-Servern	Vertragstest «warnt nicht» (V-10) und Suite mit App-Datenbank auf den Servern

Commits

`083e3c9` Scheiben 1+2 · `b04de27` Scheibe 3 · `ceab0e8` Scheibe 4 · `2bea65c` Scheibe 5 (Prüfskript, Workflow, Funde) · `95aa78c` Ersteinrichtung und Onboarding · `7336538` Protokoll · `bfb091c` symfony/mime · `fc75729` Ersteinrichtung im Browser.

Offen

- **Touch ID von Hand** (A-03): einmal einen Passkey auf dem Mac anlegen und damit anmelden — Hoang.
- **Workflow auf GitHub:** noch kein Remote; der erste Push zeigt, ob beide Aufträge laufen.
- ~~**Sicherheitshinweis** `symfony/mime` (CVE-2026-45070, mittel)~~ — erledigt 28.10.2026 auf Hoangs Freigabe: 8.1.7 → 8.1.10, `composer audit` ohne Hinweise, Prüfkette auf allen vier App-Datenbanken grün, Testmail über Herd mit Umlauten im Betreff angekommen.
- Wireframes für Ersteinrichtung und Onboarding im Generator nachziehen — in Arbeit auf einem eigenen Zweig; dort fehlt noch die Einrichtung im Browser mit Einrichtungsschlüssel (`fc75729`).
- Sichtprüfung der Gestaltung durch Hoang am eigenen Bildschirm (G-01 bis G-08).

Gelernt

- Textersetzungen per Skript immer mit Prüfung, ob die Stelle gefunden wurde — der Formatierer bricht Zeilen um, und eine stille Nicht-Ersetzung liess ein Symbol verschwinden (Lektion im Vault).
- `vue-tsc` meldet unbekannte Komponenten erst mit `checkUnknownComponents` (Lektion im Vault).
- Wayfinder: Routennamen nie auf `.form` enden lassen (Lektion im Vault).

- Der eingebaute Browser erreicht den Vite-Dev-Server nicht; für Sichtprüfungen einen Herd-Workspace mit Build nutzen und dessen `artisan dev` anhalten (Projekt-Memory).

Anhang N — Timebox 01 — Katalog und Navigator

Stand: 29.10.2026 · ✅ abgeschlossen bis auf V-06 (Favoriten, «zuletzt geöffnet»), das mit TB 07 im selben Zweig gebaut wird, weil beides an `connection_user` hängt. Gestartet auf Hoangs «nächste tb»; ab dem Abend auf seinen Auftrag «mache alle TB durch» ohne Rückfragen weitergeführt.

Ziel

Eine Verbindung öffnen und darin arbeiten können: Arbeitsbildschirm mit Navigator für alle vier Datenbanken, Struktur einer Tabelle in denselben Begriffen, Schreibschutz und Umgebung sichtbar, Konten mit zu vielen Rechten gewarnt — auf Deutsch und Englisch.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Arbeitsbildschirm öffnen	V-04, V-10	✅ Kopf mit Umgebung (Farbe, Zeichen, Wort), roter Streifen bei Produktion, Titel «PROD · …», Kontowarnung; Passwort je Sitzung, Fehlerkarte statt Fehlerseite
2 Navigator	O-01	✅ Datenbank → Schema → Gruppen (Tabellen, Sichten, Routinen, Trigger, Ereignisse, Sequenzen) mit Filter «/»; 1'000 Tabellen in einem Test
Zusatz «Alle Datenbanken anzeigen»	V-01	✅ Wunsch Hoang; Fehler aus seinem Durchlauf (Endlosschleife) behoben
3 Strukturansicht	S-01, O-04	✅ Wireframe 08 bei 1440 und 375 px
4 Favoriten, zuletzt geöffnet	V-06	⏳ im Zweig von TB 07
5 Deutsch und Englisch	N-06, O-06	✅ alle Texte über das Sprachsystem, Test über alle Schlüssel
6 Abnahme	N-04, O-04	✅ O-04 Bildschirmfotos; N-04 Untergrenzen in der Wochen-Matrix (noch nie gelaufen, kein Remote)

Wireframe-Bereiche 3 (Verbindungen), 5 (Arbeitsbildschirm), 8 (Struktur).

Was entstanden ist

Arbeitsbildschirm (`/c/{verbindung}`, `/c/{verbindung}/s/{schema}`). `TargetConnections` öffnet Zielverbindungen je Anfrage mit Zeitlimit (15 s) und Schreibschutz, wenn die Verbindung schreibgeschützt ist; jede lesende Anweisung läuft zusätzlich im Wächter des Dialekts. Fehler der Zielverbindung werden zur Fehlerkarte mit Ursache und nächstem Schritt, fehlende Passwörter («nicht speichern») fragt eine eigene Seite einmal je Sitzung ab. Fehler der eigenen App-Datenbank bleiben Fehler der App (früher als «Verbindung gescheitert» getarnt — im Durchlauf von TB 02 gefunden).

Alle Datenbanken (Wunsch Hoang). Ohne Datenbanknamen zeigt MySQL/MariaDB alle Datenbanken als Schemas, PostgreSQL öffnet `postgres` und bietet die übrigen Datenbanken im Navigator an (`?db=`). Der Fehler aus Hoangs Durchlauf («Maximum call stack size … Infinite recursion») lag in Laravels `DatabaseManager`: zwei Zielverbindungen mit gleichem Namen trennten sich gegenseitig, der Neuaufbau traf das falsche Objekt. Zielverbindungen stehen seither in keinem globalen Register; der Neuverbinder erneuert genau dieses Objekt und richtet Schreibschutz, Authorizer und Zeitlimit wieder ein. Gegenprobe: Auf dem alten Stand hängen die neuen Tests in der Schleife.

Struktur (`…/o/{objekt}/structure`). `Structure` liest Spalten mit Typ im Original und einer **Art**, die in allen vier Datenbanken gleich heisst (`ColumnKind`: Zahl, Text, Auswahl, Wahrheitswert, Datum und Zeit, JSON, Binär, Kennung, Geometrie, Liste, Sonstiges), Indizes, Fremdschlüssel in beide Richtungen, Checks, Trigger und DDL. Gleichmacher für O-04: MariaDB-JSON (Alias für `longtext` mit `json_valid`) gilt als JSON, eigene Aufzählungstypen in PostgreSQL als Auswahl, Vorgaben als SQL-Literal (`'offen'` statt `offen` bzw. `'offen'::character varying`), MariaDBs Text «NULL» als keine Vorgabe.

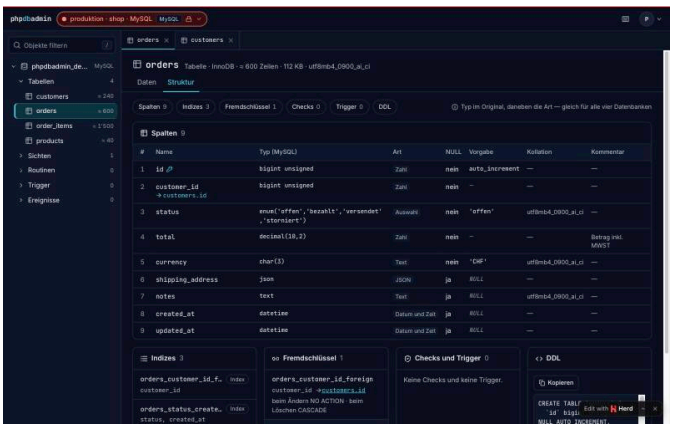
Deutsch und Englisch (N-06, O-06). Der deutsche Text ist der Schlüssel im Code (`__('…')`, `t('…')`, `tc('…')`), `lang/en.json` liefert Englisch (britisch). Sprache aus dem Konto, sonst aus dem Browser, sonst Deutsch; Mails in der Sprache des Kontos. Die Oberfläche bekommt die Übersetzungen einmal je Sprache (Once-Prop). Sprachwahl im Konto unter «Darstellung». Alle bestehenden Seiten hat ein Helfer in einer eigenen Arbeitskopie umgestellt (Zweig zusammengeführt, 322 Zeilen ohne Abweichung).

Abnahmekriterien

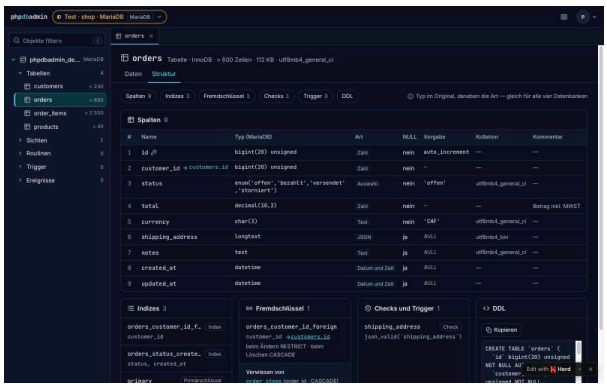
Anforderung	Kriterium (CR-001)	Nachweis
V-04	schreibgeschützte Verbindung lehnt jede schreibende Anweisung ab, auch Ausbruchsversuche	`DialectContractTest` (Wächter je Treiber), `TargetConnectionsTest` 19 Tests: Schreibschutz übersteht einen Neuaufbau (Laravel verbindet bei SQLSTATE 25006 neu)
V-10	Herd-`root` warnt, Lesekonto nicht	`DialectContractTest` gegen root und `phpdbadmin_lesen` auf MySQL, MariaDB, PostgreSQL
O-01	1'000 Tabellen flüssig	`WorkspaceControllerTest::test_navigator_with_a_thousand_tables_stays_fast` (unter 1 s inkl. Seite)
O-04	dieselbe Tabelle in allen vier Datenbanken	Bildschirmfotos unten; `StructureTest` 17 Tests (Art, Vorgaben, Fremdschlüssel beidseitig je Treiber)
S-01	Beispieltabelle mit allen Merkmalen je Treiber	`StructureTest` (Kommentar, Aufzählung, JSON, Standardwerte, eindeutiger Index, Check, Trigger, Sicht), `ObjectControllerTest`
N-06 / O-06	Sprachdateien vollständig ohne Rückfall	`LanguageFilesTest` (5): jeder Schlüssel mit englischer Zeile, keine verwaisten, gleiche Platzhalter, keine dynamischen Schlüssel, Feldnamen beider Sprachen; `LocaleControllerTest` (5)
N-04	Untergrenzen	Wochen-Matrix «untergrenzen» (MariaDB 10.6, MySQL 8.0, PostgreSQL 14) — **noch nie gelaufen** (kein Remote); lokal nur aktuelle Versionen

O-04: dieselbe Tabelle `orders` in allen vier Datenbanken

MySQL 8.4 (Produktion, schreibgeschützt)

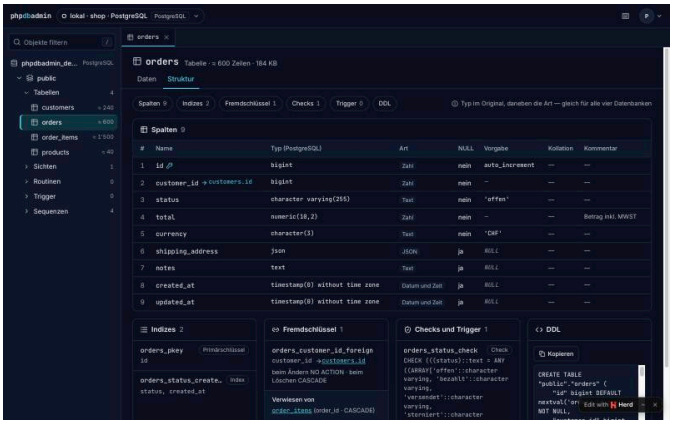
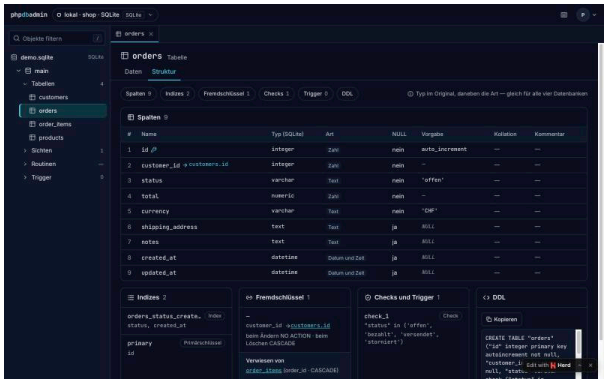


MariaDB 10.11 (Test)



PostgreSQL 18 (lokal)

SQLite (lokal)

MySQL 8.4 (Produktion, schreibgeschützt)	MariaDB 10.11 (Test)
	

Gleich: Aufbau, Begriffe, Art je Spalte, Vorgaben als Literal, Fremdschlüssel beidseitig. Erklärte Unterschiede: PostgreSQL legt Laravels `enum()` als `varchar` mit Check an (Art «Text», der Check steht daneben), SQLite kennt weder Zeilenschätzung noch Kollation noch Namen für Fremdschlüssel.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
08 Kopf	Grösse als eine Zahl statt «Daten · Indizes»	Laravel liefert die Gesamtgrösse; getrennte Werte bräuchten je Treiber eigene Abfragen — später, falls gewünscht
08 Knöpfe	«Tabelle ändern», «Exportieren», «Im Editor öffnen» fehlen	kommen mit TB 05, TB 06, TB 03
08 Reiter	«Beziehungen» und «SQL» fehlen	Beziehungsdiagramm S-07 (TB 10), SQL-Editor (TB 03)
05 Kopf	Suchfeld ⌘K fehlt	Befehlspalette O-02 (TB 03)

Tests

Neu oder erweitert: `StructureTest` (17), `ColumnKindTest` (25), `ObjectControllerTest` (Struktur-Teil), `TargetConnectionsTest` (+3, unabhängige Verbindungen, Schlüssel je Datenbank), `WorkspaceControllerTest` (+1, Ablauf aus Hoangs Durchlauf), `DialectContractTest` (+1, VACUUM trotz ATTACH-Wächter), `LanguageFilesTest` (5), `LocaleControllerTest` (5) und die Englisch-Tests des Übersetzungs-Helfers. Zielserver-Tests laufen über `PHPDBADMIN_TEST_DATABASE` auch parallel in mehreren Arbeitskopien.

Commits

`d4766fa` Scheiben 1+2 · `0842694` Dialekt · `777ce66` alle Datenbanken · `a35ad37` Endlosschleife · `3dd2e9b` Struktur lesen · `884bf8e` Testdatenbank einstellbar · `dcf9e21` Sprachgrundlage · `6db5d35` Strukturansicht · `6a72d92` · `d2227f5` VACUUM · `558b4ae` Vorgaben, Chip · `c503ced`–`7f4c944` Sprache (Helfer) · `80fed1d` Merge · `8069824` Untergrenzen · `1c8f969` MariaDB-Vorgabe.

Gelernt

- Laravels `DB::build()` mit gleichem Namen ersetzt die vorhandene Verbindung; `DB::reconnect()` erneuert das Objekt im Register, nicht das in der Hand. Zielverbindungen deshalb ausserhalb des Registers bauen (Lektion im Vault).
- «Connection refused» hält Laravel für eine verlorene Verbindung und versucht neu — ohne Neuverbinder ein `LostConnectionException`. Sofort verbinden, dann kommt der Fehler als PDO-Fehler.
- Ein leeres PHP-Array wird zu JSON `[]`, und `[].sort` ist in JavaScript eine Methode: Zuordnungen an die Oberfläche als Objekt senden.
- Symfony schickt in Tests `Accept-Language: en-us` mit — mit Sprachwahl aus dem Browser kippen sonst alle deutschen Erwartungen.

Anhang O — Timebox 02 — Datenraster und Filter

Stand: 29.10.2026 · ✅ abgeschlossen — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen gebaut.

Ziel

Die Daten jeder Tabelle und Sicht schnell und in allen vier Datenbanken gleich sehen: blättern, sortieren, filtern, suchen, zählen, Fremdschlüsseln folgen — auch bei einer Million Zeilen.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Datendienst	D-01, D-02, D-03, D-06	✅ `TableData`, `RowDetail`, `Cells` — 52 Tests über vier Treiber
2 Raster und Zeilen-Detail	D-01–D-03, D-06, O-09	✅ Wireframes 05 und 07 (Detail lesend; Bearbeiten mit TB 04)
3 Reiter und Kürzel	O-03, O-05	✅
Messung	N-01	✅ unter 20 ms für die erste Seite

Was entstanden ist

Datendienst. Blättern über den Primärschlüssel (zusammengesetzt ausgeschrieben, weil nicht jede Version Zeilenwerte vergleicht), sonst über den Versatz; der Primärschlüssel ist immer letztes Sortierkriterium. **NULL steht in beiden Richtungen zuletzt** — ausdrücklich über `case when … is null`, nicht nach Gewohnheit des Treibers. Geführte Filter (=, ≠, <, ≤, >, ≥, enthält, enthält nicht, beginnt, endet, ist NULL, ist nicht NULL) mit und/oder; alle Werte gebunden, `%` und `_` im Wert als Zeichen (Escape `!`). «enthält» ohne Gross-/Kleinschreibung: PostgreSQL `ILIKE` auf den Text der Spalte, MySQL/MariaDB nach Kollation (meist auch ohne Akzente), SQLite über eine Unicode-Faltfunktion (`phpdbadmin_fold`), die die `ConnectionFactory` registriert — sonst fände «MÜLLER» nicht «Müller». Freies WHERE als Klammerausdruck im Schreibschutz-Wächter mit Zeitlimit; Suche über alle Textspalten. Zählen genau bis `PHPDBADMIN_EXACT_COUNT_LIMIT` (50'000), darüber die Schätzung des Servers, gefiltert auf grossen Tabellen offen mit «genau zählen». Werte gleich in allen vier: Wahrheitswerte `true`/`false` (MySQL liefert 1/0), Binärdaten als Grösse und Anfang, lange Texte gekürzt, Ganzzahlen über 2^53 als Text, PostgreSQL-`bytea` aus dem Strom gelesen. Fremdschlüssel bekommen die Beschriftung der Zielzeile (bevorzugt `name`, `title`, `email` …). `TrustedSql` kennzeichnet die zwei Stellen mit absichtlich rohem SQL.

Raster (`/c/{verbindung}/s/{schema}/o/{objekt}`). Erste Seite kommt mit der Seite, weitere beim Scrollen (`rows`), dazu `count`, `row` und `view`. Virtualisiert (TanStack Virtual), Sortieren per Klick (Umschalt fügt hinzu), Breite ziehen, Spalten ausblenden — beides je Konto gemerkt (`table_views`). Tastatur: Pfeile, Pos1/Ende, Bild auf/ab, Eingabe öffnet die Zeile. Filterleiste mit Chips, freiem WHERE, Suche und Zeilenzahl. Zeilen-Detail als Seitenblatt mit «verweist auf … öffnen» und «wird verwiesen von … 3 Zeilen zeigen». Telefon: Karten mit «alle Felder».

Zustände (O-09). Leere Tabelle, kein Treffer mit «Filter zurücksetzen», abgelehntes WHERE mit Meldung des Servers, Zeitlimit, abgerissene Verbindung und abgelaufene Sitzung beim Nachladen — je mit Ursache und nächstem Schritt.

Reiter (O-03). Jede geöffnete Tabelle oder Sicht wird ein Reiter mit ihrer letzten Adresse samt Filtern; gespeichert im Browser je Verbindung, übersteht das Neuladen. **Kürzel (O-05):** Übersicht über «?» oder das Tastatur-Symbol.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
D-01	Tests je Treiber mit Sortierung inkl. NULL, Reihenfolge ausdrücklich	`TableDataTest`: NULL zuletzt in beiden Richtungen, absteigend = Umkehrung von aufsteigend, Blättern ohne Lücken und Doppel (einfacher und zusammengesetzter Schlüssel, ohne Schlüssel), stabiler Versatz
D-02	je Treiber, mit Umlauten und Werten, die wie SQL aussehen	`x' or '1'='1`, `'; drop table people; --`, `%`, `_`, «müller»/«MÜLLER»/«jürg», und/oder, NULL, freies WHERE inkl. `1 = 1;` `delete from people` (nichts gelöscht)
D-03	Test je Treiber	Beschriftung des Fremdschlüssels, verweisende Zeilen gezählt, Zeile mit Verweisen in beide Richtungen
D-06	Test mit Schwelle	geschätzt über der Grenze, genau darunter, gefiltert offen, «genau zählen»; SQLite zählt mangels Schätzung immer genau
N-01	Messung mit erzeugten Daten je Treiber	`GridPerformanceTest` (`PHPDBADMIN_LEISTUNG=1`), Zahlen unten
O-03	Durchlauf	im Browser: zwei Reiter, Neuladen, beide da
O-05	Durchlauf nur mit Tastatur	Raster mit Pfeilen, Eingabe, Escape; Kürzel-Dialog über das Symbol. Die Taste «?» selbst liess sich mit dem Browser-Werkzeug nicht auslösen (es erzeugt kein `keydown`) — **von Hand offen**
O-09	leere Tabelle, SQL-Fehler, getrennte Verbindung, abgelaufene Sitzung	`ObjectControllerTest` (ungültiges WHERE → Fehlerkarte, 401/404/422/503 der JSON-Antworten), Oberfläche erklärt jede Ursache

N-01: erste Seite einer Tabelle mit einer Million Zeilen (lokal, Herd)

Fall	MySQL 8.4	MariaDB 10.11	PostgreSQL 18	SQLite
ohne Sortierung	6,5 ms	6 ms	17,1 ms	6,6 ms
`created_at` absteigend (Index)	1,5 ms	2,3 ms	2,4 ms	5,9 ms
Filter `category = 42` (ohne Index)	4,5 ms	4,5 ms	20,3 ms	35,5 ms
Suche über Textspalten	188,5 ms	138 ms	75,4 ms	156,7 ms

Zeit im Dienst inklusive Spalten, Zählung bzw. Schätzung und Beschriftungen.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
05 Kopf	«Zeile einfügen», «Exportieren», Änderungsleiste, «Rückgängig machen» fehlen	Bearbeiten TB 04, Export TB 06
05 Navigator	«Gespeichert» und «Verlauf» fehlen	SQL-Editor TB 03
07 Detail	lesend; «Duplizieren», «Zum Löschen markieren», «Zeile kopieren als» fehlen	TB 04 (D-04, E-05)
05 Raster	Spaltenkopf mit Typ statt Kurzform «decimal(10,2)» gekürzt	Breite der Spalte; der volle Typ steht in der Struktur

Tests

`TableDataTest` (52 = 13 × 4 Treiber), `ObjectControllerTest` (13), `GridPerformanceTest` (4, nur auf Wunsch). Suite: 393 Tests, 381 bestanden, 12 übersprungen (schwere Proben).

Commits

`88a95d7` Datendienst · `b2842e7` Raster und Detail · `88dec69` Reiter und Kürzel · `8af0231` Messung N-01. Die Raster-Probe der Feasibility (`/probe/raster`) ist entfernt.

Gelernt

- `rsync -R --delete` löscht auch in den Elternordnern des relativen Pfads — im Prüf-Workspace verschwanden so `database/database.sqlite`, Factories und Seeder (nur dort; Lektion im Vault).
- Larastan verlangt für rohes SQL ein Literal; statt die Prüfung zu umgehen, markiert `TrustedSql` die bewussten Stellen sichtbar.
- PostgreSQL schneidet als Text gebundene Binärdaten am Nullbyte ab — Testdaten für `bytea` als Literal setzen.

Anhang P — Timebox 03 — SQL-Editor und Befehlspalette → Alpha

Stand: 29.10.2026 · ✅ abgeschlossen — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen gebaut. Q-07 (gefährliche Anweisungen, eigentlich TB 04) ist mitgekommen, weil der Editor es braucht.

Ziel

SQL schreiben und ausführen wie in einem Desktop-Werkzeug — mit Grenzen, die eine Datenbank in Produktion schützen, und Abbrechen, das wirklich abbricht.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Zerlegen, einordnen, ausführen	Q-02, Q-03, Q-07, Q-09	✅ `StatementSplitter`, `StatementClassifier`, `QueryRunner`, Arbeitsprozess für SQLite
2 Editor, Verlauf, Palette	Q-01, Q-04, O-02	✅ Wireframes 10 und 12

Was entstanden ist

Zerleger. Eine Anweisung je Aufruf (die Verbindungen nehmen nie mehrere an, F-7). Kennt Zeichenketten mit den Escapes des Treibers (Backslash bei MySQL/MariaDB, `E'…'` bei PostgreSQL), Bezeichner (`"…"`, `` `…` ``, `[…]` bei SQLite), Kommentare (`--`, `#` bei MySQL, verschachtelte Blockkommentare bei PostgreSQL), Dollar-Blöcke, `BEGIN … END` in Routinen und Triggern (`END IF/LOOP/WHILE/REPEAT/CASE` richtig gezählt) und `DELIMITER` aus MySQL-Dumps; findet die Anweisung an der Schreibmarke (Byte-Position, der Browser rechnet UTF-16 in UTF-8 um).

Einordnen. Lesend (SELECT ohne INTO, SHOW, EXPLAIN ohne ANALYZE einer Änderung, WITH ohne ändernde Teile, lesendes PRAGMA …) oder schreibend; gefährlich sind DROP, TRUNCATE, DELETE und UPDATE ohne WHERE auf oberster Ebene und ALTER … DROP — Kommentare, Zeichenketten und Unterabfragen zählen nicht.

Ausführen. Lesendes immer im Schreibschutz-Wächter, Schreibendes nur mit Recht «schreiben» auf beschreibbaren Verbindungen und mit Protokolleintrag ohne Werte. Zeitgrenze `PHPDBADMIN_EDITOR_TIMEOUT` (30 s), Zeilen je Ergebnis `PHPDBADMIN_EDITOR_ROWS` (1'000, «mehr laden» bis 10'000). MySQL/MariaDB lesen ungepuffert und verwerfen den Rest mit `KILL QUERY`, PostgreSQL liest über einen Server-Cursor — nie liegt ein ganzes grosses Ergebnis im Speicher. **Abbrechen** ist eine zweite Anfrage: Ein Merker im Cache entscheidet über «abgebrochen» (MySQL kehrt nach `KILL QUERY` auch ohne Fehler zurück, F-2), die laufende Anweisung wird über eine eigene Verbindung beendet. **SQLite (Q-09)** läuft in einem Arbeitsprozess (`phpdbadmin:sqlite-worker`), den Zeitgrenze oder Abbruch beenden; fertige Ergebnisse davor bleiben.

Editor (Wireframe 10). CodeMirror 6 mit dem Dialekt des Treibers, Vervollständigung aus dem Schema (eine Abfrage je Treiber, als Deferred-Prop nachgeladen), Hervorhebung mit den Farben des Stylesheets, Formatieren ohne neue Abhängigkeit. ⌘↵ Anweisung an der Schreibmarke oder Auswahl, ⇧⌘↵ alles.

Ergebnisse als Reiter im virtualisierten Raster, als TSV kopieren. Gefährliches fragt nach, in Produktion mit dem Namen der Verbindung. Abfragen sind Reiter wie Tabellen, ihr Text bleibt im Browser; der Objektkopf hat den Reiter «SQL» mit einer Vorlage.

Verlauf und gespeicherte Abfragen (Q-04). `query_history` je Konto und Verbindung mit Suche (Frist 90 Tage, täglicher `model:prune`), `saved_queries` mit Namen.

Befehlspalette (O-02, Wireframe 12). ⌘K oder Suchfeld im Kopf: Objekte der offenen Verbindung, zuletzt Geöffnetes, andere Verbindungen (Suchquelle `/palette`, nur sichtbare), Aktionen («Neue Abfrage auf …», «Struktur von …», Server, Tastenkürzel); «>» zeigt nur Aktionen; Pfeile, Eingabe, Escape.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
Q-01	Durchlauf je Treiber	im Browser mit PostgreSQL: Vorlage, Formatieren, Ausführen, Vervollständigung (Endpunkt-Test: Spalten der Tabellen); Hervorhebung je Dialekt über `@codemirror/lang-sql`
Q-02	Abbrechen beendet die Abfrage auf dem Server; «abgebrochen» auch bei MySQL ohne Fehler	`QueryRunnerTest`: zweiter Prozess mit `sleep(20)` / `pg_sleep(20)` endet nach `KILL QUERY` / `pg_cancel_backend` in unter 5 s (MySQL, MariaDB, PostgreSQL); Merker entscheidet über «abgebrochen» (vier Treiber)
Q-03	lange und schreibende Anweisung auf geschützter Verbindung je Treiber	Zeitgrenze 1 s auf vier Treibern (MySQL mit grossem Kreuzprodukt, weil `SLEEP` still abbricht); Löschen auf schreibgeschützter Verbindung abgewiesen; ohne Recht nicht ausgeführt
Q-04	Test	`SqlControllerTest`: Verlauf nur des eigenen Kontos, Suche, gespeicherte Abfragen fremd 404
Q-07	Varianten mit Kommentaren, Gross-/Kleinschreibung, mehreren Anweisungen	`StatementClassifierTest` (13 Fälle Gefahr, 15 Fälle Art), `SqlControllerTest`: ohne Bestätigung 409 und nichts geändert, Produktion mit falschem Namen 409
Q-09	Abfrage über der Grenze beendet, Oberfläche meldet es	SQLite-Arbeitsprozess mit Zeitgrenze 1 s und mit Abbruch-Merker
O-02	Test der Suchquelle; Durchlauf nur mit Tastatur	`PaletteControllerTest` (nur sichtbare Verbindungen, keine Geheimnisse); im Browser ⌘K, Tippen, Auswahl

Alpha-Kriterium «Hoang nutzt phpdbadmin eine Woche zum Nachsehen»: **offen (Hoang)**.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
10 Werkzeugleiste	Auswahl des Schemas als Liste (bei mehreren)	Wireframe zeigt eine Verbindung mit einer Datenbank
10 Ergebnis	«Ergebnis exportieren» fehlt	Export TB 06
12 Palette	«Andere Verbindungen» zeigt Verbindungen, nicht ihre Tabellen	Tabellen anderer Verbindungen hiessen, jede zu öffnen — Suchquelle bewusst schlank

Tests

`StatementSplitterTest` 22, `StatementClassifierTest` 28, `QueryRunnerTest` 28 (vier Treiber), `SqlControllerTest` 8, `PaletteControllerTest` 2. Suite nach TB 03: 611 Tests, 595 bestanden, 16 übersprungen (schwere Messungen).

Commits

`efde8fe` Zerlegen, einordnen, ausführen · `d91231a` Editor, Verlauf, Palette. Die Editor-Probe der Feasibility ist entfernt.

Gelernt

- Ein eigener Hilfsname `run()` in einem PHPUnit-Test überschreibt die finale Methode der Testklasse — Fatal Error beim Laden.
- PostgreSQL: Nach einem Fehler im Cursor ist die Transaktion abgebrochen; ein `close` im `finally` wirft dann einen neuen Fehler und überdeckt den eigentlichen (Zeitlimit wurde zu «Fehler»).
- Unter PHP-FPM zeigt `PHP_BINARY` auf FPM — für einen Arbeitsprozess die CLI suchen (Einstellung `PHPDBADMIN_PHP_BINARY`).
- Der eingebaute Browser ist zwischen parallelen Helfern geteilt; Durchläufe macht nur der Hauptstrang.

Anhang Q — Timebox 04 — Bearbeiten mit Vorschau und Rückgängig

Stand: 29.10.2026 · ✅ abgeschlossen — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen gebaut. Q-07 (gefährliche Anweisungen) ist schon mit TB 03 gekommen.

Ziel

Daten ändern, ohne Angst vor Datenverlust: alles zuerst sehen, in einer Transaktion anwenden, rückgängig machen können.

Umfang

Scheibe	Anforderungen	Ergebnis
1 RowEditor	D-04, D-07, D-09, N-03	✅ Vorschau, eine Transaktion, Sicherung, Rückgängig
2 Endpunkte	D-04, D-09, E-05, V-11	✅ inkl. Freigabe auf Zeit und Kopieren
3 Oberfläche	D-04, D-05, E-05, V-11	✅ Wireframes 5, 6, 7

Was entstanden ist

RowEditor (ADR-003). Gesammelte Änderungen — einfügen, ändern, löschen über den Primärschlüssel — zuerst als Vorschau aus `pretend()` mit eingesetzten Werten (nur zur Anzeige; ausgeführt wird mit Bindungen) und den Zeilen, die `ON DELETE CASCADE` mitlöschen würde. Anwenden auf **allen vier Treibern in einer Transaktion**: jede Änderung muss genau eine Zeile treffen, sonst wird alles zurückgerollt (`ChangeConflict`). Vorher- und Nachher-Werte in `change_snapshots` (bis 1'000 Zeilen, 7 Tage; Binärdaten als Base64), ins Protokoll nur SQL ohne Werte. **Rückgängig** kehrt rückwärts in einer Transaktion um — nur, wenn die Zeilen noch so aussehen wie nach der Änderung, sonst bleibt alles. Tabellen ohne Primärschlüssel nur lesen (Rückfall D-07), ungültiges JSON abgewiesen.

Oberfläche. Raster: Doppelklick, Eingabe oder F2 bearbeitet; geänderte Zellen, neue und zum Löschen markierte Zeilen farbig; Änderungsleiste; Vorschau als Seitenblatt mit Verfahren, nummerierten Anweisungen, Kaskaden-Hinweis und Namenseingabe in Produktion; danach «Zuletzt angewendet · Rückgängig machen». Zeilen-Detail mit Feld-Editor je Art (D-05): NULL-Haken (NULL ist nicht leer), Wahrheitswert, enum/set als Auswahl, JSON geprüft und formatierbar, lange Texte im Textfeld; Duplizieren, Löschmarke. **Kopieren (E-05)** als TSV, Markdown oder INSERT — Zeile oder Ausschnitt, auf dem Server erzeugt (Anführungszeichen je Treiber).

Schreibschutz auf Zeit (V-11). Für 15 Minuten, nur mit Recht «schreiben», mit Pflichtgrund, Grund und Ende im Protokoll; im Kopf «Schreiben erlaubt bis … · wieder sperren». Die Freigabe liegt in der Sitzung, `TargetConnections` berücksichtigt sie überall (Bearbeiten, SQL-Editor, Struktur, Server).

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
D-04	je Treiber: nichts geschrieben vor «Anwenden»; Fehler in der zweiten Anweisung rollt die erste zurück; Vorschau aus `pretend()` mit Bindungen; Kaskaden gezählt	`RowEditorTest` (vier Treiber): Vorschau ändert nichts, zählt 2 Kinder; doppelter Name in der zweiten Anweisung → erste zurückgerollt, kein Protokolleintrag
N-03	erzwungener Fehler mitten in einer Folge; Eltern mit Kindern	obiger Fall; verfehlte Zeile (Schlüssel 999) → nichts geändert; Kaskade löscht genau die Kinder des gelöschten Kunden
D-05 ↓	je Typ und Treiber byte-genau	Umlaute, Emoji, Anführungszeichen, Backslash, leer gegen NULL, Dezimal, Datum, Wahrheitswert, JSON auf vier Treibern; Binärdaten nur NULL (Rückfall: Bildvorschau später)
D-07 ↓	mehrdeutige Zeile wird nicht geändert	Tabelle ohne Schlüssel mit zwei gleichen Zeilen: abgewiesen, beide unverändert (Rückfall: nur lesen)
D-09	anwenden, rückgängig, Zustand gleich	vier Treiber: Ändern + Einfügen + Löschen, dann Rückgängig → Tabellen gleich dem Ausgangszustand; zweites Rückgängig abgewiesen; spätere fremde Änderung → Konflikt, nichts überschrieben
E-05	Test	`RowCopyTest` (Sonderzeichen, NULL, Binärdaten, Anführungszeichen je Treiber), `RowChangeControllerTest` (Zeile, Ausschnitt, Filter)
V-11	Ablauf nach 15 Minuten (Uhr gestellt), Grund und Ende im Protokoll, ohne Recht 403	`RowChangeControllerTest` : freigegeben → änderbar; nach 16 Minuten 403; Protokoll mit Grund und Ende; Leser 403; Sperren protokolliert
Q-07	—	mit TB 03

Durchlauf im Browser (MariaDB, Umgebung Test): Zelle ändern, Wahrheitswert umschalten, Vorschau, Anwenden, Rückgängig — Ausgangszustand wieder da.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
7 Detail	Binärdaten: keine Vorschau, kein Herunterladen, kein Ersetzen	Rückfall D-05: Werte als Text, Bildvorschau später
5 Kopf	«Exportieren» fehlt	TB 06
6 Vorschau	Hinweis «nicht in einer Transaktion» bei MyISAM fehlt	alle Testtabellen InnoDB; MyISAM ist selten und kennt keine Transaktion — als offene Lücke vermerkt

Tests

`RowEditorTest` 32 (vier Treiber), `RowChangeControllerTest` 8, `RowCopyTest` 3. Suite nach TB 04: 744 Tests, 728 bestanden, 16 übersprungen.

Commits

`7a82450` RowEditor · `fcf1d18` Endpunkte · `1066404` Oberfläche.

Gelernt

- `Model::create()` hat die Spaltenvorgaben der Datenbank nicht — eine Zeitzone fehlte (`setTimezone(null)`). Vorgaben gehören auch ins Modell (`$attributes`).
- MySQL quotet in `substituteBindingsIntoRawSql()` mit Backslash, die anderen mit verdoppeltem Hochkomma — Tests auf beides auslegen.

Anhang R — Timebox 05 — Struktur ändern

Stand: 29.10.2026 · ✅ gebaut und getestet in drei Scheiben auf einem eigenen Zweig (Arbeitskopie), parallel zu TB 04 — Browser-Durchlauf und Zusammenführen im Hauptstrang offen.

Ziel

Tabellen anlegen, ändern, umbenennen, kopieren, leeren und löschen sowie Datenbanken bzw. Schemas anlegen und löschen — auf allen vier Treibern mit einer Vorschau, die genau das zeigt, was danach läuft, und angewendet so, dass keine Daten verloren gehen (ADR-003, N-03).

Umfang

Nr.	Anforderung (CR-001 § 4.5, § 4.10)	Prio	Ergebnis
S-02	Tabelle anlegen mit geführtem Formular (Spalten, Primärschlüssel, Indizes, Fremdschlüssel) und SQL-Vorschau	M	✅
S-03	Tabelle ändern: Spalten hinzufügen, ändern, umbenennen, löschen; Indizes und Fremdschlüssel; Kommentare — je Treiber angewendet	M	✅
S-04	Tabelle umbenennen, kopieren (Struktur, mit Daten), leeren, löschen — Leeren und Löschen nur mit Namenseingabe	M (Plan: S ↓)	✅
S-05	Datenbank bzw. Schema anlegen und löschen	S	✅ (SQLite: nur anlegen, siehe Entscheide)
N-03	Kein Datenverlust durch die Anwendung: Vorschau, Transaktion wo möglich, DDL ohne Transaktion ehrlich angezeigt, SQLite-Umbau nie mit aktiven Fremdschlüsseln	M	✅ für Strukturänderungen (Datenänderungen: TB 04)

Wireframes 9 (Tabelle ändern, MySQL und PostgreSQL, Telefon) und 6 (SQL-Vorschau als Dialog).

Was entstanden ist

Planen (`app/Services/Schema`). `SchemaPlans` baut aus geprüften Eingaben einen `SchemaPlan`: Aktion und Ziel fürs Protokoll, die Anweisungen, das Verfahren (`ApplyMode`), Hinweise, was verloren geht, und welchen Namen man zur Bestätigung eintippt. Anlegen und Ändern kommen aus Laravels `Blueprint::toSql()` — nie aus `pretend()` (F-5) —, umbenennen, kopieren, leeren, löschen und Schemas baut der Plan selbst, Namen je Segment gequotet (`SchemaName`). Jeder Name eines bestehenden Objekts muss im Katalog stehen. Vorschau und Anwenden planen über dieselbe Methode; die Vorschau über die Leseverbindung, das Anwenden noch einmal über die beschreibbare (`withWritable()`).

Ändern in einer Reihenfolge für alle vier. Neue Spalten (MySQL/MariaDB mit `AFTER`), Fremdschlüssel weg, Spalten ändern (unter altem Namen), neue Fremdschlüssel, umbenennen, Indizes weg, Spalten weg, neue Indizes, Tabellenkommentar. So baut SQLite die Tabelle höchstens einmal um, und zwar vor jedem Umbenennen.

Anwenden je Treiber (`SchemaChanges`, ADR-003).

Treiber	Verfahren	Nach einem Fehler
MySQL, MariaDB	Anweisung für Anweisung, ohne Transaktion	angehalten; die Oberfläche zeigt je Anweisung «gelaufen», «gescheitert», «nicht gelaufen»

Treiber	Verfahren	Nach einem Fehler
PostgreSQL	alles in einer Transaktion	nichts angewendet
SQLite	`PRAGMA foreign_keys = OFF` ausserhalb, Umbau in der Transaktion, `PRAGMA foreign_key_check` (nur neue Verletzungen zählen), Commit, `ON` — auch nach einem Fehler	nichts angewendet

Beim SQLite-Umbau legt Laravel nur die Indizes neu an. Dazu kommen jetzt die Check-Constraints (in die neue Tabelle) und — nach sqlite.org — alle Sichten und Trigger der Datei: vorher gelöscht, danach mit ihrem gespeicherten Text neu angelegt. Ohne das scheitert das Umbenennen an jeder Sicht auf die Tabelle («error in view …: no such table»), und die Trigger der Tabelle gingen mit `DROP TABLE` verloren.

Seiten und Dialoge.

Methode	Adresse (unter `c/{connection}`)	Name
GET	`s/{schema}/new-table`	`workspace.table.create` (Seite `workspace/TableCreate`)
POST	`s/{schema}/new-table/preview`	`workspace.table.preview` (JSON)
POST	`s/{schema}/new-table`	`workspace.table.store`
GET	`s/{schema}/o/{object}/alter`	`workspace.object.alter` (Seite `workspace/TableAlter`)
POST	`s/{schema}/o/{object}/alter/preview`	`workspace.object.alter.preview` (JSON)
POST	`s/{schema}/o/{object}/alter`	`workspace.object.alter.apply`
POST	`s/{schema}/o/{object}/actions/preview`	`workspace.object.actions.preview` (JSON)
POST	`s/{schema}/o/{object}/actions`	`workspace.object.actions.apply` (umbenennen, kopieren, leeren, löschen)
GET	`schemas/sqlite-folders`	`workspace.schemas.folders` (JSON, nur SQLite)
POST	`schemas/preview`	`workspace.schemas.preview` (JSON)
POST	`schemas`	`workspace.schemas.apply`

Einstieg: «Tabelle ändern» und ein Menü «Weitere Aktionen» oben auf der Struktur-Seite, «Tabelle anlegen» und ein Menü für Datenbank bzw. Schema (SQLite: neue Datei) auf der Schema-Seite. Ohne Recht «schreiben» oder bei Schreibschutz bleiben die Knöpfe sichtbar, aber gesperrt, mit Grund daneben; die Seiten selbst antworten dann mit 403.

Rechte und Protokoll. Jede Änderung braucht die Policy-Fähigkeit `write` (Form-Request bzw. `Gate::authorize`), nie auf einer schreibgeschützten Verbindung (403 mit Klartext), eine fremde Verbindung ist 404; auf Produktion zusätzlich der Name der Verbindung (nur beim Anwenden). Leeren, Löschen und Spalten löschen verlangen den Namen der Tabelle bzw. des Schemas — geprüft in `SchemaChanges` vor der ersten Anweisung, ohne Eintrag im Protokoll. Jede angewendete Änderung ergibt genau einen Eintrag (`schema.create`, `schema.alter`, `schema.rename`, `schema.copy`, `schema.truncate`, `schema.drop`, `database.create`, `database.drop`), gelungen oder gescheitert, mit Verfahren, Zahl der gelaufenen Anweisungen und dem SQL über `SqlMask::mask()`; `AuditLog::record()` wird mit benannten Argumenten gerufen (passt zur neuen Signatur aus TB 07).

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
S-02	Vorschau (aus `toSql()`) = ausgeführtes SQL; Tabelle danach per Introspektion gleich der Eingabe	`SchemaChangesTest` je Treiber: Mitschnitt der Anweisungen = Vorschau; Spalten, NULL, Typen je Treiber, Auto-Increment, Primärschlüssel, Index, eindeutiger Index, Fremdschlüssel mit `ON DELETE CASCADE`. Endpunkt je Treiber mit Weiterleitung, Meldung und einem Eintrag (Vorgabe im Protokoll maskiert)
S-03	Tests je Treiber mit Elterntabelle und abhängigen Zeilen: Daten, Indizes, Fremdschlüssel und Kinder bleiben	`SchemaChangesTest` je Treiber: Spalte ändern, umbenennen, löschen, hinzufügen, Index anlegen an der Elterntabelle — 2 von 2 Kindern, Daten, eindeutiger Index und Fremdschlüssel der Kinder bleiben; SQLite: `foreign_key_check` leer, Fremdschlüssel wieder an; SQLite behält Checks, Trigger und Sichten. Rundlauf je Treiber: alle Typen der Liste auf ihre eigene Definition geändert — Typ, NULL und Vorgabe unverändert
N-03	Test mit erzwungenem Fehler mitten in einer Folge	je Treiber: neue Spalte, dann eindeutiger Index auf doppelte Werte — MySQL/MariaDB: 1 Anweisung bleibt und wird gemeldet, PostgreSQL/SQLite: nichts angewendet; Kinder bleiben; ein Eintrag `failed`. Über den Endpunkt (MySQL): Meldung mit «gelaufen 1, gescheitert 2»
S-04	ohne korrekten Namen keine Ausführung	je Treiber: Leeren ohne Namen und Löschen mit falscher Schreibweise abgewiesen, Zeilen bleiben, kein Eintrag; mit Namen geleert bzw. gelöscht. Dazu je Treiber: Umbenennen (Fremdschlüssel der Kinder folgen), Kopieren mit Daten (gleiche Spalten, eindeutiger Index, Zähler läuft in Original und Kopie weiter), Elterntabelle mit Kindern weder leeren noch löschen
S-05	Test je Treiber (SQLite: neue Datei im erlaubten Ordner)	MySQL, MariaDB (Verbindung ohne festen Namen) und PostgreSQL (Schema): anlegen, Löschen ohne Namen abgewiesen, mit Namen gelöscht, zwei Einträge. SQLite: neue Datei mit SQLite-Kopf im erlaubten Ordner; Ordner ausserhalb, `../` im Namen, falsche Endung, bestehende Datei abgewiesen

Querschnitt (`SchemaChangeRequestTest`, alle acht ändernden Endpunkte und beide Seiten): Recht «lesen» 403, schreibgeschützt 403 auch für den Besitzer, fremde Verbindung 404 — Datenbank und Protokoll unberührt; «schreiben» genügt; Produktion erst mit dem Namen der Verbindung, die Vorschau ohne. Namen, die Laravels Grammatik zerlegen würde (Punkt, « as », «->», «*», Anführungszeichen, Semikolon, über 63 Bytes), werden vor jeder Anweisung abgewiesen.

Abweichungen von den Wireframes

Wireframe	Abweichung	Grund
9 Typ	Typ als Liste der Originalnamen des Treibers, Länge bzw. Genauigkeit in eigenen Feldern, «eigener Typ» als Freitext	eine Liste je Treiber plus freier Typ (CR S-02); Länge getrennt prüfbar
9 Reihenfolge	keine Griffe zum Verschieben; neue Spalten kommen ans Ende (MySQL/MariaDB mit `AFTER`)	Umsortieren bestehender Spalten ist nicht in S-03
9 «+ Check»	fehlt; Checks bleiben beim SQLite-Umbau erhalten, werden aber nicht bearbeitet	nicht in S-03 (S-06 bzw. SQL-Editor)
9 Reiter «Beziehungen»	fehlt	S-07 (TB 10); den Objektkopf bearbeitet der Hauptstrang
9 Verlust je Anweisung	«Werte in ≈ N Zeilen» aus der Schätzung, ohne «davon M nicht NULL»	die Zählung bräuchte einen vollen Durchlauf grosser Tabellen
9 Telefon	voller Entwurf als Karten statt nur Zusammenfassung, darüber der Hinweis «für den Desktop ausgelegt»	gleiche Seite für alle Breiten; Bearbeiten darf auf dem Telefon eingeschränkt sein (O-07)

Wireframe	Abweichung	Grund
6	Dialog für S-04 und S-05; «Tabelle ändern» zeigt die Vorschau rechts daneben (Wireframe 9)	Wireframe 9 hat Vorrang für das Ändern

Entscheide in der Timebox

Entscheid	Begründung
Enge Namensregel (`SchemaName`): kein Punkt, keine Anführungszeichen, kein « as », «->», «*», «;», keine Steuerzeichen, höchstens 63 Bytes — auch für bestehende Namen, die ein Plan einsetzt	Laravels `wrap()` zerlegt an Punkten und « as », lässt «*» ungequotet und liest «->» als JSON-Pfad; PostgreSQL kürzt längere Bezeichner still. Was die Regel nicht erfüllt, geht im SQL-Editor
Vorgaben und `enum`-Werte über `PDO::quote` als `TrustedSql`	Laravel verdoppelt bei Vorgaben nur `'` (MySQL liest `\` als Escape) und setzt `enum`-Werte gar nicht in Anführungszeichen
Eigener Typ nur in einer engen Form (Wörter, eine Klammer mit Zahlen, `[]`); die bisherige Definition einer Spalte gilt wörtlich	freier Typ ohne Weg in fremdes SQL; bestehende Spalten bleiben änderbar
Ausdruck als Vorgabe: `CURRENT_TIMESTAMP` oder wörtlich die bisherige Vorgabe der Spalte	kein freies SQL in der Vorgabe
Bestehende Spalten: Typ aus der Liste nur, wenn Laravel ihn wieder genau so erzeugt (`ColumnTypes::specOf`), sonst wörtlich	eine Änderung von Kommentar oder NULL darf den Typ nicht umbauen
Auto-Increment und Primärschlüssel nur beim Anlegen	Umstellen eines Schlüssels ist je Treiber ein eigener Umbau, nicht in S-03
Leeren und Löschen ohne CASCADE; PostgreSQL `TRUNCATE … RESTART IDENTITY`	eine Elterntabelle mit Kindern lehnt der Server ab (SQLite: `foreign_key_check`) — nie mitgelöschte Kinder; Zähler wie bei MySQL zurückgesetzt
Kopieren je Treiber: MySQL/MariaDB `CREATE TABLE … LIKE`; PostgreSQL `LIKE … INCLUDING ALL`, eigene Sequenzen für serial, `OVERRIDING SYSTEM VALUE`, `setval`; SQLite der gespeicherte CREATE-Text mit neuem Namen samt Indizes	gleiche Struktur, Zähler laufen weiter; Fremdschlüssel und Trigger werden nicht mitkopiert (steht im Hinweis)
S-05 MySQL/MariaDB nur über eine Verbindung ohne festen Datenbanknamen; PostgreSQL `DROP SCHEMA … CASCADE` mit Anzahl der Tabellen im Hinweis	eine neue Datenbank erschiene sonst nirgends; Löschen eines Schemas heisst «mit allem darin» wie bei MySQL
SQLite: neue Datei nur in einem erlaubten, beschreibbaren Ordner (Kopf per `VACUUM` geschrieben); Dateien löscht phpdbadmin nicht	eine SQLite-«Datenbank» ist die Datei — Löschen gehört ins Dateisystem, nicht in eine Web-Oberfläche
Eigenes Zeitlimit `PHPDBADMIN_SCHEMA_TIMEOUT_MS` (Vorgabe 10 Minuten)	MariaDBs `max_statement_time` gilt auch für `ALTER TABLE` und hätte grosse Umbauten nach 15 s abgebrochen
Recht «schreiben» (nicht «verwalten» wie in TB 09)	Struktur ändern ist Arbeit an den Daten der Verbindung, Server-Verwaltung nicht
Ein- und Mehrzahl als eigene Texte statt `trans_choice()`	deutsche Schlüssel ohne `de.json`: `trans_choice()` fällt auf die Rückfallsprache Englisch zurück
Feasibility-Probe `DdlPreviewProbeTest` entfernt	ihre Fälle (Vorschau = Ausführung, Kinder bleiben, Fehler mitten in der Folge) tragen jetzt `SchemaChangesTest`; die negativen Proben (`pretend()`, naive Transaktion) prüften Laravel und SQLite, nicht phpdbadmin — Befund bleibt in der Feasibility

Tests

126 neue Tests (davon 28 Einheitstests ohne Datenbank); die Probe mit 24 Fällen entfällt. Suite gesamt 713, davon 697 bestanden und 16 übersprungen (die schweren Proben auf Wunsch). Läufe mit `PHPDBADMIN_TEST_DATABASE=phpdbadmin_test_struktur`; Datenbanken aus S-05 heissen `phpdbadmin_s05_…` und werden im Test wieder entfernt (`finally`).

Datei	Tests	MySQL	MariaDB	PostgreSQL	SQLite
`Services/Schema/SchemaChangesTest`	17	4	4	4	5
`Http/Controllers/Workspace/Schema/TableCreateControllerTest`	15	1	1	1	12
`Http/Controllers/Workspace/Schema/TableAlterControllerTest`	7	1	—	—	6
`Http/Controllers/Workspace/Schema/TableActionControllerTest`	18	4	4	5	5
`Http/Controllers/Workspace/Schema/SchemaControllerTest`	11	1	2	2	6
`Http/Requests/Workspace/Schema/SchemaChangeRequestTest`	29	—	—	—	29
`Http/Controllers/Workspace/WorkspaceControllerTest` (neu: `canWrite`)	1	—	—	—	1
`Unit/Services/Schema/ColumnTypesTest`	28	—	—	—	—
zusammen	**126**	**11**	**11**	**12**	**64**

Prüfkette vor jedem Commit: `vp check`, `vue-tsc`, Pint, Larastan, PHPUnit — grün; `vp build` baut.

Commits

`273eaa3` Scheibe 1: planen, je Treiber anwenden, Seiten · `3de69d9` Scheibe 2: Endpunkte je Treiber getestet, Rechte, Bestätigung · `d6f6d40` Zielspalte und Fehler der Vorschau · Doku (dieser Commit).

Offen

- **Browser-Durchlauf** bei 1440, 768 und 375 px, hell und dunkel, gegen Wireframe 9 und 6 — Hauptstrang (in dieser Arbeitskopie ohne Browser gebaut).
- Spalten umsortieren, Primärschlüssel ändern, Checks bearbeiten, Index-Arten (Volltext, räumlich); zusammengesetzte Fremdschlüssel kann der Server, die Oberfläche bietet je Fremdschlüssel eine Spalte.
- SQLite-Tabellen mit Inline-`UNIQUE` (`sqlite_autoindex_…`): Laravels Umbau will den internen Index neu anlegen (vertauschte Argumente in `str_starts_with`) und scheitert — zurückgerollt, nichts verloren, aber nicht änderbar. Nicht getestet.
- `MaintenanceController` (TB 09) nutzt `trans_choice()` und zeigt auf Deutsch englische Mehrzahl — besser an einer Stelle lösen (Basalto: `SourceLocaleTranslator`, Lektion «Deutsche Mehrzahl fällt auf Englisch zurück» im Vault).
- Laravels `compileColumns` für MySQL setzt den Tabellennamen über `quoteString()` ohne Escape ein — die Strukturansicht (S-01) scheitert an Tabellennamen mit `'`. Vorbestehend, nicht in TB 05 behoben.

Gelernt

- **Rundlauf je Treiber:** Eine Spalte auf ihre eigene Definition ändern und Typ, NULL und Vorgabe vergleichen fand sofort eine falsche Zuordnung (`tinyint(1)` wurde zu `tinyint`, weil die Ganzzahl-Regel vor dem Wahrheitswert griff). Jede Rückübersetzung aus der Introspektion braucht so einen Test.
- **SQLite prüft beim `RENAME TABLE` die ganze Datei:** Eine Sicht auf die umgebaute Tabelle liess den Umbau scheitern. Sichten und Trigger vor dem Umbau löschen und danach neu anlegen (sqlite.org, Abschnitt 7).
- **Laravels Grammatik ist kein Escaping:** `quoteString()` für `enum`-Werte verdoppelt nichts, Vorgaben nur `'` — Literale über `PDO::quote`, Bezeichner nur nach enger Regel.
- `trans_choice()` mit deutschen Schlüsseln ist eine bekannte Falle (Lektion im Vault) — und wurde trotzdem wieder gebaut, weil der Lektionen-Index nicht zu Beginn gelesen war; ein Endpunkt-Test mit der deutschen Meldung hat es gefunden.

Anhang S — Timebox 06 — Export und Import im Hintergrund, Statistik

Stand: 29.10.2026 · ✅ abgeschlossen — gebaut auf einem eigenen Zweig (parallel zu TB 02/03/07), zusammengeführt mit `2f0301c`, im Browser geprüft.

Ziel

Tabellen und ganze Datenbanken als CSV, JSON oder SQL exportieren und CSV- oder SQL-Dateien importieren — in Paketen, im Hintergrund, mit Probelauf, der nichts schreibt, und aktueller Statistik danach; gleich auf allen vier Treibern.

Umfang

Anforderung	Priorität im Plan	Ergebnis
E-01 Export (Tabelle, Datenbank; CSV, JSON, SQL)	Must	✅ — Export einer **Abfrage** folgt mit dem SQL-Editor (TB 03), «gefilterte Zeilen» mit dem Knopf im Objektkopf
E-02 Import (CSV mit Zuordnung und Probelauf, SQL-Datei)	Must ↓	✅ beides, SQL-Datei samt Zerleger
E-06 Statistik nach Importen und auf Knopfdruck	Should	✅
B-07 Warteschlange, ohne Worker synchron mit Grenze	Should	✅

Wireframe-Bereich 11 (Export und Import).

Was entstanden ist

Export (`App\Services\Transfer\Exporter`, ADR-007). Gelesen wird im Schreibschutz-Wächter (`Dialect::runReadOnly`, eine Nur-lesen-Transaktion für den ganzen Export) und in Paketen zu 5'000 Zeilen über den Primärschlüssel — auch zusammengesetzt, `(a, b) > (x, y)` ausgeschrieben. Ohne Schlüssel und für Sichten liest ein Cursor, der nie das ganze Ergebnis hält: PostgreSQL `DECLARE … CURSOR` / `FETCH 5000`, MySQL und MariaDB ungepuffert, SQLite streamt ohnehin. Einen Versatz (`OFFSET`) gibt es nicht — ohne eindeutige Sortierung liefert er Zeilen doppelt oder gar nicht. Geschrieben wird als Strom nach `storage/app/private/transfers/<Auftrag>/`.

- **CSV:** Trennzeichen nach Wahl, UTF-8 mit BOM für Excel, Kopfzeile; NULL ist ein leeres Feld ohne Anführungszeichen, leerer Text `""` — so kommen beide beim Re-Import zurück. Binärspalten als Base64 oder weggelassen. CSV nur für eine Tabelle.
- **JSON:** Liste von Objekten, eine Zeile je Objekt; ganze Datenbank als Objekt «Tabelle → Liste». Zahlen und Wahrheitswerte als solche, JSON-Spalten eingebettet (MySQL, PostgreSQL), Binärdaten als Base64.
- **SQL:** Struktur, Daten oder beides, wahlweise mit `DROP … IF EXISTS`. `INSERT` zu 100 Zeilen bzw. 1 MB, Werte über Laravels `escape` je Treiber (`x'…'` bzw. `'\x…'::bytea`, `true`/`1`). Tabellen nach Fremdschlüsseln geordnet; MySQL/MariaDB und SQLite ohne Schema im Namen (Import in eine andere

Datenbank) und mit abgeschalteter Fremdschlüsselprüfung; PostgreSQL mit Schema wie die DDL des Dialekts, Sequenzen der `serial`-Spalten vorher, `setval` nach den Daten, Fremdschlüssel am Ende per `ALTER TABLE` (Verweise auf die eigene Tabelle und Kreise gehen so auch). MySQL-Sichten ohne `DEFINER` und ohne Schema vor den Spalten. Nicht enthalten: Trigger, Routinen, Ereignisse (SQLite-Trigger würden beim Einspielen feuern) — dafür E-03.
- Kommazahlen verlustfrei: `(string)` rundet in PHP auf 14 Stellen, auch in Laravels `escape`; der Export nimmt die kürzeste exakte Darstellung (`var_export`).

Import (`CsvImporter`, `SqlImporter`). Hochladen legt einen Entwurf an; danach Zuordnung, Probelauf, Import (Wireframe 11).

- **CSV:** Kopfzeile, Trennzeichen und Kodierung (UTF-8 mit/ohne BOM, sonst Windows-1252) erkennt die Analyse und lassen sich ändern; Zuordnung nach Namen vorgeschlagen (`E-Mail` → `email`). Eigener Leser statt `fgetcsv` (NULL gegen leeren Text, Zeilennummer des Satzanfangs auch bei Umbrüchen in Feldern). Eingefügt wird in Paketen zu höchstens 1'000 Zeilen (weniger bei vielen Spalten, unter den Grenzen für gebundene Werte); scheitert ein Paket, finden Einzelversuche hinter einem **Sicherungspunkt, der danach freigegeben wird**, die schuldigen Zeilen — Laravels verschachtelte Transaktionen geben ihre Sicherungspunkte nie frei, und PostgreSQL bricht ohne sie die ganze Transaktion ab. Fehler mit Zeilennummer und Grund (leer und NOT NULL, kein Base64, falsche Feldzahl, Ablehnung der Datenbank mit ihrer Meldung ohne SQL). Wahrheitswerte aus Tabellenkalkulationen (`ja`, `WAHR`, `x`) → 1/0. Leere Pflichtfelder mit Vorgabe oder Autowert setzen `DEFAULT` (SQLite kennt das nicht). Binärwerte als Strom gebunden (`PDO::PARAM_LOB`) — als Text verlöre PostgreSQL am Nullbyte und SQLite speicherte TEXT.
- **Probelauf schreibt nichts:** alle Zeilen in einer Transaktion einfügen, dann zurückrollen; auf Tabellen ohne Transaktionen (MyISAM) nur die Werte prüfen. Import: «Fehlerzeilen überspringen» oder «bei einem Fehler nichts importieren» — beides in einer Transaktion.
- **SQL-Datei:** eigener Zerleger `SqlScript` in Stücken von 1 MB — Zeichenketten (`''`, bei MySQL/MariaDB `\'`, bei PostgreSQL `E'…'`), Namen in `"…"`, `` `…` ``, `[…]`, Kommentare (`--` mit Leerraum bei MySQL, `#`, verschachtelt bei PostgreSQL), ausführbare `/*! … */`, `$$`-Blöcke, `DELIMITER` des MySQL-Clients, `BEGIN … END` in `CREATE TRIGGER/PROCEDURE/FUNCTION/EVENT`. Jede Anweisung einzeln vorbereitet (T-07); was MySQL nicht vorbereiten kann (Fehler 1295: `LOCK TABLES`, `CREATE TRIGGER` …), läuft ohne Vorbereitung — die Verbindung lässt nie mehrere Anweisungen in einem Aufruf zu. PostgreSQL und SQLite in einer Transaktion (eigene `BEGIN`/`COMMIT` der Datei übergangen, SQLite mit Fremdschlüsselprüfung am Ende); MySQL/MariaDB Anweisung für Anweisung, bei einem Fehler bleibt, was davor lief — die Meldung sagt es. Probelauf: nur zerlegen und zählen, Warnung vor `DROP`, `TRUNCATE`, `DELETE`/`UPDATE` ohne `WHERE`.
- **Schreiben nur mit Recht:** `Gate::authorize('write', $connection)` (vorläufige `DatabaseConnectionPolicy::write`: Besitzer oder «verwalten»; die vollständige Policy aus TB 07 ersetzt sie beim Zusammenführen), nie über schreibgeschützte Verbindungen (403), auf Produktion beim echten Lauf mit dem Namen der Verbindung. Jeder echte Import ein Eintrag `data.import` (Ziel, Zeilen, übersprungene, Statistik); eine SQL-Datei, die auf MySQL/MariaDB mittendrin scheitert, ebenfalls (mit `failed`).

Statistik (E-06). `Dialect::analyze`: `ANALYZE TABLE` bei MySQL/MariaDB (meldet Fehler als Ergebniszeile, deshalb gelesen und geprüft; aktualisiert auch die bis zu 24 Stunden alte Zahl in `information_schema` von MySQL 8), sonst `ANALYZE schema.tabelle`. Nach jedem CSV-Import automatisch (abwählbar), auf der Seite auf Knopfdruck mit Eintrag `data.analyze`.

Aufträge und Warteschlange (B-07). Tabelle `transfers` (Datenmodell § 2.4, ULID, Frist 24 Stunden, täglicher `model:prune` löscht Auftrag und Ordner). `RunExport` / `RunImport` tragen **nur die Kennung**; der Worker liest Zugangsdaten aus der App-Datenbank. Im Hintergrund läuft ein Auftrag, wenn ein Worker konfiguriert ist (`QUEUE_CONNECTION` nicht `sync`, `PHPDBADMIN_TRANSFERS_BACKGROUND` nicht `false`), **sich in den letzten 90 Sekunden gemeldet hat** (`App\Support\QueueHeartbeat`, Lebenszeichen bei `JobPopping` / `JobProcessing`, höchstens alle 15 s in den Cache) **und** die Verbindung ihr Passwort speichert — «nur für diese Sitzung» kommt nie in die Warteschlange. Sonst läuft er in der Anfrage bis zu Grenzen (Vorgabe 100'000 Zeilen bzw. 20 MB) mit einer Meldung, die den Grund und den Ausweg nennt. Wer zuerst «wartet» auf «läuft» setzt, führt aus (bedingtes Update gegen Doppelläufe); `retry_after` der Warteschlange auf 3'660 s angehoben, weil ein Auftrag bis zu einer Stunde laufen darf. Fortschritt in `progress` und `rows`, höchstens zweimal je Sekunde geschrieben; dabei fragt der Lauf, ob abgebrochen wurde. Zeitlimit je Anweisung während eines Auftrags 600 s statt 15 s.

Oberfläche. Seite `workspace/transfers/Index` unter `c/{connection}/transfers` (Einstieg im Menü des Verbindungs-Chips): Exportieren, Importieren, Statistik, Aufträge mit Fortschritt, Abbrechen, Herunterladen bis zur Frist und Hinweis, wenn eine Minute lang kein Worker abholt. Die Oberfläche fragt den Stand alle 1,5 s über `transfers/status` ab (nur App-Datenbank) — und nur, solange ein Auftrag läuft. `?schema=…&table=…` belegt den Export vor (`transfersHref` in `resources/js/lib/transfers.ts`) — die Adresse für den Knopf «Exportieren» im Objektkopf. Seite `workspace/transfers/Import`: Schritte, Datei mit «anders lesen», Zuordnung mit Beispielwerten und Hinweis auf fehlende Pflichtspalten, Probelauf mit Fehlerzeilen, Import.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
E-01	Export und Re-Import ergeben dieselben Daten je Treiber	`RoundTripTest`: SQL (ganze Datenbank: Struktur und Daten → leere Datenbank) und CSV (Tabelle) je Treiber; Werte mit Umlauten, Emoji, `'`, `"`, `\`, `;`, Zeilenumbrüchen, `$$`, Text wie SQL, leerem Text gegen NULL, Nullbyte-Binärdaten, `0.30000000000000004`, `-1.0E-12`, Wahrheitswerten, JSON; Fremdschlüssel auf die eigene Tabelle, zusammengesetzter Schlüssel, Tabelle ohne Schlüssel, Sicht; nach dem Import zählt der Schlüssel weiter (PostgreSQL `setval`)
E-01	1 Mio. Zeilen unter 64 MB	`LargeTransferTest` (Gruppe `transfer-large`, nur mit `PROBE_GROSS=1`) — Zahlen unten
E-02	Tests mit Fehlerzeilen; Probelauf schreibt nichts	`CsvImporterTest` je Treiber: vier Fehlerzeilen mit Nummer 5–8 (auch nach einem Feld über zwei Zeilen) und Grund, Tabelle danach leer; überspringen, nichts importieren, Werte wie SQL wörtlich; `SqlImporterTest`: Probelauf führt nichts aus, Fehler nennt Anweisung und Zeile, PostgreSQL/SQLite rollen zurück; `ImportControllerTest`: 403 für schreibgeschützte Verbindung und Lesemitglied, Name der Verbindung auf Produktion, Zuordnung mit `id; drop table …` abgewiesen
E-06	Nach Import ist die Schätzung der Zeilenzahl aktuell	`StatisticsTest`: `Dialect::tables()` → `rows` = 500 ± 10 % auf MySQL, MariaDB, PostgreSQL; SQLite (keine Schätzung): `sqlite_stat1` = 500. Mutationsprobe ohne `ANALYZE`: MySQL (0), PostgreSQL (null), SQLite rot — **MariaDB bleibt grün**, InnoDB schätzt dort auch ohne
B-07	Test beider Wege	`TransferRunnerTest`: mit Worker in die Warteschlange (Auftrag trägt nur die Kennung) und vom Job ausgeführt; ohne Worker in der Anfrage; Passwort nur in der Sitzung → in der Anfrage trotz Worker, Grenze mit Erklärung; Job ohne gespeichertes Passwort scheitert mit Erklärung; Abbrechen während des Lesens; `TransferControllerTest`: Grenze ohne Worker, Warteschlange mit Worker

Messwerte

Eine Million Zeilen (`id`, Zahl, Text, Dezimalzahl, Zeitpunkt), je Treiber in einem eigenen Prozess gemessen (`LargeTransferTest`, 29.10.2026, MacBook mit Herd: MySQL 8.4, MariaDB 10.11, PostgreSQL 18, SQLite 3.45). Speicher: Spitze des PHP-Speichers über dem Stand davor; Prozess: Zuwachs von `ru_maxrss` während des Schritts (sieht auch libpq, F-8).

	MySQL	MariaDB	PostgreSQL	SQLite
Export CSV (49 MB)	5,0 s · PHP 4,1 MB · Prozess +4,4 MB	5,6 s · 4,1 MB · +4,5 MB	4,9 s · 3,8 MB · +4,5 MB	6,9 s · 3,6 MB · +5,8 MB
Export SQL, nur Daten (61 MB)	3,2 s · 3,2 MB · +0 MB	4,4 s · 3,2 MB · +0 MB	3,5 s · 3,0 MB · +0,2 MB	2,3 s · 2,8 MB · +0,1 MB
Import CSV (Pakete zu 1'000, mit ANALYZE)	9,0 s · 2,0 MB	6,1 s · 2,0 MB	9,2 s · 2,3 MB	3,6 s · 1,7 MB
Import SQL (10'000 INSERT zu 100 Zeilen)	11,3 s · 2,1 MB	5,9 s · 2,1 MB	16,5 s · 2,1 MB	2,8 s · 2,1 MB

Ergebnis: Abnahme E-01 (unter 64 MB) mit Abstand erfüllt — höchstens 4,1 MB PHP-Speicher und 5,8 MB Prozesswachstum; der Prozess wächst auch bei PostgreSQL nicht, weil Pakete über den Schlüssel libpq nie das ganze Ergebnis halten lassen. Der erste gemessene Schritt je Prozess trägt das Laden der Klassen mit.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
11 Export	eigene Seite je Verbindung statt Dialog über dem Raster; «gefilterte Zeilen» fehlt	der Knopf im Objektkopf und das Raster gehören dem Hauptstrang; er verlinkt mit `?schema=…&table=…` hierher
11 Export	ohne «noch etwa 20 s»	Fortschritt in Prozent und Zeilen; eine Restzeit wäre bei geschätzten Zeilenzahlen geraten
11 Import	Zuordnung nur nach gleichem Namen (nicht «Vorname» → `first_name`); kein Datumsformat «TT.MM.JJJJ»	Daten gehen wörtlich an die Datenbank; Formatumwandlung wäre eine eigene Anforderung
11 Import	Fehlerzeilen nicht als CSV zum Herunterladen	offen, siehe unten

Tests

Neu: `RoundTripTest` (16), `CsvImporterTest` (14), `SqlImporterTest` (10), `StatisticsTest` (4), `TableReaderTest` (8), `TransferRunnerTest` (8), `TransferTest` (1, Frist), `SqlScriptTest` (9, Zerleger), `TransferControllerTest` (11), `ImportControllerTest` (9), `LargeTransferTest` (4, nur auf Wunsch). Je Treiber gegen die Herd-Dienste: MySQL 12, MariaDB 14, PostgreSQL 13, SQLite 13 Fälle plus die HTTP-Tests auf SQLite; zweites Konto für fremde Aufträge (404) und Lesemitglied (403).

Stand der Suite: 480 Tests, 464 bestanden, 16 übersprungen (die schweren Proben mit 1 Mio. Zeilen, bewusst nur auf Wunsch); Pint, Larastan Stufe 7, `vp check` und `vue-tsc` ohne Befund.

Commits

`d466d6c` englische Zeilen des Rasters · `dd0dd7e` Scheibe 1: Server · `c7cf477` Scheibe 2: Oberfläche · Messtest und Protokoll (dieser Stand).

Offen

- Export einer **Abfrage** (mit dem SQL-Editor) und der **gefilterten Zeilen** des Rasters (Knopf «Exportieren» im Objektkopf).
- Fehlerzeilen eines Imports als CSV zum Korrigieren herunterladen (Wireframe 11).
- CSV-Export ohne Schutz vor Formeln (`=`, `+`, `-`, `@` am Anfang öffnet Excel als Formel) — bewusst, weil er die Daten verändern würde; Option für TB 08 (Sicherheitsprüfung).
- MySQL mit `sql_mode` `ANSI_QUOTES` oder `NO_BACKSLASH_ESCAPES`: der Zerleger erkennt das nicht.

Nach dem Zusammenführen

Sichtprüfung bei 1440 und 375 px (MariaDB, Umgebung Test): Knopf «Exportieren» im Objektkopf belegt Tabelle und Schema vor, CSV-Export von 240 Zeilen fertig mit Herunterladen, Abbrechen, Statistik aktualisieren mit Meldung. **Befund:** Laravels Vorgabe `QUEUE_CONNECTION=database` ohne laufenden Worker liess jeden Export für immer «wartet» stehen — `hasWorker()` sah nur die Konfiguration. Behoben mit dem Lebenszeichen des Workers (siehe oben, Test `test_queue_counts_as_worked_only_while_a_worker_reports_in`). Der gleiche deutsche Schlüssel «Aktualisieren» bedeutete zweierlei (Prozessliste: *Refresh*, Statistik: *Update*) — der Statistik-Knopf heisst jetzt «Statistik aktualisieren».

Gelernt

- Laravel bindet Kommazahlen als Text mit 14 Stellen (`0.30000000000000004` → `0.3`), und `Connection::escape` tut dasselbe — für verlustfreie Werte `var_export` bzw. Werte aus der Datei als Text (Lektion im Vault).
- `Model::query()->value('status')` liefert mit Enum-Cast das Enum, nicht den Text; der Abbruch-Merker sah den Abbruch deshalb nie — erst ein Test mit echtem Abbruch zeigte es (`toBase()->value()`).
- MySQL 8 liefert Spalten aus `information_schema` in Grossbuchstaben (`ENGINE`) — immer mit Alias lesen.
- `pdf-bauen.py --help` kennt kein `--help` und baut alle sieben Dossiers neu (Lektion im Vault).

Anhang T — TB 07 — Konten, Rechte je Verbindung, Teilen, Protokoll

Stand: 29.10.2026 · ✅ gebaut auf einem eigenen Zweig (Basis `80fed1d`), noch nicht zusammengeführt — vorgezogen vor TB 04, parallel zu Raster und Export. Dazu V-06 (Favoriten, «zuletzt geöffnet») aus TB 01.

Ziel

Mehrere Personen arbeiten sicher auf einer Installation: Rechte je Verbindung, Teilen durch die Verwaltung, Konten verwalten im Browser und ein Protokoll, in dem nie ein Wert aus einer Zieldatenbank steht.

Umfang

Anforderung	Inhalt	Ergebnis
B-04 (Must)	Protokoll mit Verbindung, Ziel, SQL, Zeilen; Maskierung; Frist	✅ Dienst und Ansicht; schreibende Aktionen auf Zielverbindungen gibt es erst ab TB 04 — sie rufen `AuditLog::record()` mit `connection`, `target`, `sql`, `bindings`, `affectedRows`, `error`
A-07	Rollen und Rechte je Verbindung	✅ `DatabaseConnectionPolicy` (`view`, `write`, `manage`, `share`)
V-09	persönlich oder geteilt	✅ Dialog «Teilen …» und Rechte je Konto in der Verwaltung
A-05, A-06	Name und Adresse durch die Verwaltung; Einladung im Browser	✅
V-06	Favoriten, sortieren, «zuletzt geöffnet»	✅ (Suche und Gruppen gab es schon)

Wireframe-Bereiche 14 (Verwaltung: Konten, Einladungen, Protokoll) und 03 (Verbindungen: Favoriten, Sortierung).

Was entstanden ist

Rechte (A-07). Die Policy beantwortet vier Fähigkeiten; die Rechte bauen aufeinander auf (lesen ⊂ schreiben ⊂ verwalten):

Fähigkeit	erlaubt	wer
`view`	Daten und Struktur ansehen, SQL lesend, exportieren	jedes Recht
`write`	Daten und Struktur ändern, importieren, Schreibschutz auf Zeit aufheben (V-11)	«schreiben» und «verwalten»
`manage`	Verbindung bearbeiten, mit gespeichertem Passwort testen, entfernen	«verwalten»
`share`	Teilen und Rechte vergeben	nur Administratoren

Besitzer haben an ihren Verbindungen «verwalten», Administratoren an allen **geteilten**. Persönliche Verbindungen anderer sieht niemand, auch kein Administrator — sie bleiben 404 (Routenbindung, T-16); sichtbare ohne Recht antworten 403. `DatabaseConnection::permissionFor()` ist die eine Regel, `scopeVisibleTo` und `isManageableBy` folgen ihr.

Teilen (V-09). In der Übersicht öffnet «Teilen …» (nur Administratoren) einen Dialog: Teilen ein/aus und je Konto «kein Zugriff · lesen · schreiben · verwalten». Ausschalten entzieht allen ausser dem Besitzer den Zugriff. Besitzer und Administratoren bekommen kein gespeichertes Recht. Karten zeigen «geteilt» und das eigene Recht.

Verwaltung (Wireframe 14). `/admin/accounts`, `/admin/invitations`, `/admin/audit`, nur Administratoren (Gate `administrate`, sonst 403), Einstieg im Kontomenü. Konten mit Rolle, letzter Anmeldung, Passkey; je Konto Name und Adresse ändern (A-05), Rolle, Sperren, Entfernen und «Rechte je Verbindung». `UserPolicy`: Rolle, Sperre und Entfernen nie am eigenen und nie am letzten aktiven Administratorkonto. **Gesperrt** (neue Spalte `users.disabled_at`): kein Code, kein Passkey-Zutritt, laufende Sitzungen enden mit der nächsten Anfrage (`EnsureAccountIsActive`), Sitzungen in der Datenbank werden gelöscht. **Entfernen:** Passkeys, persönliche Verbindungen und Rechte gehen mit; geteilte Verbindungen des Kontos übernimmt der Administrator. **Rolle verloren:** wer nicht mehr Administrator ist, startet ohne Rechte an fremden Verbindungen. **Einladungen** im Browser mit Ablauf, erneut senden, zurückziehen; nach dem Senden erscheint der Link einmal (für Installationen ohne Mail). `phpdbadmin:invite` nutzt denselben `InvitationSender`.

Protokoll (B-04). `audit_log` hat neu `connection_id` (ohne Fremdschlüssel — ein Eintrag überlebt seine Verbindung) sowie `user_name` und `connection_name` (lesbar nach dem Entfernen, Datenmodell § 4). Ansicht mit Filtern nach Konto, Verbindung, Art und Zeitraum, Suche in Anweisung und Ziel, Seiten zu 50, Export als CSV (Zellen mit `= + - @` entschärft; der Export steht selbst im Protokoll).

Maskierung (T-11). `App\Services\Audit\Masking`:

- `Masking::sql($sql, $bindings, $doubleQuotedStrings)` — Zeichenketten- und Zahl-Literale werden zu `?`; Bezeichner (`` `…` ``, `"…"`), Platzhalter (`?`, `$1`, `:name`) und Kommentare bleiben; `"…"` gilt bei MySQL und MariaDB als Zeichenkette.
- `Masking::message($message, $bindings)` — eingesetzte Bindungen (an Wortgrenzen, «1» trifft nicht «1062»), `'…'`, «near '…' at line», PostgreSQL «Key (…)=(…)», «Failing row contains (…)», «invalid input syntax … "…"».
- `Masking::exception($throwable)` — Meldung des Servers plus Anweisung mit Platzhaltern.

`AuditLog` speichert Anweisung und Fehler nur maskiert; Datenbankfehler gehen maskiert ins Log (Reporter in `bootstrap/app.php`, ersetzt den üblichen Eintrag); Zielverbindungen setzen `mask_bindings_in_exception_messages`.

Aufbewahrung. `phpdbadmin.audit.retention_days` (`PHPDBADMIN_AUDIT_RETENTION_DAYS`, Vorgabe 365, 0 = ohne Frist; Datenmodell § 5 entschieden: Konfiguration, angezeigt in der Verwaltung), `AuditEntry` ist `Prunable`, `model:prune` täglich im Scheduler (braucht den Cron-Eintrag für `schedule:run`).

Favoriten und «zuletzt geöffnet» (V-06). Stern je Karte, Filter «Favoriten», Favoriten oben; Sortierung umschaltbar (nach Gruppe dann zuletzt geöffnet · nach Name · zuletzt geöffnet zuerst), im Browser gemerkt. `WorkspaceController::show` setzt `last_opened_at`. Die Merker liegen in `connection_user`; Besitzer und Administratoren bekommen dafür eine Zeile mit ihrem wirksamen Recht «verwalten» — eine Merker-Zeile gibt so nie mehr Recht, als das Konto ohnehin hat.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
A-07	Policy-Tests je Recht; fremde Verbindung 404 statt 403	`DatabaseConnectionPolicyTest` (10 Fälle: Rolle × Besitz × Teilen × Recht, dazu Sichtbarkeit = Policy), `ConnectionControllerTest` (403 für «lesen» auch vor der Validierung, 200 für «verwalten», Administrator 404 an persönlicher Verbindung)
V-09	persönliche Verbindung ist für andere 404	`SharingControllerTest` (9): Teilen, «kein Zugriff», Teilen beenden → 404 für Mitglied und Administrator, Besitzer weiter 200; nur Administratoren (403), Validierung
B-04	jede schreibende Aktion genau ein Eintrag; Bereinigung nach Frist; ein gebundener Wert erscheint in keiner Meldung	`AuditLogTest` (7): echte Fehler auf **MySQL, MariaDB, PostgreSQL, SQLite** (doppelter Schlüssel, falscher Zahlwert) — der Wert steht in keinem Feld des Eintrags; maskiertes Log; Frist 30 Tage und 0; `model:prune` täglich. `MaskingTest` (5 Tests, 14 Fälle). «Genau ein Eintrag je schreibender Aktion» prüfen die Aktionen ab TB 04
A-05	Name und E-Mail nur durch die Verwaltung	`AccountControllerTest` (13), `UserPolicyTest` (6 Fälle)
A-06	Einladung mit Ablauf	`Admin\InvitationControllerTest` (6), Rückzug macht den Link ungültig
V-06	Durchlauf	`FavoriteControllerTest` (6), `WorkspaceLastOpenedTest` (2); Browser-Durchlauf der Übersicht steht aus (siehe unten)

Gesperrte Konten: `LoginControllerTest`, `PasskeyLoginControllerTest`, `EnsureAccountIsActiveTest`.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
14 Legende «verwalten»	«zusätzlich Verbindung bearbeiten und entfernen» statt «… und mit anderen teilen»	Teilen ist nach V-09 und CR Sache der Administratoren
14 Konten	offene Einladungen stehen im Reiter «Einladungen», nicht zusätzlich in der Kontenliste; «Erneut senden» und «Zurückziehen» dort	eine Liste je Sache; der Reiter zählt die offenen
14 Konten	Rechte-Tabelle zeigt geteilte Verbindungen und die eigenen persönlichen des Administrators, keine persönlichen des gewählten Kontos	persönliche Verbindungen anderer sieht niemand (V-09)
14 Konten	«Entfernen» verlangt den Namen zur Bestätigung	G-08: zerstörende Handlung
14 Protokoll	«ältere laden» als Seiten (neuere · ältere)	einfache Seiten mit Filtern in der Adresse
14 Protokoll	Ergebnis «abgelehnt» für Fehlversuche, «Fehler» bei gescheiterten Aktionen; «schreibgeschützt» erst mit TB 04	es gibt noch keine schreibenden Aktionen
03 Verbindungen	Seitenleiste «Zuletzt geöffnet» (Tabellen, Abfragen) fehlt; die Karten zeigen «zuletzt geöffnet» je Verbindung	Verlauf je Objekt kommt mit Q-04 (TB 03)
03 Verbindungen	Favoriten oben in jedem Abschnitt, nicht in einem eigenen Abschnitt	wie im Wireframe (Stern an der Karte); Filter «Favoriten» zeigt nur sie

Die Wireframes 03 und 14 sind im Generator noch nicht nachgezogen.

Tests

Neu: 11 Testdateien mit 60 Tests (davon zwei Matrizen mit 16 Fällen); erweitert: `ConnectionControllerTest`, `LoginControllerTest`, `PasskeyLoginControllerTest`. Die Maskierung ist gegen echte Fehlermeldungen aller vier Treiber geprüft. Die neuen Tests laufen grün mit SQLite, MariaDB, MySQL und PostgreSQL als App-Datenbank; die Migrationen laufen auf allen vier. Zweites Konto für Rechte: in jedem Endpunkt-Test (Mitglied 403, fremde Verbindung 404, Gast zur Anmeldung).

Gesamtlauf zuletzt: 476 Tests, 464 bestanden, 12 übersprungen (die schweren Proben).

Entscheide in der Timebox

Entscheid	Begründung
Administratoren sehen alle geteilten, aber keine persönlichen Verbindungen anderer	«Administrator darf alles» ohne Einblick in persönliche Zugänge; Datenmodell § 2.2
Merker-Zeilen für Besitzer und Administratoren mit Recht «verwalten» statt `permission` leer erlauben	keine Spaltenänderung (der Migrations-Wächter lehnt `change()` ab); die Zeile gibt nie mehr Recht als die Rolle; beim Rollenverlust werden die Zeilen an fremden Verbindungen gelöscht
`connection_id` im Protokoll ohne Fremdschlüssel, mit Namen von Konto und Verbindung	Einträge überleben Konto und Verbindung (Datenmodell § 4)
Neue Migrationen ohne `down()`	Protokoll- und Kontodaten sollen beim Zurückrollen nicht still verschwinden; der Migrations-Wächter weist `dropColumn` ab
Einladungslink einmal auf der Seite	wie der Befehl; der Link steht nur bis zur nächsten Anfrage in der Sitzung
Datenbankfehler maskiert ins Log statt des üblichen Eintrags	Datenmodell § 1.9 und Solution-Architektur § 13: keine Werte aus Zieldatenbanken im Log

Commits

`4322f65` fehlende englische Zeilen der Basis · `f720d71` Scheibe 1 Policy · `863b47b` Scheibe 2 Teilen · `3359df0` Scheibe 3 Verwaltung · `6d2f991` Scheibe 4 Protokoll · `f7eac95` Scheibe 5 Favoriten · Doku.

Offen

- **Browser-Durchlauf** nach dem Zusammenführen (Hauptstrang): Übersicht mit Favoriten und Sortierung, Protokoll mit echten Einträgen, bei 1440 und 375 px, hell und dunkel. Teilen, Konten und Einladungen sind vor dem Zusammenführen im Workspace `tb07-rechte` durchgespielt.
- **Fehlermeldungen im Raster** (TB 02): `TableData` sollte Meldungen der Server mit `Masking::exception()` zeigen; Zielverbindungen maskieren schon die eingesetzten Bindungen, nicht aber die Werte in den Meldungen der Server.
- **Schreibende Aktionen** (TB 04 ff.) protokollieren mit `AuditLog::record(..., connection:, target:, sql:, bindings:, affectedRows:, error:)` und prüfen mit `write`/`manage` der Policy.
- Fristen für Anmeldecodes und Einladungen (`model:prune`, Datenmodell § 4) — der tägliche Lauf steht, die Modelle sind noch nicht `Prunable`.
- Wireframes 03 und 14 im Generator nachziehen.

Gelernt

- Der Migrations-Wächter (Plugin) weist `dropColumn` und `change()` ab — Schemaänderungen additiv planen (Spalte dazu, keine Typänderung).
- Merker und Rechte in derselben Tabelle: Eine Merker-Zeile darf nie ein Recht geben, das die Rolle nicht schon gibt — sonst bleibt beim Rollenwechsel still ein Zugriff zurück.
- `mask_bindings_in_exception_messages` maskiert nur Laravels eingesetzte Anweisung; MySQL und PostgreSQL zitieren Werte auch in ihrer eigenen Meldung («Duplicate entry '…'», «Key (…)=(…)»).

Anhang U — Timebox 08 — Sicherheit, Barrierefreiheit, Mobil, Hilfe, Installation → v1.0

Stand: 29.10.2026 · ✅ abgeschlossen bis auf den axe-Lauf (siehe O-08) — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen.

Ziel

phpdbadmin so weit bringen, dass jemand es installieren und sicher betreiben kann: geprüft gegen das Bedrohungsmodell, bedienbar mit Tastatur und auf dem Telefon, mit kurzer Hilfe und einer Installation in einem Befehl.

Umfang

Anforderung	Plan	Ergebnis
N-02 Sicherheit	**Must**	✅ Prüfbericht nach OWASP, ein hoher Befund gefunden und behoben (Bericht)
O-07 Tablet und Telefon	Should	✅ Durchlauf bei 375, 768 und 1440 px, ein Befund behoben
O-08 WCAG 2.2 AA	Should	◐ Ersatzprüfung ohne axe (siehe unten)
N-05 kurze Hilfe	Should	✅ Seite «Hilfe», Verweise aus der Oberfläche, Test über alle Anker
B-05 Installation	Should	✅ `composer setup` in einem leeren Ordner: 11 s mit warmen Caches, Anmeldung per Code geprüft

Was entstanden ist

Sicherheit (N-02). Ein eigener, nur lesender Prüfdurchgang über Code, Tests und Proben gegen die lokalen Server. Er fand eine DSN-Injektion über den Datenbanknamen: `postgres' host='10.0.0.5` verband an den erlaubten Hosts vorbei — jetzt am Feld und bei jedem Verbinden abgewiesen. Dazu: App-Datenbank auch auf einem Server nie als Ziel, Aufträge prüfen Recht und Schreibschutz beim Ausführen erneut, Drosselung der Code-Prüfung je Adresse, CSP mit Nonce und Sicherheits-Header, keine `storage`-Route, Einladungen melden nie in bestehende Konten an, CSRF-Test mit echter Middleware, Passwörter im Verlauf maskiert, Leser nur mit lesender Sitzung. Alles im Bericht mit Test je Befund.

Mobil (O-07). Durchlauf bei 375 px (Verbindungen, Raster als Karten, Struktur, SQL-Editor, Server, Verwaltung, Export), 768 px (Raster, Export) und 1440 px. Befund: bei 768 px quetschte das Suchfeld im Kopf den Verbindungs-Chip auf einen Buchstaben — der Chip schrumpft nicht mehr, das Suchfeld schon. Bearbeiten bleibt auf dem Telefon möglich, ist aber für den Desktop gebaut (O-07: «Bearbeiten am Desktop»).

Barrierefreiheit (O-08). Ersatzprüfung, weil axe hier nicht laufen kann (siehe unten): Kontraste aller Farbpaare in Hell und Dunkel über `werkzeuge/kontraste.py` (alle ≥ 4,5:1, Fokus ≥ 3:1); jeder Knopf und Link ohne sichtbaren Text hat einen Namen (Skript über alle Vue-Dateien, zwei Fundstellen geprüft: beide über Inhalt bzw. `<label>` benannt); Bedienbaumstruktur im Browser gelesen; neu: Sprunglink «Zum Inhalt springen» als erster Tab-Halt, `main` als Ziel; `lang` am Dokument je Konto. **Offen:** axe selbst — er braucht

ein Skript im Browser, und jeder Skriptaufruf löst bei Hoang einen Bestätigungsdialog aus. Vorschlag: einmal von Hand mit der axe-Erweiterung über Anmeldung, Verbindungen, Raster, Zeilen-Detail, Vorschau, SQL-Editor, Export.

Hilfe (N-05). Seite `help` aus `resources/help/de.md` und `en.md` (Markdown, HTML darin wird entfernt), elf Abschnitte mit festen Ankern. Einstiege: Kontomenü, Befehlspalette, Kürzel-Dialog; Verweise im Freigabe-Dialog, in der SQL-Vorschau, in der Server-Verwaltung und in der Verwaltung. «Doku-Prüfung im Commit»: `HelpControllerTest` prüft, dass jeder `<HelpLink anchor>` in beiden Sprachen einen Abschnitt trifft und beide Sprachen dieselben Abschnitte haben.

Installation (B-05). `composer setup` (Abhängigkeiten, `.env`, Oberfläche bauen, `phpdbadmin:install`) und `composer upgrade` nach `git pull`. `phpdbadmin:install` erzeugt einen fehlenden `APP_KEY` (einen vorhandenen nie), migriert, zeigt die Systemprüfung und legt das erste Konto mit Einrichtungslink an; ohne Terminal nennt es die fehlenden Optionen. `phpdbadmin:check` meldet Hindernisse (PHP, Erweiterungen, Schlüssel, Debug in Produktion, App-Datenbank, Oberfläche, Schreibrechte) und Betriebslücken (Mail im Log, kein Worker, kein Scheduler, HTTPS für Passkeys, Cookie, App-Datenbank auf erlaubtem Server). README mit Voraussetzungen, Installation, Betrieb (Cron, Worker), Aktualisieren; LICENSE (MIT). Durchlauf: `git clone` in einen leeren Ordner, `composer setup`, Konto angelegt, mit `php artisan serve` angemeldet (Code aus dem Log), Ersteinrichtung bis zur Verbindungsübersicht — **11 Sekunden** mit warmen Caches für Composer und npm; ohne Caches schätzungsweise 2–5 Minuten (Download), weit unter 15.

Nebenbei behoben.

- Laravels Vorgabe `QUEUE_CONNECTION=database` ohne laufenden Worker liess jeden Export für immer «wartet» stehen — der Worker meldet sich jetzt mit einem Lebenszeichen (TB 06, `QueueHeartbeat`).
- `trans_choice()` fiel für deutsche Schlüssel auf Englisch zurück (Laravel wählt die Mehrzahlregel der Sprache, in der es den Schlüssel findet; `de.json` gibt es nicht) — `SourceLocaleTranslator` behandelt Deutsch als Quellsprache.

Abnahme v1.0 (Timebox-Plan § 3)

Kriterium	Stand
Kernabläufe (Bearbeiten, Struktur, Export, Import) auf vier Treibern getestet, inklusive Elterntabelle mit Kindern	✅ TB 04 (`RowEditorTest`), TB 05 (Elterntabelle mit Kindern je Treiber), TB 06 (Rundlauf je Treiber)
White-Box-Sicherheitsprüfung nach OWASP ohne hohen Befund	✅ nach Behebung von H-01 (`cab3ec5`)
axe ohne Befund in den Kernbildschirmen	◐ Ersatzprüfung; axe-Lauf offen (Hoang, von Hand)
Installation in einem leeren Ordner unter 15 Minuten	✅ 11 s mit Caches

Tests

Neu: `CheckCommandTest` (4), `InstallCommandTest` (+2), `HelpControllerTest` (4), `SecurityHeadersTest` (2), `RequestForgeryTest` (3), `SourceLocaleTranslatorTest` (2), dazu Fälle in `HostPolicyTest` (+9), `TestConnectionControllerTest` (+2), `TransferRunnerTest` (+3), `LoginControllerTest` (+1), `InvitationControllerTest` (+1), `QueryRunnerTest` (+1), `MaskingTest` (+5), `SqlControllerTest` (+1). Suite: 996 Tests, 976 bestanden, 20 übersprungen (schwere Proben).

Commits

`b43bf31` Lebenszeichen des Workers · `6ed5766`, `52d6802` Installation und Systemprüfung · `fbfcbd8` Hilfe · `58f2f72` Sprunglink, Kopf auf dem Tablet · `cab3ec5` Befunde der Sicherheitsprüfung.

Gelernt

- Ein Schutzmuster, das nicht kompiliert, lässt alles durch: `preg_match` gibt `false`, und `!== 1` liest das als «kein Treffer». Prüfungen fail-closed schreiben (`=== 0`) — im Vault: `regex-pruefung-scheitert-offen`.
- Konfiguration ist kein Lebenszeichen: Eine Warteschlange, die konfiguriert ist, hat noch keinen Worker — im Vault: `warteschlange-ohne-worker-wartet-ewig`.
- Herd fügt lokalen Seiten ein Inline-Skript ohne Nonce hinzu; eine strikte CSP sperrt es. Lokal deshalb `Report-Only`, überall sonst durchgesetzt.

Anhang V — Timebox 09 — Server-Verwaltung

Stand: 29.10.2026 · ✅ gebaut und getestet in drei Scheiben auf einem eigenen Zweig (Arbeitskopie), vorgezogen vor v1.0 — Browser-Durchlauf und Zusammenführen im Hauptstrang offen.

Ziel

Den Server hinter einer Verbindung ansehen und geführt verwalten — Übersicht, Prozesse, Variablen, Konten und Rechte, Wartung —, mit derselben Bedienung für MySQL, MariaDB, PostgreSQL und SQLite und ohne dass etwas ungesehen passiert.

Umfang

Nr.	Anforderung (CR-001 § 4.8)	Prio	Ergebnis
W-01	Server-Übersicht: Version, Laufzeit, Grösse je Datenbank, Zeichensätze	S	✅
W-02	Prozesse und Sitzungen anzeigen und beenden	S	✅
W-03	Variablen und Status lesen	C	✅ (Durchlauf offen)
W-04	Benutzer, Rollen und Rechte geführt verwalten mit Vorschau	S	✅
W-05	Wartung: ANALYZE, VACUUM, OPTIMIZE, Integritätsprüfung	C	✅ (Durchlauf offen)

Kein eigenes Wireframe (Timebox-Plan: «in TB 09 entworfen»); gestaltet im Stil der Wireframes 05 (Arbeitsbildschirm), 06 (SQL-Vorschau) und 08 (Struktur).

Was entstanden ist

Dialekt (ADR-001). Der Vertrag kennt jetzt `serverOverview`, `sessionId`, `processes`, `endProcessStatement` / `endProcess`, `variables`, `statusValues`, `accounts`, `currentAccount`, `privilegeChoices`, `createAccountStatements`, `dropAccountStatements`, `privilegeStatements`, `maintenanceOperations`, `maintenanceForTables`, `maintenanceStatements` und `runMaintenance` — umgesetzt in allen vier Dialekten; SQLite gibt `null` bzw. leere Listen, wo es Prozesse und Konten nicht gibt. Was ändert, plant der Dialekt nur als `PlannedStatement` (echter Text und Text für Vorschau und Protokoll, Passwort maskiert).

Ändernde Aktionen. `TargetConnections::withWritable()` öffnet für genau eine Aktion eine Verbindung ohne Schreibschutz — nie für eine schreibgeschützte (`ReadOnlyConnection`), getrennt von den Leseverbindungen und danach sofort freigegeben. `ServerPlans` plant aus geprüften Eingaben; jeder Name eines Ziels muss im Katalog stehen. `ServerChanges` zeigt die Vorschau über die Leseverbindung und führt über die beschreibbare aus, Anweisung für Anweisung, ohne Abfrage-Ereignisse (dort stünde das Passwort); jede Aktion ergibt genau einen Eintrag im Protokoll — `server.kill`, `server.user.create`, `server.user.drop`, `server.grant`, `server.revoke`, `server.maintenance` —, gelungen oder gescheitert, mit SQL ohne Geheimnisse. `AuditLog::record()` nimmt dafür das SQL auf.

Seiten. Unter `c/{connection}/server` fünf Inertia-Seiten im Arbeitsbildschirm (`workspace/server/Overview`, `Processes`, `Variables`, `Accounts`, `Maintenance`), Einstieg «Server» im Menü der Verbindung:

Methode	Adresse	Name
GET	`server`	`workspace.server`
GET	`server/processes`	`workspace.server.processes`
POST	`server/processes/{process}/end`	`workspace.server.processes.end`
GET	`server/variables`	`workspace.server.variables`
GET	`server/accounts`	`workspace.server.accounts`
POST	`server/accounts/preview`	`workspace.server.accounts.preview` (JSON)
POST	`server/accounts`	`workspace.server.accounts.apply`
GET	`server/maintenance`	`workspace.server.maintenance`
POST	`server/maintenance/preview`	`workspace.server.maintenance.preview` (JSON)
POST	`server/maintenance`	`workspace.server.maintenance.run`

Abnahmekriterien

Anforderung	Kriterium	Nachweis
W-01	Test je Treiber	Vertrag: Version, Laufzeit (SQLite keine), Zeichensatz, Grösse der Test-Datenbank > 0, PostgreSQL zusätzlich je Schema, SQLite Seitengrösse, Journal und Datei — vier Treiber; Seite je Server-Treiber und SQLite
W-02	beendete Sitzung ist weg	Vertrag je Server-Treiber: zweite Sitzung öffnen, Kennung über `connection_id()` bzw. `pg_backend_pid()`, beenden — danach nicht mehr in der Liste und tot (eigene PDO-Verbindung scheitert), ein zweites Beenden meldet «gibt es nicht»; Abbrechen stoppt `sleep` in einer asynchronen Sitzung (mysqli, pgsql), die Sitzung bleibt. Endpunkt je Server-Treiber: Weiterleitung, Meldung, genau ein Eintrag `server.kill`. Die eigene Sitzung ist markiert und abgewiesen
W-03	Durchlauf	Vertrag vier Treiber, Seite für SQLite und PostgreSQL; Browser-Durchlauf offen
W-04	Tests je Treiber (ohne SQLite)	Vertrag je Server-Treiber: Konto mit einem Passwort, das wie SQL aussieht, anlegen, mit diesem Passwort anmelden, Recht auf eine Tabelle erteilen, mit `SHOW GRANTS` bzw. `has_table_privilege` und mit einem echten Zugriff prüfen, entziehen, löschen; Rechte auf Server, Datenbank und (PostgreSQL) Schema erscheinen in der Liste und verschwinden wieder. Endpunkt je Server-Treiber: Vorschau maskiert und ändert nichts, Lebenszyklus mit vier Einträgen im Protokoll ohne Passwort
W-05	Durchlauf	Vertrag je Treiber und Art (11 Fälle) für eine Tabelle und das ganze Schema ohne Fehler; SQLite erkennt einen beschädigten Index; Endpunkt mit Ergebnis je Tabelle, VACUUM auf SQLite trotz ATTACH-Wächter; Browser-Durchlauf offen

Querschnitt: ohne «verwalten» 403, schreibgeschützt 403 (auch für den Besitzer), fremde Verbindung 404, Produktion erst mit dem eingetippten Namen — für alle fünf ändernden Endpunkte, ohne Eintrag im Protokoll. Eingaben, die wie SQL aussehen (Kontoname, Host, Recht, Datenbank, Tabelle «*», Ebene, Prozesskennung), werden vor jeder Anweisung abgewiesen.

Gestaltungsentscheide (statt Wireframe-Abweichungen)

Bereich	Entscheid	Grund
Einstieg	Eintrag «Server» im Menü der Verbindung; Navigator unverändert, zeigt die Schemas ohne Auswahl	Navigator und Layout bearbeitet parallel jemand anderes

Bereich	Entscheid	Grund
Bereiche	Reiter wie in der Strukturansicht (Wireframe 08), brechen um statt zu scrollen	gleiche Bedienung wie im übrigen Arbeitsbildschirm
Bestätigung	Dialog im Stil der SQL-Vorschau (Wireframe 06): nummerierte Anweisungen, Hinweise je Treiber, auf Produktion Name der Verbindung, Fehler des Servers im Dialog	eine Bestätigung für Prozesse, Konten und Wartung
Konten	erst ein Formular (anlegen bzw. Ebene, Ziel, Rechte), dann die Vorschau; Löschen geht direkt zur Vorschau	nichts läuft ohne Vorschau, auch nicht «nur» ein REVOKE
Ebenen	MySQL/MariaDB: Server, Datenbank, Tabelle; PostgreSQL: Server (vordefinierte Rollen wie `pg_monitor`), Datenbank, Schema (USAGE/CREATE und Rechte auf allen Tabellen darin), Tabelle	PostgreSQL kennt keine Rechte auf «.»
Schmale Fenster	Prozesse, Konten und Variablen als Karten bzw. umbrechende Zeilen, keine breiten Tabellen	375 px lesbar (O-07)
Gesperrt	Knöpfe bleiben sichtbar, aber gesperrt, mit Grund (schreibgeschützt bzw. ohne «verwalten»)	man sieht, was ginge
SQLite	gleiche Reiter; Prozesse und Konten mit klarer Erklärung, Variablen = lesbare PRAGMAs, Status = Optionen der Bibliothek	gleiche Bildschirme für alle vier (O-04)

Entscheide in der Timebox

Entscheid	Begründung
Bezeichner je Segment selbst quoten (Anführungszeichen verdoppelt) statt `Grammar::wrap()`	`wrap()` trennt an Punkten und « as » und lässt «» *ungequotet — eine Tabelle* «» würde zu «alle Tabellen»
Zeichenketten (Passwort, Host, Kontoname bei MySQL) über `PDO::quote`, Rechte nur aus der festen Liste je Ebene	DDL wie `CREATE USER` nimmt keine Platzhalter
MySQL/MariaDB: `_` und `%` im Datenbanknamen eines GRANT maskiert	auf Datenbank-Ebene sind sie Platzhalter — «my_db» träfe auch «myXdb»
Vorschau als JSON (`useHttp`), Ausführen als Inertia-POST mit Weiterleitung und Meldung	dieselbe Planung für beide; die Seite lädt nach einer Änderung neu
Ein Eintrag im Protokoll auch für gescheiterte Versuche (`outcome: failed`, Meldung maskiert)	wer was versucht hat, gehört ins Protokoll
Meldungen des Servers aus `getPrevious()` der `QueryException`, Geheimnisse maskiert	Laravel hängt die Anweisung samt Werten an die Meldung
PostgreSQL meldet sich als `application_name = phpdbadmin`	eigene Sitzungen in `pg_stat_activity` erkennbar
PostgreSQL: `DROP ROLE` ohne `DROP OWNED`	`DROP OWNED` löscht Objekte; lieber ablehnen lassen und den Hinweis zeigen
Wartung mit eigenem Zeitlimit (`PHPDBADMIN_MAINTENANCE_TIMEOUT_MS`, Vorgabe 10 Minuten)	OPTIMIZE und VACUUM grosser Tabellen brauchen länger als eine Abfrage
SQLite-VACUUM: `ATTACH ''` im Authorizer erlaubt (Zeile aus dem Hauptstrang)	VACUUM hängt intern eine leere Datenbank an

Tests

126 neue Tests; Suite gesamt 449, davon 437 bestanden und 12 übersprungen (die schweren Proben auf Wunsch). Testläufe mit `PHPDBADMIN_TEST_DATABASE=phpdbadmin_test_server`; Konten heissen `phpdbadmin_w04_…` und werden im Test wieder entfernt (`finally`).

Datei	Tests	MySQL	MariaDB	PostgreSQL	SQLite
`Dialects/DialectServerContractTest`	46	12	15	11	8
`Http/Controllers/Workspace/Server/*` (fünf Dateien)	53	5	25	6	17
`Http/Requests/Workspace/Server/ServerChangeRequestTest`	18	—	—	—	18
`Services/Server/ServerPlansTest`	7	3	2	2	—
`Connections/TargetConnectionsTest` (neu: beschreibbare Verbindung)	2	—	—	—	2
zusammen	**126**	**20**	**42**	**19**	**45**

Prüfkette vor jedem Commit: `vp check`, `vue-tsc`, Pint, Larastan, PHPUnit — grün.

Commits

`5d54279` Scheibe 1: Dialekt für die Server-Verwaltung · `2272649` Scheibe 2: Grundlage für ändernde Aktionen · `87b5a20` Scheibe 3: Seiten und Endpunkte · Doku (dieser Commit).

Offen

- **Browser-Durchlauf** bei 1440, 768 und 375 px, hell und dunkel, gegen einen Herd-Workspace — Hauptstrang.
- **PostgreSQL schickt das Passwort im Klartext** (`CREATE ROLE … PASSWORD '…'`); mit `log_statement = ddl` steht es im Server-Log. Abhilfe wäre ein SCRAM-Verifier, im Client berechnet — PHP fehlt dafür SASLprep.
- **Rollen in MariaDB** werden gelistet, aber nicht gelöscht (`DROP ROLE`); MySQL-Rollen erscheinen als gesperrte Konten.
- **Anonyme Konten in MariaDB** (Herd: `''@'localhost'`) gehen `'app'@'%'` vor — ein neues Konto meldet sich lokal erst mit Host `localhost` an. Ein Hinweis in der Oberfläche fehlt.
- Integritätsprüfung für PostgreSQL (`amcheck`) nicht im Umfang; die Prozessliste aktualisiert sich nur auf Knopfdruck.
- Grosse Wartung kann an der Laufzeitgrenze von PHP scheitern, bevor das Zeitlimit des Servers greift.

Gelernt

- `Grammar::wrap()` ist kein Quoting für beliebige Namen (Punkte, « as », «*») — für Bezeichner aus Eingaben eigenes Quoting je Segment (Lektion für den Vault).
- MariaDB hängt an `SHOW GRANTS` den Passwort-Hash (`IDENTIFIED BY PASSWORD '*…'`) — vor der Anzeige entfernen.
- Inertia-Tests verlangen die Seitendatei: Endpunkte und Seiten gehören in dieselbe Scheibe.
- Pint (`php_unit_method_casing`) benennt private Hilfsmethoden, die mit «test» beginnen, in Testklassen in `test_…` um — Hilfsmethoden anders benennen.

Anhang W — Timebox 12 — Sitzungen verwalten, SQL-Hilfe mit KI → v1.2

Stand: 29.10.2026 · ✅ gebaut — vorgezogen vor TB 10 und 11, weil sie unabhängig ist; v1.2 gilt, sobald TB 10 und 11 zusammengeführt sind.

Ziel

Sehen und beenden, wo das eigene Konto angemeldet ist; im SQL-Editor auf Wunsch eine Anweisung aus einer Frage in Worten — ohne dass Daten das Haus verlassen.

Umfang

Anforderung	Plan	Ergebnis
A-08 Aktive Sitzungen sehen und beenden	Could	✅
Q-08 SQL-Hilfe mit KI — nur eingeschaltet, nur mit eigenem Schlüssel, nur das Schema	Could	✅

Was entstanden ist

Sitzungen (A-08). Karte «Sitzungen» im Konto: Browser und System (aus dem User-Agent, ohne Bibliothek), Adresse, zuletzt aktiv; die eigene Sitzung markiert. Fremde Sitzungen einzeln oder alle auf einmal beenden — eine beendete Sitzung ist eine gelöschte Zeile in `sessions`, der nächste Aufruf mit ihrem Cookie landet bei der Anmeldung (phpdbadmin kennt kein «angemeldet bleiben»). Der Browser sieht nie die Sitzungskennung, nur einen HMAC davon. Geht nur mit `SESSION_DRIVER=database` (Vorgabe); sonst zeigt das Konto keine Liste. Beides steht im Protokoll.

SQL-Hilfe mit KI (Q-08). Aus, bis die Installation `PHPDBADMIN_AI=true` setzt. Dann hinterlegt jedes Konto seinen eigenen Schlüssel von Anthropic (verschlüsselt mit `APP_KEY`, `$hidden`, im Browser nur die letzten vier Zeichen). Im SQL-Editor «KI fragen»: Frage in Worten → Vorschlag mit kurzer Erklärung → «In den Editor übernehmen» hängt ihn an; ausgeführt wird nichts von selbst. Gesendet werden **nur die Frage und ein Umriss des Schemas aus dem Katalog** (Tabellen, Spalten mit Typ, NULL, Primär- und Fremdschlüssel; zwei Abfragen je Treiber) — keine Datenzeile, kein Text aus dem Editor, keine Vorgabewerte. Zu grosse Schemas werden um ganze Tabellen gekürzt, und die Antwort sagt, wie viele fehlen. Jede Anfrage steht im Protokoll, ohne Frage und Antwort. Je Konto höchstens zehn Anfragen je Minute.

Technik: Claude Opus 5.5 (`PHPDBADMIN_AI_MODEL`, Vorgabe `claude-opus-5-5`, Aufwand `medium`) über die Messages API mit strukturierter Ausgabe (JSON-Schema `{sql, explanation}`) und `fallbacks: "default"` — eine abgelehnte Anfrage läuft serverseitig auf einem anderen Modell weiter; eine Ablehnung (`stop_reason: refusal`) wird als Meldung gezeigt. **Abweichung vom Standard:** Aufruf über Laravels Http-Client statt über das offizielle PHP-SDK von Anthropic, weil neue Abhängigkeiten eine Freigabe brauchen (AGENTS.md). Der Aufruf steckt in `App\Services\Ai\SqlAssistant` und lässt sich auf das SDK umstellen, sobald es freigegeben ist.

Abnahmekriterien

Anforderung	Kriterium (Timebox-Plan, PRL)	Nachweis
A-08	beendete Sitzung ist abgemeldet	`SessionControllerTest::test_an_ended_session_is_signed_out` (Sitzung eines «anderen Geräts» mit eigenem Cookie: erst angemeldet, nach dem Beenden zur Anmeldung umgeleitet; Gegenprobe: ohne Löschen bleibt sie angemeldet), dazu: nur eigene Sitzungen, alle anderen beenden lässt die aktuelle, fremde Sitzung 404, ohne Datenbank-Treiber keine Liste
Q-08	Test mit Attrappe: keine Datenzeilen im Auftrag	`SqlAssistControllerTest::test_sends_only_question_and_schema_never_rows` (Tabelle mit Erkennungswert in den Zeilen; der Auftrag enthält Spalten und Fremdschlüssel, nie den Wert; Gegenprobe: ein eingeschleustes `select *` lässt den Test scheitern), aus = 404 ohne Aufruf, ohne Schlüssel kein Aufruf, abgelehnter Schlüssel, Ablehnung und kaputte Antwort als Meldung, fremdes Schema abgewiesen, Schlüssel verschlüsselt und nie im Browser; `SchemaOutlineTest` je Treiber (4)

Tests

`SessionControllerTest` 5, `SqlAssistControllerTest` 6, `SchemaOutlineTest` 4 (vier Treiber).

Commits

`facc4cd` Sitzungen, KI-Hilfe, deutsche Mehrzahl.

Gelernt

- In Feature-Tests überleben Wächter, Sitzungsobjekt und Cookies die Anfrage; eine «beendete» Sitzung wirkte deshalb noch angemeldet. Für den Test «anderes Gerät» Wächter, Sitzungsobjekt und Cookies zwischen den Anfragen zurücksetzen (`asNextProcess()` im Test).
- Laravels Sitzungen sind seit 12 JSON-serialisiert (`session.serialization`) — eine von Hand angelegte Sitzung braucht `base64(json)`, nicht `serialize()`.